

BACKEND ENGINEERING · SYSTEM DESIGN

# INSIDE E-COMMERCE

## Volume 1

## Understanding E-commerce

Business Domain · Marketplace · Domain Modeling · Pricing · Order Lifecycle

Tiếng Việt · Bản đọc PDF offline · 23/09/2026

# Mục lục · Volume 1

- 1. 01 — The E-commerce Ecosystem
- 2. 02 — E-commerce Business Models
- 3. 03 — Domain Modeling
- 4. 04 — Product & Pricing Engineering
- 5. 05 — Order Lifecycle

Source Markdown và sơ đồ chỉnh sửa được nằm trong gói ZIP cùng Volume.

# INSIDE E-COMMERCE

## Volume 1 — Understanding E-commerce

### Chapter 01 — The E-commerce Ecosystem

**Câu hỏi dẫn đường:** Điều gì thực sự diễn ra ở backend khi một khách hàng nhấn **Mua ngay**?

## Lời mở đầu

Người mua mở website, tìm một chiếc điện thoại, thêm vào giỏ và nhấn mua. Với họ, đó chỉ là vài thao tác. Nhưng hệ thống phải trả lời hàng loạt câu hỏi: sản phẩm có tồn tại, phiên bản đang chọn có còn hàng, giá và voucher còn hiệu lực, đơn được tạo thành công chưa, tiền đã được ghi nhận chưa, kho có thể giao hàng không? Nếu bất kỳ bước nào thất bại, hệ thống phải làm gì?

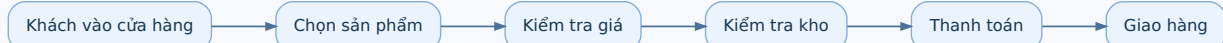
E-commerce không chỉ xử lý dữ liệu. Nó thực hiện những cam kết giữa người mua, người bán, đơn vị thanh toán và đơn vị vận chuyển. Cuốn sách bắt đầu từ **ng nghiệp vụ và tính đúng đắn**, rồi mới đi đến Redis, RabbitMQ, Kubernetes và hệ thống phân tán.

### Sau chương này, bạn có thể

- Kể lại vòng đời của một giao dịch mua hàng từ discovery đến fulfillment.
- Phân biệt Product, SKU, Inventory, Cart, Order, Payment và Shipment.
- Nhận diện **dòng thông tin, dòng hàng hóa, dòng tiền** và lý do chúng có thể lệch trạng thái.
- Viết các **business invariant** hệ thống không được vi phạm.
- Phát hiện những failure scenario ẩn phía sau một flow trông có vẻ đơn giản.

## 1.1. E-commerce dưới góc nhìn Backend Developer

Hãy hình dung một cửa hàng điện thoại truyền thống: khách xem hàng, nhân viên báo giá, kiểm tra kho, thu tiền, giao sản phẩm. Đưa cửa hàng lên Internet không làm các bước đó biến mất; nó biến chúng thành những hành vi phần mềm có thể diễn ra đồng thời ở quy mô lớn.



Sơ đồ 01 · SVG có thể phóng to

| Cửa hàng vật lý | E-commerce backend               |
|-----------------|----------------------------------|
| Khách xem hàng  | Product Catalog / Product Detail |

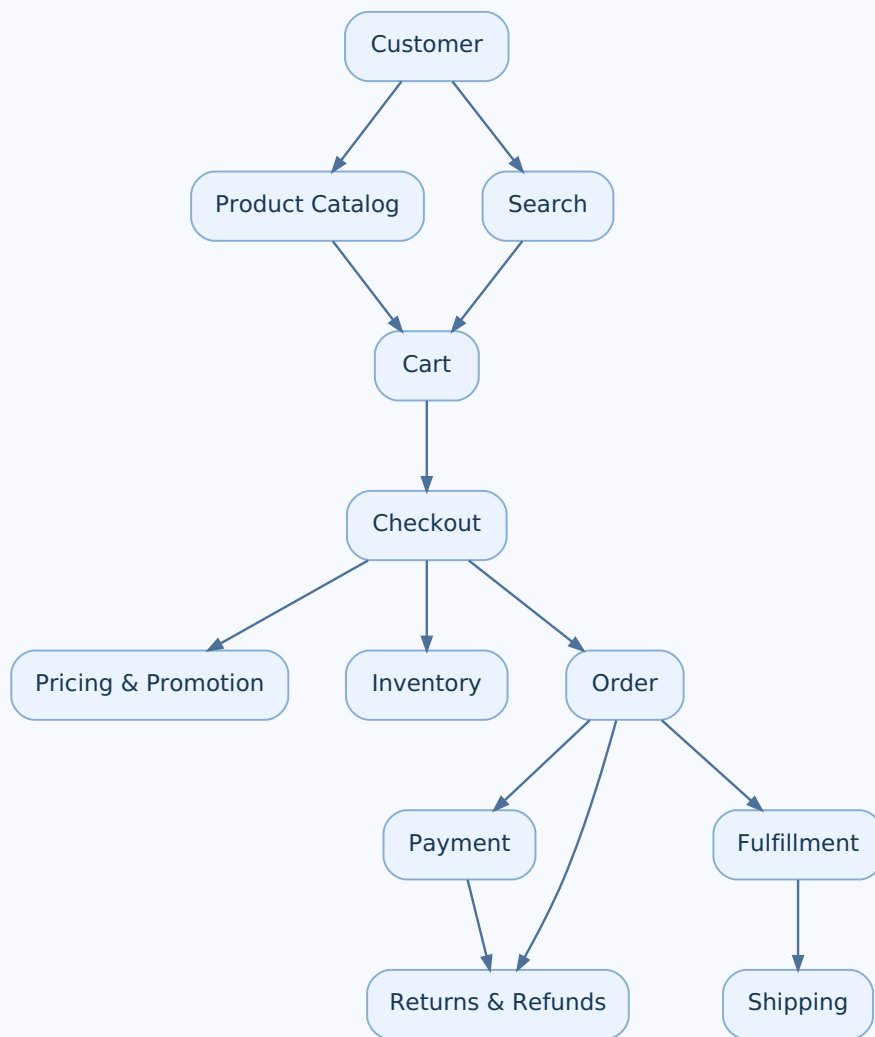
| Cửa hàng vật lý        | E-commerce backend      |
|------------------------|-------------------------|
| Nhân viên tìm hàng     | Search / Recommendation |
| Khách chọn nhiều món   | Shopping Cart           |
| Nhân viên báo giá      | Pricing / Promotion     |
| Nhân viên kiểm tra kho | Inventory               |
| Khách xác nhận mua     | Checkout / Order        |
| Thu tiền               | Payment                 |
| Giao sản phẩm          | Fulfillment / Shipping  |
| Trả hàng               | Return / Refund         |

Một nhân viên có thể nhận ra hai khách cùng muốn mua chiếc điện thoại cuối cùng. Một backend cần đảm bảo điều đó khi có hàng nghìn request đến gần như cùng lúc. Bởi vậy, **business logic + state + concurrency** là những mảnh ghép bắt buộc.

**Định nghĩa:** E-commerce backend là hệ thống quản lý quy tắc kinh doanh, dữ liệu, giao dịch và trạng thái của quá trình mua bán trực tuyến.

## 1.2. Các domain cốt lõi

Sơ đồ sau mô tả **ranh giới trách nhiệm logic**, không bắt buộc mỗi ô là một microservice:



Sơ đồ 02 · SVG có thể phóng to

## A. Product Catalog — Chúng ta bán cái gì?

Product Catalog quản lý danh tính và nội dung sản phẩm: tên, mô tả, thương hiệu, danh mục, hình ảnh, trạng thái hiển thị. **Product tồn tại không đồng nghĩa có thể mua ngay.** Một điện thoại vẫn xuất hiện trên website khi hết hàng.

```
{
  "productId": 1001,
  "name": "Smartphone X",
  "brand": "Example Brand",
  "category": "Smartphones",
  "status": "ACTIVE"
}
```

SKU (Stock Keeping Unit) đại diện cho một biến thể hàng hóa cần quản lý riêng. Ví dụ, điện thoại màu đen 256 GB và màu trắng 512 GB thường là hai SKU. Sự khác nhau này quyết định cách thiết kế giá, tồn kho và dòng đơn hàng.

## B. Inventory — Chúng ta có thể bán bao nhiêu?

Inventory theo dõi khả năng cung cấp theo SKU, đôi khi theo từng kho.

```
{
  "skuId": "PHONE-X-BLACK-256",
  "warehouseId": "WH-HN-01",
  "physicalQuantity": 100,
  "reservedQuantity": 20,
  "availableQuantity": 80
}
```

Trong mô hình đơn giản: `available = physical - reserved`. Hệ thống thật có thể phải trừ hàng hỏng, hàng đang khóa, hàng chờ kiểm kê hoặc theo chính sách backorder.

## C. Shopping Cart — Khách muốn mua gì?

Giỏ hàng biểu thị ý định mua. Mặc định, **thêm vào giỏ không phải là giữ tồn kho**. Có thể 1.000 người cùng thêm một SKU vào giỏ trong khi kho chỉ có 100 chiếc. Thời điểm giữ hàng là quyết định nghiệp vụ, thường nằm gần checkout hoặc bước xác nhận đơn.

```
{
  "userId": 101,
  "items": [{ "skuId": "PHONE-X-BLACK-256", "quantity": 2 }]
}
```

## D. Order — Giao dịch đã được xác nhận điều gì?

Order không chỉ là bản sao Cart. Nó cần lưu **snapshot giao dịch**: tên/phiên bản hàng cần thiết, đơn giá áp dụng, số lượng, giảm giá, địa chỉ giao, tổng tiền, loại tiền và trạng thái nghiệp vụ.

```
{
  "orderId": "ORD-1001",
  "userId": 101,
  "status": "PENDING_PAYMENT",
  "totalAmount": 20000000,
  "currency": "VND"
}
```

Nếu ngày mai admin tăng giá catalog, đơn đã chốt không tự đổi theo. Chính sách chỉnh đơn/hủy đơn là một hành vi tường minh, không phải side effect từ việc sửa sản phẩm.

## E. Payment — Dòng tiền đang ở trạng thái nào?

Một Order có thể có **nhiều payment attempt**: thanh toán lần đầu timeout, lần hai dùng phương thức khác, hoặc có nhiều lần thử bị từ chối. Không nên gộp khái niệm Order, Payment Attempt và Payment Transaction thành một trạng thái duy nhất. Payment Gateway có thể xác nhận kết quả bất đồng bộ qua webhook.

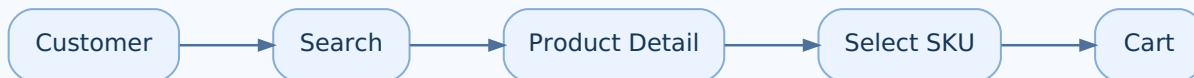
## F. Fulfillment — Làm sao hàng đến tay người mua?

Fulfillment bao gồm phân bổ kho, lấy hàng, đóng gói, bàn giao vận chuyển, theo dõi giao và xử lý ngoại lệ. **Đã trả tiền ≠ đã giao hàng**. Đơn có thể được thanh toán thành công nhưng kho phát hiện hàng bị hỏng hoặc đơn vị vận chuyển không thể tiếp nhận.

## 1.3. End-to-end: Một giao dịch mua thực sự

Giả sử khách chọn một điện thoại giá 10 triệu đồng, thanh toán online và nhận hàng tại Hà Nội.

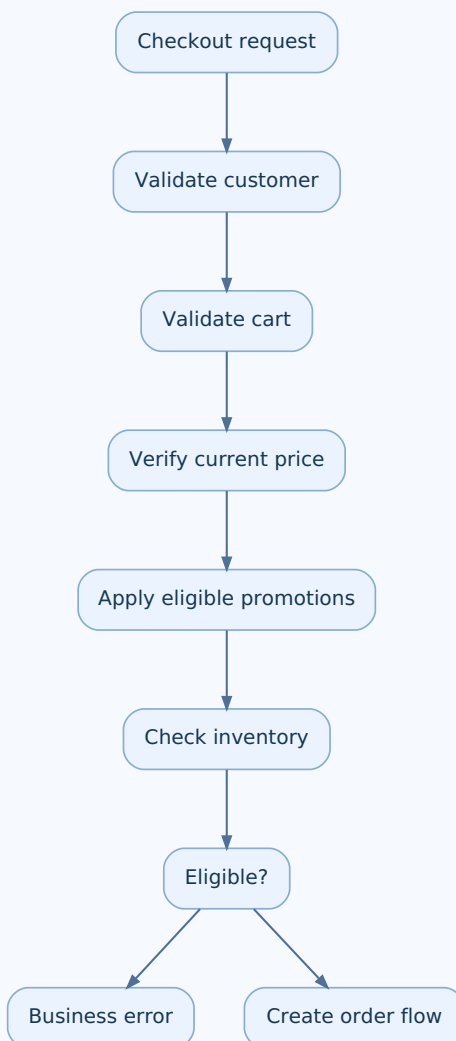
### Phase 1 — Discovery: chủ yếu là READ



Sơ đồ 03 · SVG có thể phóng to

Dữ liệu thường đọc gồm product, variant, image, category, price, inventory summary. Khi tải đọc tăng, cache, index, search engine và CDN có thể trở nên hữu ích — nhưng chưa cần áp dụng chỉ vì thấy công nghệ đó phổ biến.

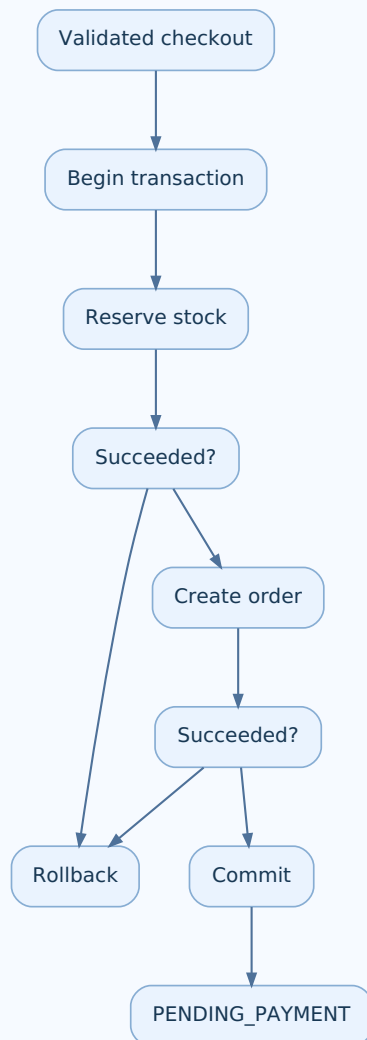
### Phase 2 — Checkout: kiểm tra lại sự thật



Sơ đồ 04 · SVG có thể phóng to

Giỏ có thể được tạo lúc 10:00; khuyến mãi kết thúc 10:05; khách thanh toán 10:10. Backend phải chốt giá theo chính sách thực tế và thông báo nếu điều kiện thay đổi. Giá frontend gửi lên không phải nguồn sự thật.

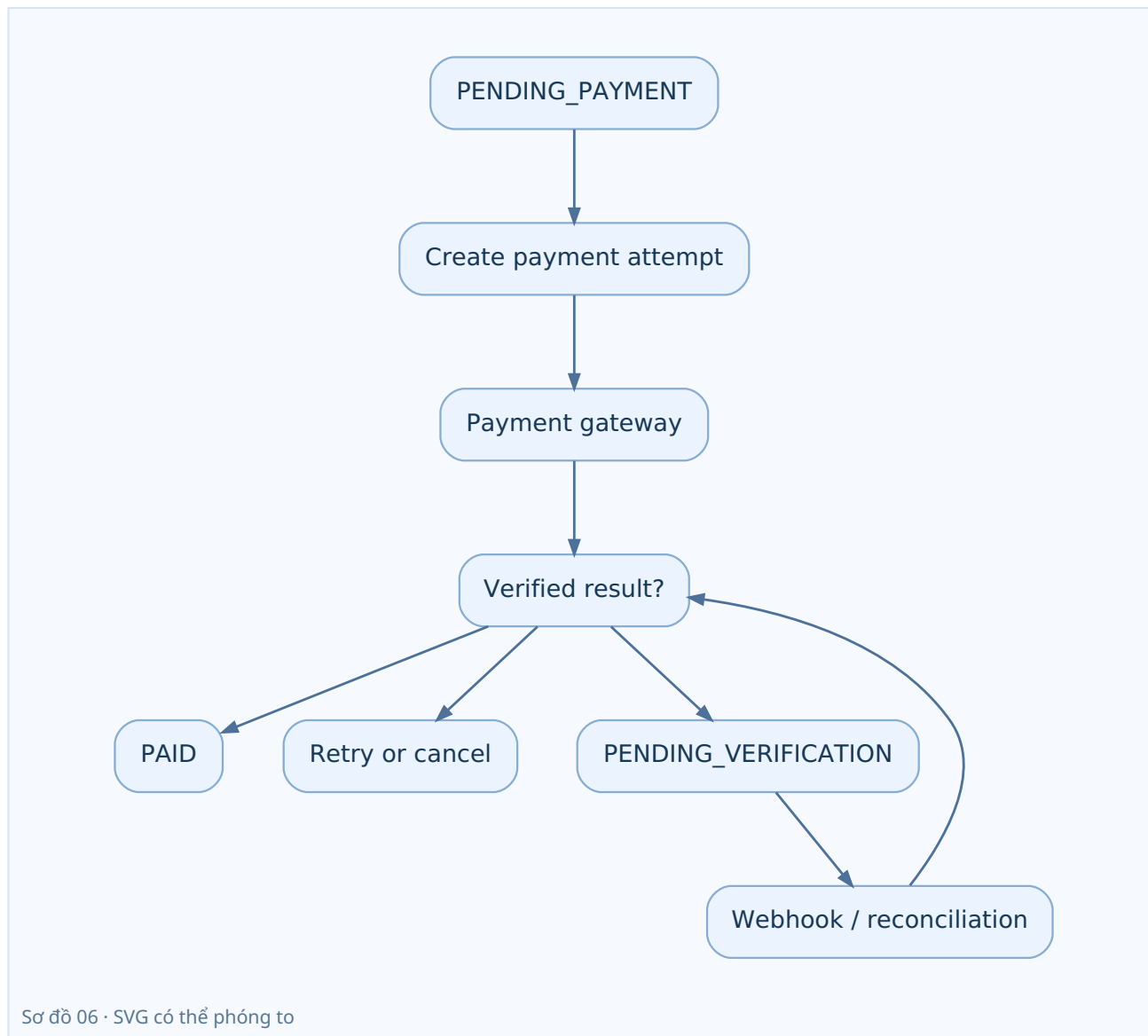
### Phase 3 — Order creation và stock reservation



Sơ đồ 05 · SVG có thể phóng to

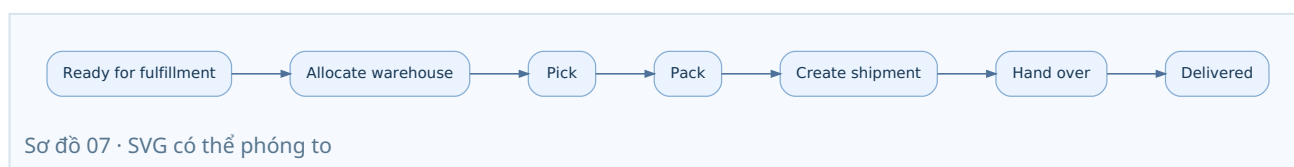
Sơ đồ giả định Order và Inventory nằm trong **cùng database transaction**. Nếu hai domain nằm ở hai service/database khác nhau, một transaction SQL thông thường không còn bao trùm được cả hai; các chương Distributed Systems sẽ giải quyết ranh giới đó.

## Phase 4 — Payment: timeout không có nghĩa thất bại



Nếu gọi Payment Gateway bị timeout, tiền **có thể đã bị trừ** trong khi backend chưa nhận phản hồi. Đừng đánh dấu **FAILED** chỉ dựa trên lỗi mạng. Tùy hợp đồng tích hợp, cần tra soát giao dịch, nhận webhook và bảo đảm idempotency.

## Phase 5 — Fulfillment



Kho thiếu hàng thực tế, đổi địa chỉ, người nhận từ chối hàng hoặc hãng vận chuyển thất bại đều là những nhánh cần thiết của nghiệp vụ. Một cột **SUCCESS/FAILED** duy nhất thường quá nghèo để mô tả toàn bộ vòng đời.

## 1.4. Ba dòng chảy khác nhau

| Dòng chảy        | Hành trình mẫu                                       | Câu hỏi chính                                        |
|------------------|------------------------------------------------------|------------------------------------------------------|
| Information flow | Product → Cart → Order snapshot                      | Ta biết gì về giao dịch tại từng thời điểm?          |
| Inventory flow   | Warehouse → Reservation → Allocation → Shipment      | Hàng đang ở đâu và có còn được bán không?            |
| Money flow       | Payment attempt → Confirmation → Settlement → Refund | Tiền đã được ghi nhận, đối soát và hoàn trả thế nào? |

Giả sử khách đặt và thanh toán thành công, nhưng kho phát hiện hàng bị hỏng:

- Information flow: Order đã tạo và có lịch sử rõ ràng.
- Inventory flow: Không thể thực hiện việc giao hàng như dự kiến.
- Money flow: Tiền đã được thanh toán; có thể cần hoàn tiền.

Đổi `order.status = FAILED` **không tự động** hoàn tiền hoặc sửa tồn kho. Chính chỗ lệch trạng thái này dẫn đến nhu cầu thiết kế Saga, Compensation và Reconciliation.

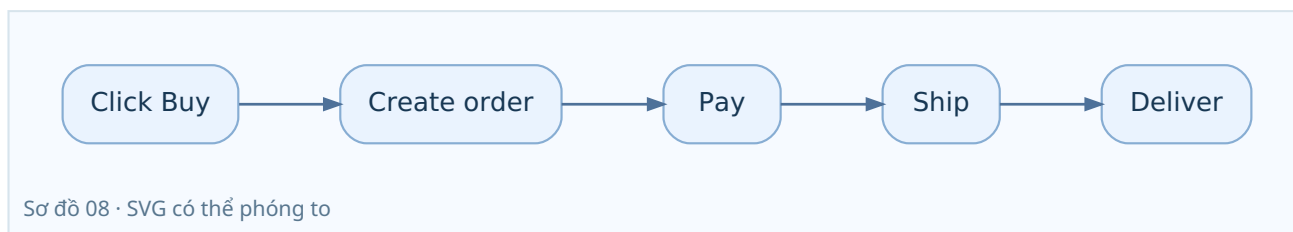
## 1.5. Business invariants: Điều hệ thống không được phép vi phạm

Invariant là điều kiện phải đúng tại ranh giới nghiệp vụ được xác định, ngay cả khi có concurrent requests hoặc một số thành phần bị lỗi.

| Invariant        | Ý nghĩa                                                                                        |
|------------------|------------------------------------------------------------------------------------------------|
| Inventory        | Không xác nhận giữ thành công vượt khả năng phân bổ, trừ khi nghiệp vụ chủ động cho backorder. |
| Order snapshot   | Giá trị đã chốt của đơn không tự biến đổi vì catalog đổi giá.                                  |
| Payment          | Nhận lại cùng một thông báo không tạo tác động tài chính/ng nghiệp vụ trùng.                   |
| Reservation      | Một reservation chỉ được giải phóng/hoàn hàng một lần.                                         |
| State transition | Không chuyển trạng thái bất hợp lệ, ví dụ giao hàng thành công trước khi có shipment hợp lệ.   |

**Performance** trả lời *hệ thống nhanh đến đâu*. **Correctness** trả lời *hệ thống có đưa ra kết quả đúng không*. Một hệ thống cache cực nhanh nhưng oversell hay hoàn tiền trùng vẫn là một thiết kế chưa đạt yêu cầu.

## 1.6. Failure scenarios: Thiết kế ngoài happy path



Đó chỉ là happy path. Production còn những kịch bản sau:

| Tình huống                        | Rủi ro                     | Keyword để học tiếp        |
|-----------------------------------|----------------------------|----------------------------|
| Hai khách mua sản phẩm cuối       | Overselling                | Concurrency, Atomic Update |
| Khách nhấn Mua hai lần            | Duplicate order            | Idempotency                |
| DB commit nhưng mất HTTP response | Client tưởng chưa đặt hàng | Request identity, Retry    |
| Payment Gateway timeout           | Kết quả thanh toán chưa rõ | Reconciliation             |
| Webhook đến trễ hoặc trùng        | Order bị cập nhật sai      | Idempotent handler         |
| Worker chết sau khi giữ hàng      | Reservation bị treo        | Expiration, Recovery       |
| Khách hủy khi kho đang đóng gói   | Tranh chấp trạng thái      | State machine              |
| Giá đổi trong checkout            | Price inconsistency        | Pricing snapshot           |

Nhiều lỗi không nằm ở cú pháp Java hay SQL; chúng phát sinh do **thời điểm, thứ tự thực hiện và trạng thái phân tán**.

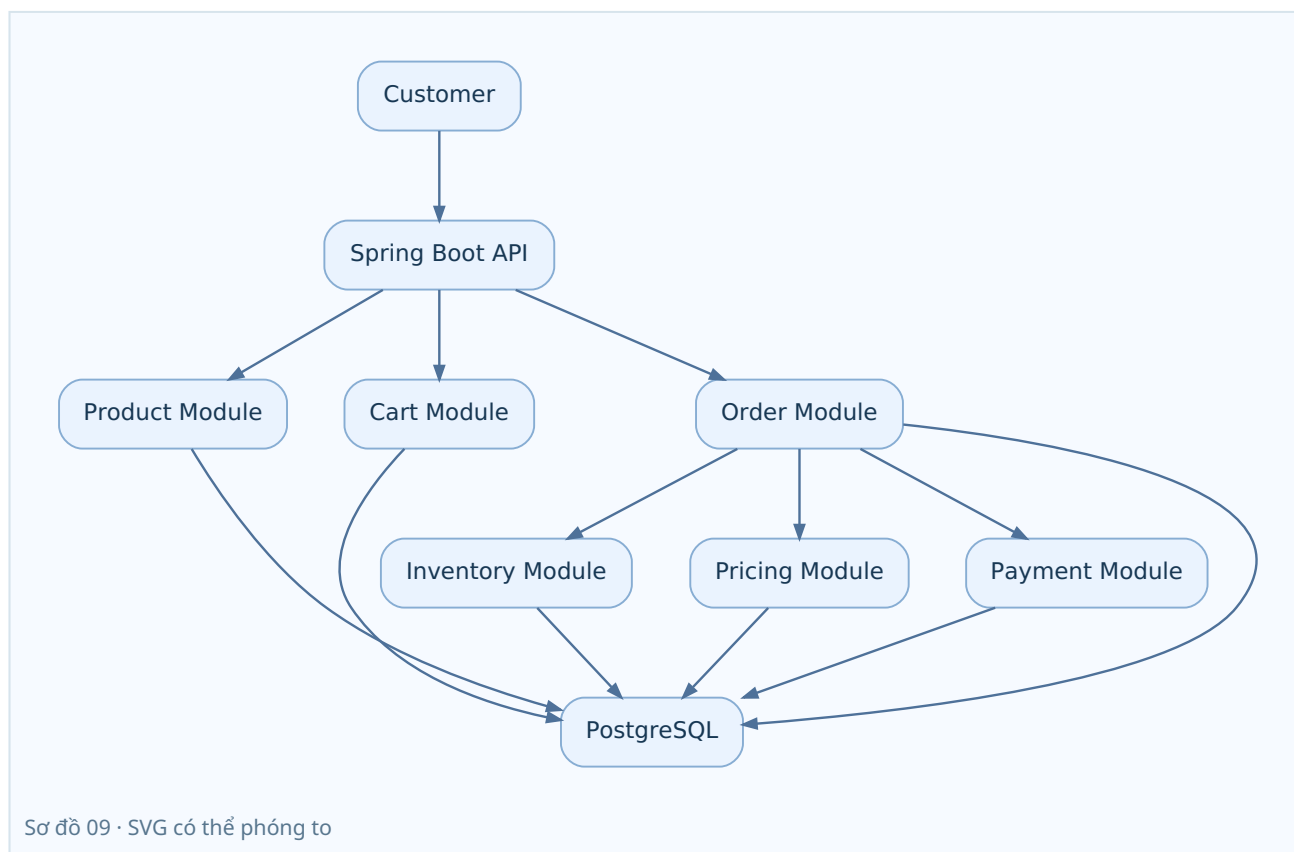
## 1.7. Engineering Lab — Cửa hàng điện thoại đầu tiên

**Phạm vi giả định:** 1.000 người dùng; 100 product; một kho; Spring Boot + PostgreSQL; kiến trúc modular monolith. Đây là ràng buộc bài tập, **không phải** benchmark năng lực chịu tải.

### Lab A — Gán đúng domain

1. Khách hỏi điện thoại có những màu gì? → **Product Catalog**.
2. Khách hỏi kho còn mấy máy đen 256 GB? → **Inventory**.
3. Khách đã thanh toán nhưng chưa nhận hàng? → **Payment + Fulfillment**.
4. Admin đổi giá sau khi khách đặt? → **Giữ snapshot giao dịch của đơn đã chốt**.

## Lab B — Vẽ module boundary



Các module là các phần logic của một ứng dụng; không cần tách thành microservice ngay.

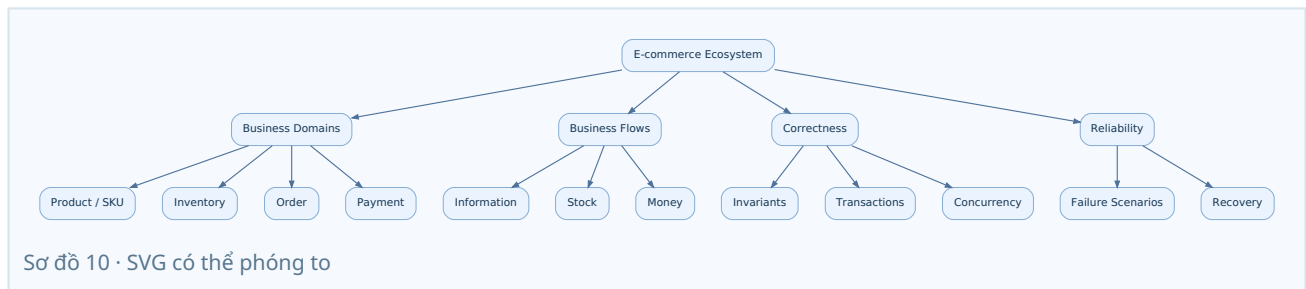
## Lab C — Tìm điểm yếu schema

Xem thiết kế cơ ý quá đơn giản:

```
CREATE TABLE products (  
  id BIGINT PRIMARY KEY,  
  name TEXT NOT NULL,  
  price NUMERIC(18,2) NOT NULL,  
  stock INTEGER NOT NULL  
);  
  
CREATE TABLE orders (  
  id BIGINT PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  product_id BIGINT NOT NULL,  
  status TEXT NOT NULL  
);
```

Hãy tự thiết kế lại để xử lý: nhiều item mỗi đơn; nhiều SKU mỗi product; snapshot đơn giá; nhiều kho; voucher; hoàn tiền/hoàn hàng một phần. Chúng ta sẽ giải bài toán này ở chương Domain Modeling và Database Design.

# Knowledge Map



## Key Takeaways

1. E-commerce không chỉ là CRUD sản phẩm và đơn hàng.
2. Product, SKU, Cart, Inventory, Order, Payment, Fulfillment có những trách nhiệm riêng.
3. Information, stock và money flow có thể ở trạng thái không đồng bộ.
4. Business invariants cho biết tính đúng đắn nào cần được bảo vệ.
5. Phải thiết kế cả happy path lẫn failure scenarios.
6. Hiểu domain trước khi chọn Redis, RabbitMQ, microservices hoặc Kubernetes.

**Chương tiếp theo:** Chapter 02 — E-commerce Business Models: B2C, B2B, C2C, D2C, Marketplace; seller ownership, commission, settlement, multi-seller orders và hệ quả đối với kiến trúc dữ liệu.

Inside E-commerce · Chapter 01 · Bản nội dung và sơ đồ dùng được khi offline. Mermaid source nằm trong Markdown.

# INSIDE E-COMMERCE

## Volume 1 — Understanding E-commerce

### Chapter 02 — E-commerce Business Models

**Câu hỏi dẫn đường:** Tại sao một cửa hàng tự bán hàng và một marketplace có nhiều người bán đều có Product, Cart, Order, Payment nhưng cần **mô hình dữ liệu, phân quyền, checkout, tồn kho và dòng tiền** khác nhau?

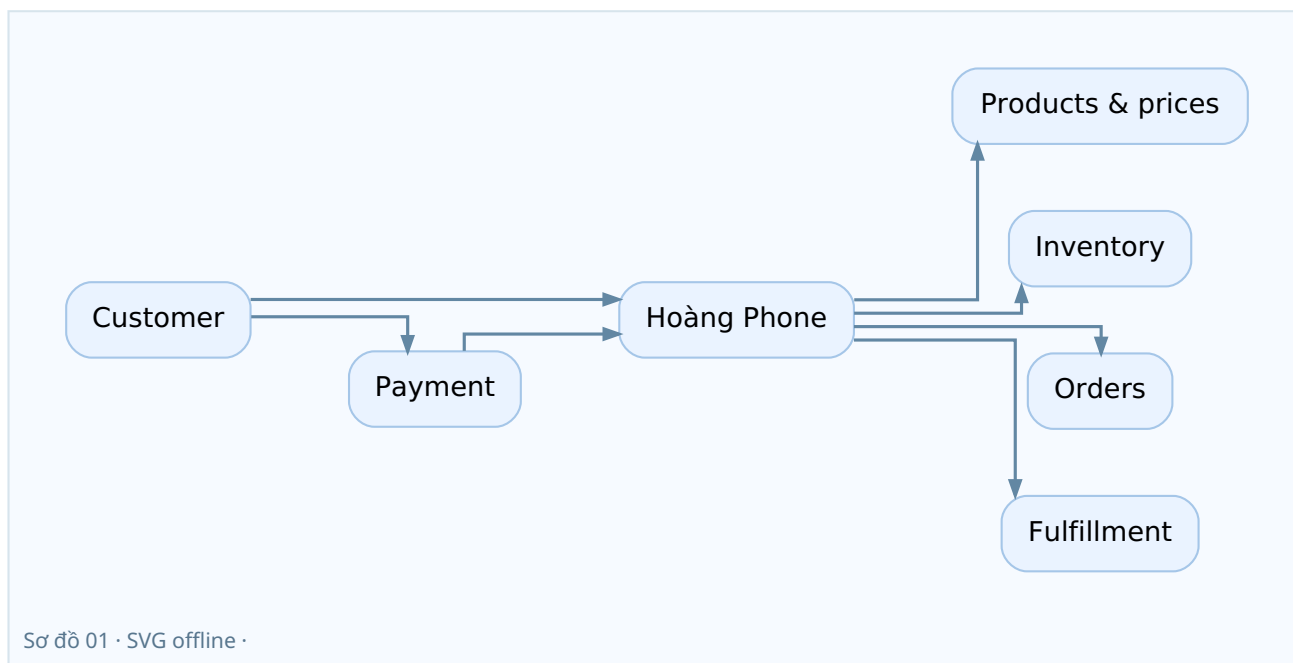
**Giới hạn chương:** Đây là tài liệu học kiến trúc phần mềm. Thuế, tư cách người bán chính thức, trách nhiệm hoàn tiền và quy trình chuyển tiền phụ thuộc hợp đồng, cổng thanh toán và từng thị trường; ví dụ trong sách là mô hình nghiệp vụ giả định, không thay cho tư vấn pháp lý/kế toán.

### Sau chương này, bạn có thể

- Phân biệt **B2C, B2B, C2C, D2C, 1P retail, 3P marketplace và hybrid** mà không nhầm các trục phân loại.
- Nêu ai là **seller of record**, ai sở hữu hàng, ai định giá, ai đóng gói, ai tiếp nhận tiền và ai giải quyết trả hàng cho từng mô hình.
- Giải thích vì sao một cart nhiều seller cần **order group + seller orders**, và có thể có nhiều shipment, refund, settlement.
- Phân biệt **catalog product, seller listing/offer, SKU, seller inventory, order item snapshot**.
- Phác thảo schema, API và authorization boundaries cho marketplace bằng Spring Boot/PostgreSQL.
- Tránh sai lầm đồng nhất **PAID, DELIVERED, SETTLED** hoặc coi Seller Balance là tiền chắc chắn đã chuyển tới ngân hàng.

### 2.1. Bắt đầu từ một cửa hàng điện thoại

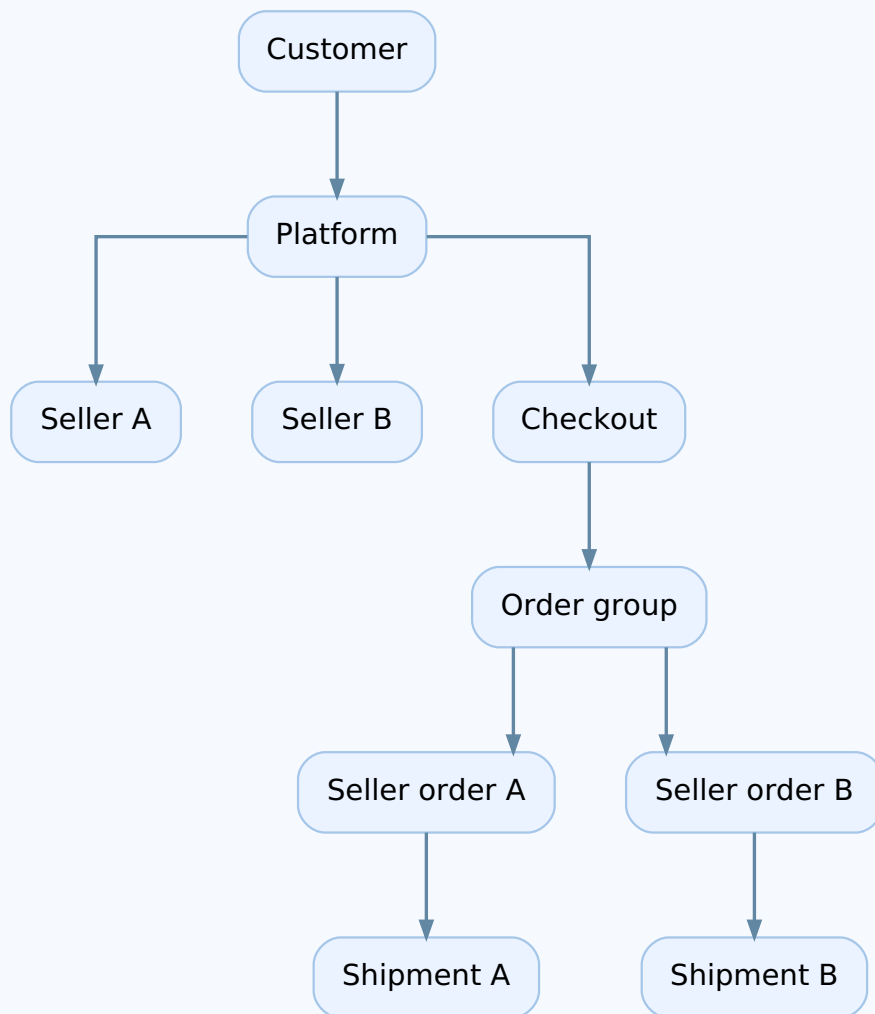
Ở chương 01, bạn xây cửa hàng **Hoàng Phone**: cửa hàng nhập điện thoại từ nhà phân phối, đăng sản phẩm, lưu kho, bán hàng, nhận thanh toán và giao hàng. Toàn bộ nghiệp vụ khá dễ hình dung vì phần lớn trách nhiệm tập trung về một đơn vị bán hàng.



Một hôm, Hoàng Phone muốn bán thêm laptop, tai nghe, đồng hồ và phụ kiện mà không tự nhập tất cả vào kho. Cửa hàng mở cổng cho hai đối tác:

- **Seller A** bán điện thoại và tai nghe từ kho Hà Nội.
- **Seller B** bán đồng hồ và phụ kiện từ kho TP.HCM.

Khách hàng vẫn nhìn thấy **một website, một giỏ hàng, một nút checkout**. Nhưng đằng sau, hai seller có tồn kho, giá, trạng thái xác nhận đơn và vận chuyển riêng. Dòng tiền còn phải chia theo thỏa thuận giữa nền tảng và từng seller.



Sơ đồ 02 · SVG offline ·

**Vấn đề hệ thống không còn là chỉ có bao nhiêu khách hàng.** Câu hỏi đầu tiên cần trả lời là: **trong một giao dịch, mỗi bên có quyền gì, phải làm gì và chịu trách nhiệm tới đâu?**

## 2.2. Các thuật ngữ không nằm trên cùng một trục

Đây là chỗ tài liệu tổng quan thường khiến người học nhầm lẫn. B2C/B2B/C2C mô tả **các bên giao dịch**; D2C mô tả **kênh đưa sản phẩm tới người mua**; 1P/3P mô tả **vai trò của nền tảng đối với người bán**. Chúng không phải sáu lựa chọn loại trừ lẫn nhau.

| Khái niệm                  | Câu hỏi nó trả lời       | Ví dụ giả định                     | Hàm ý backend nổi bật                                          |
|----------------------------|--------------------------|------------------------------------|----------------------------------------------------------------|
| B2C (Business-to-Consumer) | Doanh nghiệp bán cho ai? | Hoàng Phone → khách cá nhân        | Catalog, cart, consumer checkout, refund                       |
| B2B (Business-to-Business) | Doanh nghiệp bán cho ai? | Hoàng Phone → công ty mua 100 máy  | Organization, approver, quote, price tier, credit terms        |
| C2C (Consumer-to-Consumer) | Ai giao dịch với ai?     | Cá nhân A bán máy cũ cho cá nhân B | Identity/trust, listing moderation, dispute, seller onboarding |

| Khái niệm                    | Câu hỏi nó trả lời                               | Ví dụ giả định                          | Hàm ý backend nổi bật                                      |
|------------------------------|--------------------------------------------------|-----------------------------------------|------------------------------------------------------------|
| D2C (Direct-to-Consumer)     | Thương hiệu có bán trực tiếp tới khách không?    | Hãng X bán máy trên site của chính hãng | Brand-owned pricing, CRM, fulfillment; có thể vẫn là B2C   |
| 1P Retail (First Party)      | Nền tảng có tự đứng vai trò người bán?           | Nền tảng tự bán hàng do mình kinh doanh | Own stock, own order liability, centralized fulfillment    |
| 3P Marketplace (Third Party) | Nền tảng có cho bên thứ ba bán?                  | Nhiều seller bán qua cùng một site      | Seller tenancy, listing, split orders, fees, settlement    |
| Hybrid 1P + 3P               | Nền tảng có vừa tự bán vừa cho seller ngoài bán? | Cùng một catalog có hàng 1P và 3P       | Nhận diện seller, offer selection, nhiều luồng trách nhiệm |

**Ví dụ về việc nhiều nhãn cùng đứng:** Một hãng bán laptop trực tiếp cho khách cá nhân qua website của hãng vừa là **B2C**, vừa là **D2C**, đồng thời có thể là **1P** đối với website đó. Một nền tảng B2B vẫn có thể vận hành marketplace nhiều nhà cung cấp.

## Dropshipping nằm ở đâu?

Dropshipping chủ yếu mô tả **cách thực hiện đơn hàng**: bên nhận đơn không nhất thiết nắm giữ hàng, supplier có thể giao trực tiếp. Nó không tự quyết định các vai trò B2C/marketplace hay ai là người bán chịu trách nhiệm với khách. Đừng nhìn thấy hàng xuất từ supplier rồi kết luận backend phải là marketplace.

## SaaS bán công cụ E-commerce có phải marketplace?

Một phần mềm cho nhiều merchant tạo website riêng là **commerce-enablement SaaS**. Merchant có thể tự sở hữu khách hàng, checkout và quan hệ thanh toán. Đó khác với một marketplace nơi nhiều seller cùng bán dưới một trải nghiệm mua hàng tổng hợp. Đôi khi một công ty cung cấp cả hai tính năng, nhưng hệ thống vẫn cần phân biệt domain.

## 2.3. Ma trận trách nhiệm — thiết kế trước khi viết entity

Trước khi lập bảng `sellers`, tôi muốn bạn điền một ma trận cho **một giao dịch cụ thể**.

| Quyết định                        | Single-vendor 1P (giả định) | Marketplace 3P (giả định)                                        |
|-----------------------------------|-----------------------------|------------------------------------------------------------------|
| Ai đăng sản phẩm?                 | Store admin                 | Seller, có thể chờ platform duyệt                                |
| Ai quyết định giá niêm yết?       | Store                       | Seller hoặc quy tắc platform theo hợp đồng                       |
| Ai sở hữu tồn kho vật lý?         | Store/đối tác kho của store | Từng seller hoặc mạng kho do platform vận hành                   |
| Ai nhận yêu cầu checkout?         | Store                       | Platform                                                         |
| Ai giữ stock?                     | Inventory của store         | Inventory theo seller + SKU + warehouse                          |
| Ai thực hiện giao hàng?           | Store hoặc 3PL              | Seller, platform fulfillment hoặc 3PL theo từng offer            |
| Ai nhận tiền từ payment provider? | Theo tài khoản của store    | Tùy payment flow: seller trực tiếp hoặc platform nhận rồi chuyển |
| Ai hưởng hoa hồng/fee?            | Không bắt buộc có           | Platform theo chính sách từng giao dịch                          |
| Ai xử lý hoàn tiền?               | Store                       | Seller/platform/payment provider phối hợp theo thỏa thuận        |

**Lưu ý:** “Ai sở hữu hàng”, “ai xuất hàng”, “ai là merchant of record”, “ai nhận tiền ban đầu” và “ai chịu phí chargeback” có thể là **những chủ thể khác nhau**. Không dùng một boolean `isMarketplace` để suy ra tất cả trách nhiệm.

**Quy tắc thiết kế:** Business ownership → Domain boundaries → Data ownership → API permissions → Payment architecture. Làm ngược lại thường dẫn tới schema thiếu trường và flow xử lý khó sửa.

## 2.4. Chuyển từ một Product sang nhiều Offer

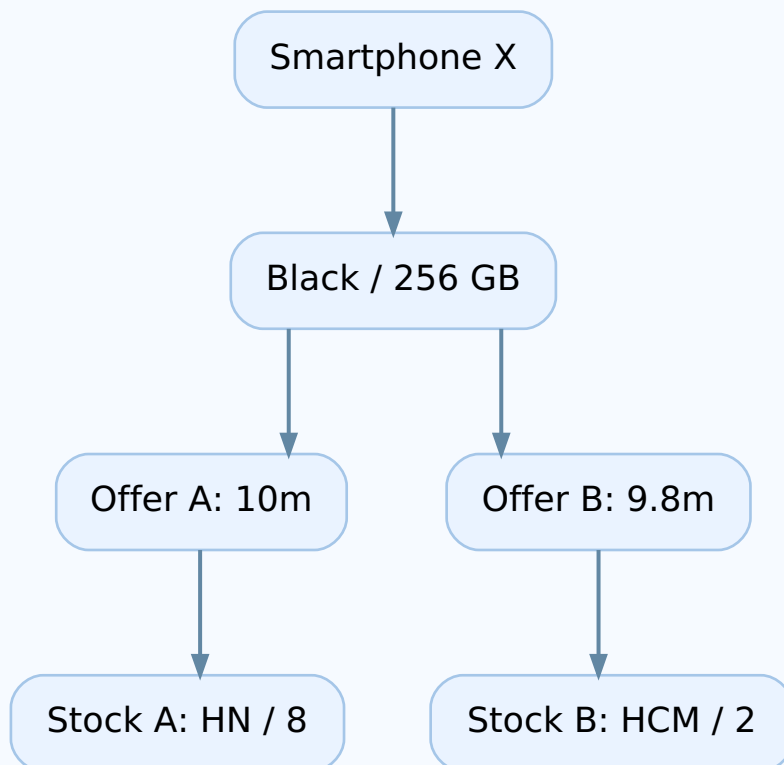
### 2.4.1. Sai lầm khi chỉ thêm `seller_id` vào `products`

Trong store 1P, một bảng `products(id, name, price, stock)` có thể dùng để minh họa. Sang marketplace, hãy giả sử hai seller cùng bán một mẫu điện thoại:

- Seller A: điện thoại màu đen 256 GB, giá 10.000.000đ, còn 8 chiếc, giao từ Hà Nội.
- Seller B: cùng biến thể, giá 9.800.000đ, còn 2 chiếc, giao từ TP.HCM.

Nếu `products.price = 10000000` và `products.stock = 10`, dữ liệu đó thuộc về ai? Cập nhật giá của Seller B có làm thay đổi giá Seller A không? Khách xem đánh giá chung của mẫu điện thoại hay đánh giá riêng chất lượng dịch vụ của Seller B?

Câu trả lời là bạn đang gộp nhiều khái niệm khác nhau.



Sơ đồ 03 · SVG offline ·

### 2.4.2. Phân biệt bốn đối tượng

| Đối tượng              | Định nghĩa vận hành                                                           | Ví dụ                      |
|------------------------|-------------------------------------------------------------------------------|----------------------------|
| Canonical Product      | Danh tính sản phẩm dùng chung trong catalog, nếu nền tảng có product matching | Smartphone X               |
| Product Variant / SKU  | Biến thể cần định danh                                                        | Black / 256 GB             |
| Seller Listing / Offer | Lời chào bán cụ thể của seller với giá/chính sách/trạng thái                  | Seller A chào giá 10 triệu |
| Seller Inventory       | Khả năng cung cấp của seller theo variant/kho                                 | Seller A còn 8 tại HN      |

Không phải mọi marketplace đều có canonical product; với hàng thủ công hoặc đồ cũ, **listing có thể chính là mặt hàng độc nhất**. Hãy thiết kế theo domain thực tế, không ép mọi loại hàng vào một catalog matching model.

### 2.4.3. Ai được sửa trường nào?

Seller A chỉ được sửa offer của mình. Platform có thể chuẩn hóa tên thương hiệu và danh mục. Nhân viên kho chỉ cập nhật inventory thuộc kho được giao. Customer chỉ được đọc những offer đang được phép hiển thị.

```
PATCH /seller/offers/{offerId}
Authorization check:
  authenticatedSellerId == offer.sellerId
AND seller.status == APPROVED
AND offer editable in current state
```

Không tin `sellerId` người dùng gửi trong JSON để xác định quyền. Backend phải lấy actor từ danh tính đã xác thực và kiểm tra ownership trên bản ghi thực tế.

## 2.5. Single-seller checkout và multi-seller checkout khác nhau ở đâu?

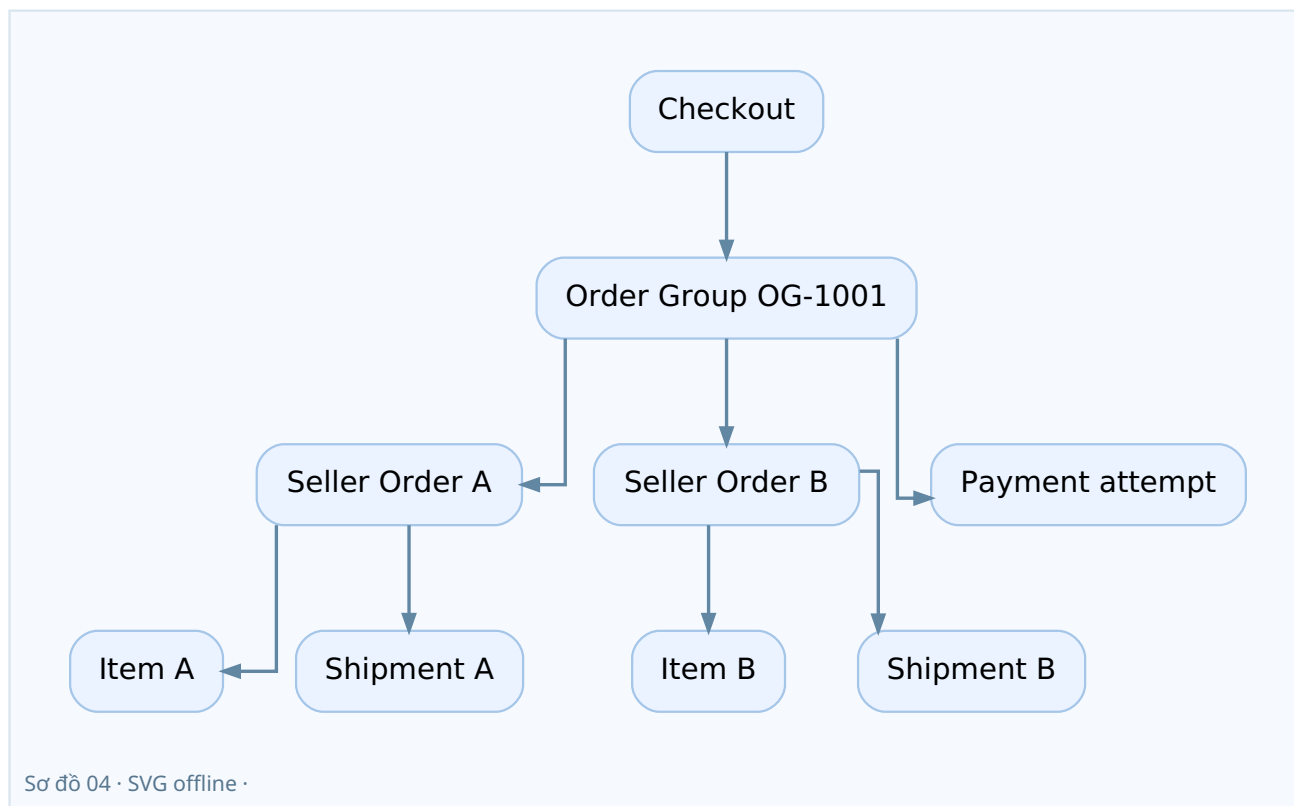
Giỏ hàng sau có một khách nhưng hai seller:

```
{
  "customerId": "U-101",
  "items": [
    {"offerId": "OFFER-A-01", "sellerId": "SELLER-A", "quantity": 1},
    {"offerId": "OFFER-B-09", "sellerId": "SELLER-B", "quantity": 2}
  ]
}
```

`sellerId` trong payload trên chỉ để minh họa; backend phải tự resolve và xác minh seller sở hữu offer, không chấp nhận client tự quyết định.

Trong store 1P, bạn có thể hình dung một order, một tổng tiền, một shipment (dù thực tế store cũng có thể ship nhiều kiện). Với marketplace, **một lần checkout không đồng nghĩa một seller order hoặc một lần giao hàng**.

### 2.5.1. Thiết kế khái niệm order group và seller order



- **Order Group:** đơn tổng hợp mà khách nhìn thấy ở checkout.
- **Seller Order:** tập dòng hàng của một seller với lifecycle thực hiện tương đối độc lập.
- **Order Item:** snapshot mặt hàng, giá và số lượng tại thời điểm chốt.
- **Shipment:** một lần/kiện vận chuyển; một seller order có thể cần nhiều shipment.
- **Payment:** có thể gắn với order group hoặc từng seller order tùy mô hình thanh toán.

Tên thực tế có thể là `orders + sub_orders`, `purchase + fulfillment_order`, hoặc `order_groups + seller_orders`. **Tên không quan trọng bằng invariant và quan hệ.**

### 2.5.2. Vì sao không lưu `seller_id` trực tiếp trong bảng `orders`?

Vì một Order Group nhiều seller **không có đúng một seller\_id**. Nếu ép lưu một seller, bạn sẽ mất thông tin hoặc buộc mỗi lần checkout thành nhiều giao dịch thanh toán riêng.

Một hệ thống vẫn có thể chỉ dùng một bảng `orders` với `parent_order_id`; điều đó khả thi, nhưng cần làm rõ row nào là group và row nào là seller order, unique constraints và state transition tương ứng. Với mục đích học, dùng hai bảng là cách thể hiện ranh giới rõ ràng hơn.

### 2.5.3. Partial failure là trạng thái bình thường

Nếu Seller A hết hàng nhưng Seller B còn hàng, hệ thống cần quy định: **rollback cả checkout** hay **cho phép chỉ mua các offer hợp lệ với sự đồng ý của khách**? Không có một đáp án bắt buộc cho mọi sản phẩm. Nhưng quyết định này cần đi trước việc viết transaction.

## 2.6. Dòng tiền marketplace — ba loại số tiền không được trộn

Có ba lớp khác nhau:

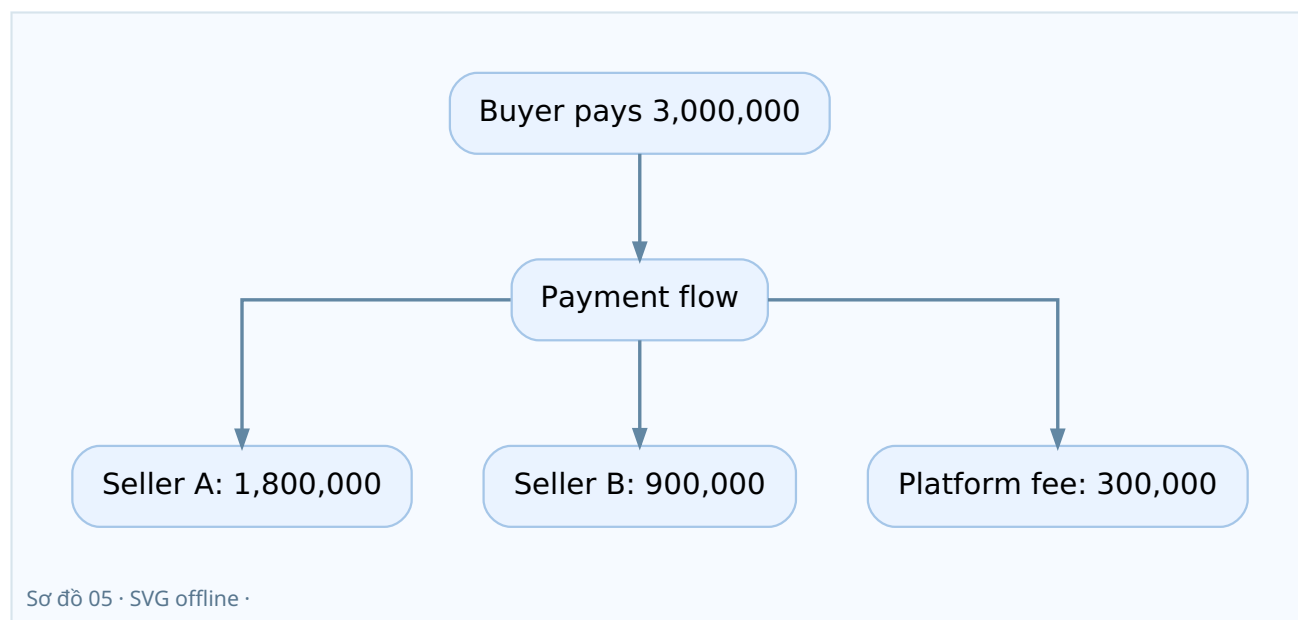
1. **Buyer payment:** khách phải trả bao nhiêu.
2. **Seller payable:** sau khi áp dụng quy tắc phí, hoàn tiền, hỗ trợ, người bán được nhận bao nhiêu.
3. **Platform revenue / costs:** nền tảng hưởng bao nhiêu và chịu phí nào.

### 2.6.1. Ví dụ tính tiền minh họa

Một order group gồm:

| Thành phần                                 | Giá trị        |
|--------------------------------------------|----------------|
| Hàng của Seller A                          | 2.000.000đ     |
| Hàng của Seller B                          | 1.000.000đ     |
| Gross merchandise subtotal                 | 3.000.000đ     |
| Platform commission giả định               | 10% mỗi seller |
| Seller A payable trước các điều chỉnh khác | 1.800.000đ     |
| Seller B payable trước các điều chỉnh khác | 900.000đ       |
| Platform commission trước các chi phí khác | 300.000đ       |

Ở ví dụ **không tính phí giao hàng, voucher, thuế, payment fee hoặc hoàn tiền**. Đây là phép tính nghiệp vụ giả định, không phải công thức hạch toán chung cho mọi thị trường.



Nếu platform tự tài trợ voucher 100.000đ mà cam kết seller vẫn nhận nguyên số như trên, số khách trả còn 2.900.000đ và phần còn lại dành cho platform trước các chi phí khác còn 200.000đ. **Ai tài trợ voucher là dữ liệu nghiệp vụ bắt buộc**; cùng mức giảm giá hiển thị nhưng quyền lợi của seller có thể khác.

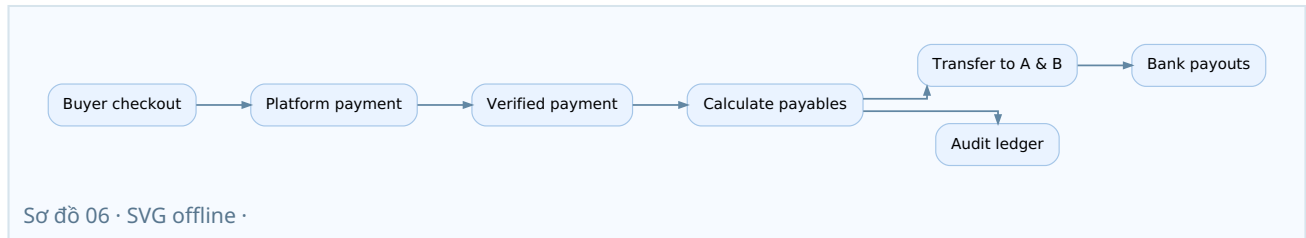
### 2.6.2. Charge khác transfer và payout

- **Charge/payment:** hành động thu tiền của khách theo provider.

- **Transfer:** luân chuyển quỹ giữa các tài khoản liên kết theo mô hình provider hỗ trợ.
- **Payout:** chuyển tiền từ số dư tại provider ra tài khoản ngân hàng/đích nhận của seller.
- **Settlement:** đối soát/ghi nhận số tiền phải thanh toán cho các bên theo một kỳ hoặc quy trình.

`payment.status = PAID` **không suy ra** `seller_payout.status = COMPLETED`.

Stripe Connect mô tả nhiều mô hình như **direct charges, destination charges, separate charges and transfers**. Mô hình chuyển tiền tới nhiều connected accounts là tình huống được Stripe tài liệu hóa; lựa chọn cụ thể phụ thuộc nơi hoạt động và quy tắc của provider. [S1][S2]



Sơ đồ là **mô hình chức năng giả định**, không phải hợp đồng API của một payment provider cụ thể.

### 2.6.3. Refund một phần làm thay đổi điều gì?

Khách chỉ trả lại hàng của Seller A. Hệ thống cần:

- Xác định item, số lượng và amount đủ điều kiện refund.
- Điều chỉnh seller payable hoặc thu hồi khoản đã chuyển theo cơ chế được phép.
- Xác định phí platform có hoàn lại hay không theo hợp đồng.
- Xử lý phí thanh toán, phí giao hàng, voucher theo quy tắc được phê duyệt.
- Lưu audit trail: **không sửa mất dấu khoản thu ban đầu**.

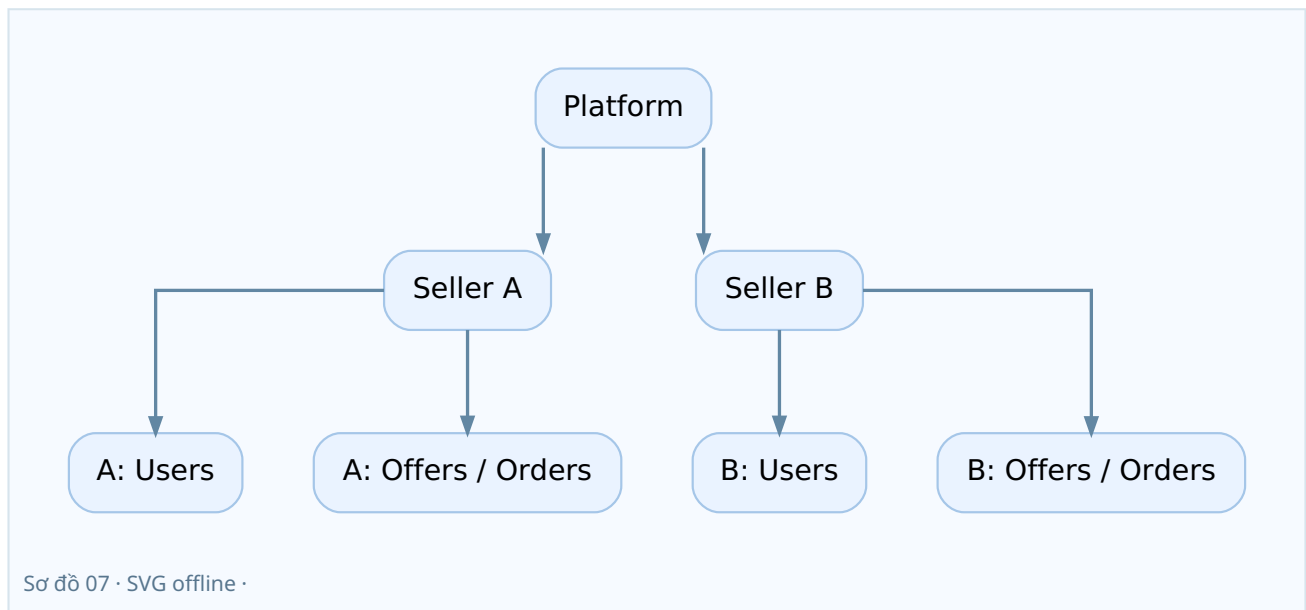
Cơ chế charge/dispute/refund phụ thuộc loại charge; tài liệu Stripe phân biệt trách nhiệm platform/seller theo payment architecture. [S3]

### 2.6.4. Marketplace khác ví điện tử

Một bảng `seller_balance` **không đồng nghĩa** bạn có thể tự ý giữ và di chuyển tiền thật như ngân hàng. Nó có thể chỉ là **số cái nội bộ ghi nhận nghĩa vụ phải trả**. Chức năng thu hộ, giữ tiền, chuyển tiền, KYC và payout phải phù hợp khả năng/tính hợp lệ của đối tác thanh toán và quy định áp dụng. Không tự triển khai “ví tiền” từ CRUD balance rồi coi là hoàn chỉnh.

## 2.7. Vì sao Marketplace cần seller onboarding và tenant isolation?

Store 1P chủ yếu quản lý customer, staff, admin. Marketplace thêm một actor quan trọng: **seller**. Seller có thể có nhiều nhân viên, nhiều kho, nhiều offer, nhiều payment/settlement record và nhiều order.



### 2.7.1. Onboarding state machine

```
REGISTERED → UNDER_REVIEW → APPROVED → ACTIVE
                                   ↘ REJECTED
ACTIVE → SUSPENDED → ACTIVE (if reapproved)
```

Seller đăng ký không tự động có quyền xuất bản offer hoặc nhận thanh toán. Thực tế còn cần xác minh thông tin theo provider và khu vực vận hành.

### 2.7.2. Data isolation là invariant, không phải một câu WHERE tùy hứng

```
SELECT *
FROM seller_orders
WHERE seller_id = :authenticated_seller_id
ORDER BY created_at DESC;
```

Dữ liệu của Seller B không được trả về cho Seller A dù A thay đổi URL từ `/seller/orders/1001` sang `/seller/orders/1002`.

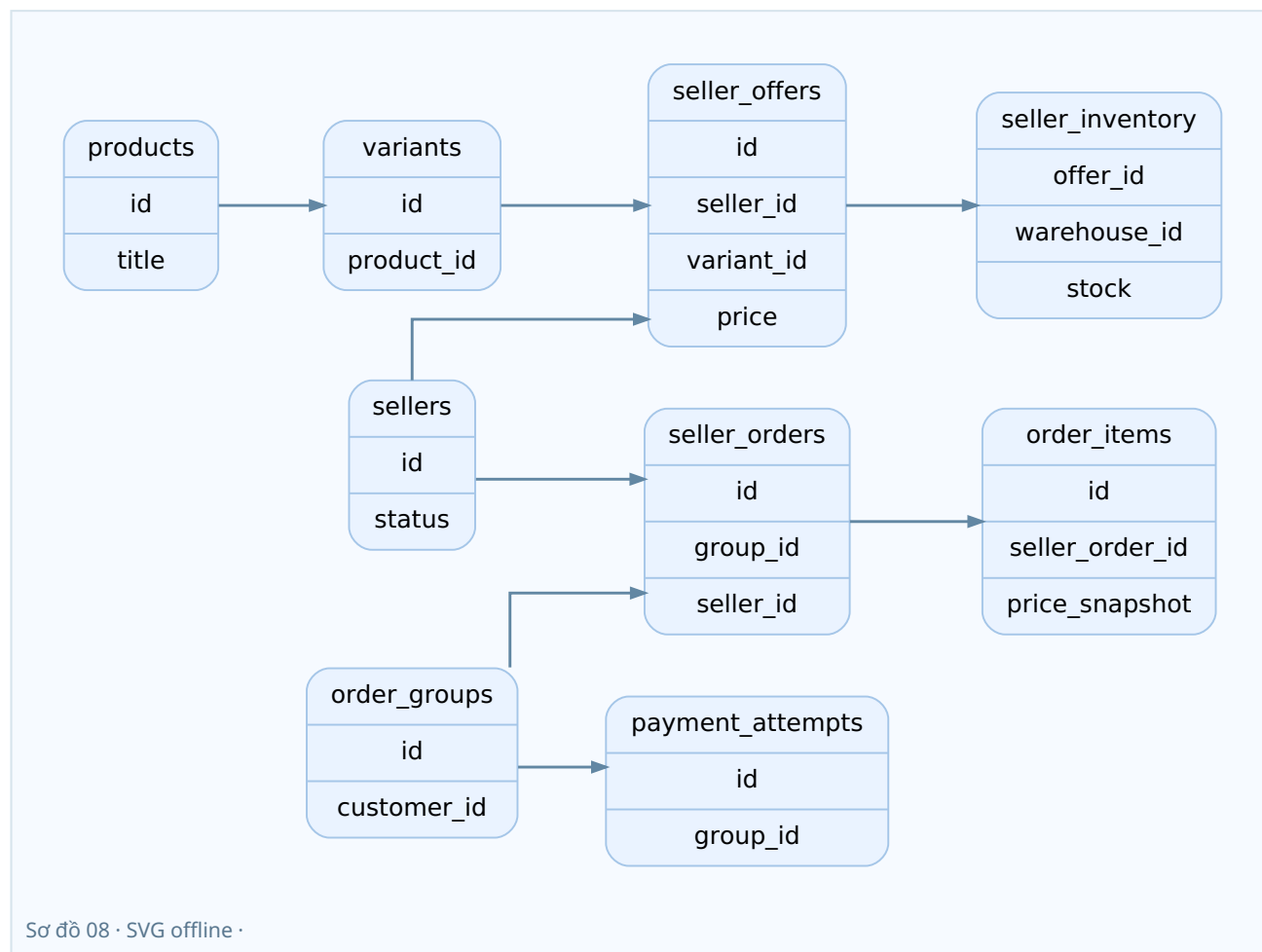
Nên có các lớp bảo vệ:

- AuthN: người gọi là ai?
- AuthZ: actor có quyền đối với seller này không?
- Ownership check: bản ghi này có thuộc seller được phép không?
- Audit log: ai sửa giá, kho, trạng thái đơn, bank payout settings?
- Tests: truy cập chéo seller phải bị từ chối.

PostgreSQL Row-Level Security có thể là một lớp phòng vệ bổ sung tùy kiến trúc; không thay thế hoàn toàn authorization tại application.

## 2.8. Schema tham khảo cho Marketplace — không bắt đầu bằng microservices

Bạn vẫn có thể xây marketplace MVP bằng một Spring Boot modular monolith và PostgreSQL. Khác biệt đầu tiên cần xử lý là **mô hình nghiệp vụ**, không nhất thiết là số lượng service.



Schema SQL tối giản dưới đây chỉ minh họa **ownership và ranh giới order**, không phải migration production hoàn chỉnh:

```
CREATE TABLE sellers (
  id BIGINT PRIMARY KEY,
  name TEXT NOT NULL,
  status VARCHAR(32) NOT NULL
);

CREATE TABLE products (
  id BIGINT PRIMARY KEY,
  title TEXT NOT NULL
);

CREATE TABLE product_variants (
  id BIGINT PRIMARY KEY,
  product_id BIGINT NOT NULL REFERENCES products(id),
  variant_code TEXT NOT NULL UNIQUE
);

CREATE TABLE seller_offers (
```

```

    id BIGINT PRIMARY KEY,
    seller_id BIGINT NOT NULL REFERENCES sellers(id),
    variant_id BIGINT NOT NULL REFERENCES product_variants(id),
    price_vnd BIGINT NOT NULL CHECK (price_vnd >= 0),
    status VARCHAR(32) NOT NULL
);

CREATE TABLE order_groups (
    id BIGINT PRIMARY KEY,
    customer_id BIGINT NOT NULL,
    status VARCHAR(32) NOT NULL,
    total_vnd BIGINT NOT NULL CHECK (total_vnd >= 0)
);

CREATE TABLE seller_orders (
    id BIGINT PRIMARY KEY,
    order_group_id BIGINT NOT NULL REFERENCES order_groups(id),
    seller_id BIGINT NOT NULL REFERENCES sellers(id),
    status VARCHAR(32) NOT NULL,
    UNIQUE (id, seller_id)
);

CREATE TABLE order_items (
    id BIGINT PRIMARY KEY,
    seller_order_id BIGINT NOT NULL REFERENCES seller_orders(id),
    offer_id BIGINT NOT NULL REFERENCES seller_offers(id),
    product_title_snapshot TEXT NOT NULL,
    unit_price_vnd BIGINT NOT NULL CHECK (unit_price_vnd >= 0),
    quantity INT NOT NULL CHECK (quantity > 0)
);

```

**Điểm thiết kế cần chú ý:** `order_items.offer_id` giúp truy ngược offer gốc, nhưng `product_title_snapshot` và `unit_price_vnd` bảo vệ dữ liệu giao dịch khi seller đổi listing. Bảng inventory phải chứa seller/offer + warehouse + stock state nếu bạn cho phép nhiều kho. Với tiền, có thể dùng số nguyên đơn vị tiền nhỏ nhất được provider hỗ trợ hoặc `NUMERIC` theo currency; ví dụ dùng `BIGINT VND` để tránh float trong bài thực hành.

Để bảo đảm `order_items.offer_id` thực sự thuộc seller của `seller_order_id`, có thể xây composite foreign key/schema chặt hơn hoặc validate nhất quán bằng domain service và test. Schema minh họa chưa thực thi invariant này ở DB; **đừng xem nó như dữ liệu an toàn tuyệt đối.**

## API tối thiểu

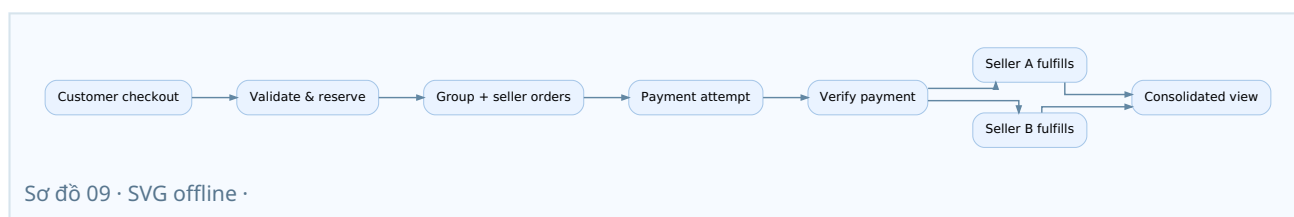
```

GET    /products/{productId}
GET    /products/{productId}/offers
POST   /checkout
GET    /orders/{orderGroupId}
GET    /seller/orders
PATCH /seller/offers/{offerId}
POST   /seller/orders/{sellerOrderId}/confirm
POST   /seller/orders/{sellerOrderId}/shipments

```

**Phân quyền bằng actor + ownership**, không chỉ phân theo HTTP path hoặc role `SELLER` chung.

## 2.9. Sequence: một checkout, hai seller, hai lần giao hàng



Sơ đồ cố tình không đặt lệnh gọi payment provider bên trong một DB transaction kéo dài. Khi provider timeout, checkout phải có trạng thái có thể khôi phục hoặc đối soát, không tự động coi tất cả là thất bại.

### Câu hỏi về trạng thái tổng hợp

- Seller A đã giao, Seller B đang chuẩn bị: order group là `PARTIALLY_FULFILLED` hay `IN_PROGRESS`?
- Seller A hủy vì hết hàng, Seller B đã giao: refund phần nào, phí ship tính lại không?
- Một seller order được chia thành hai shipment: seller order `SHIPPED` khi một kiện rời kho hay khi tất cả rời kho?

Không gộp bừa `order_group.status = min(child_status)` hoặc `max(child_status)`. Hãy định nghĩa **state aggregation policy** bằng business rule rõ ràng, có test cho các tổ hợp trạng thái.

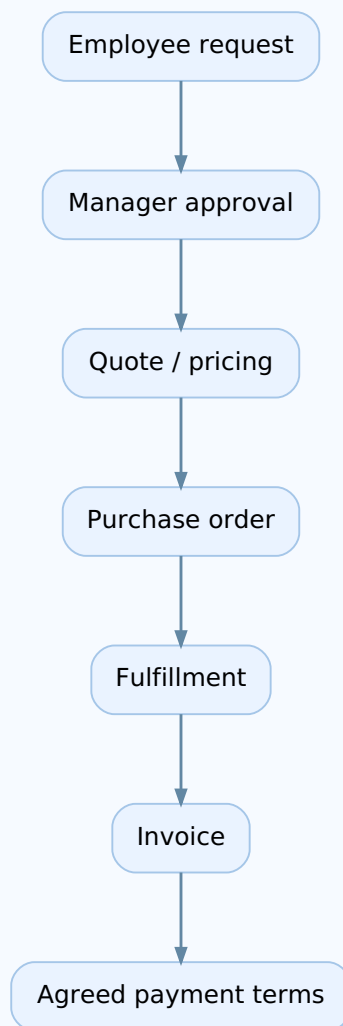
## 2.10. Business-model decisions → System-design consequences

| Quyết định kinh doanh        | Hệ quả domain/data                      | Hệ quả hệ thống                                    |
|------------------------------|-----------------------------------------|----------------------------------------------------|
| Mở third-party sellers       | Sellers, offers, seller inventory       | Seller onboarding, authZ theo seller               |
| Cho nhiều seller cùng SKU    | Canonical variant + nhiều offer         | Offer selection, price competition, offer identity |
| Một checkout nhiều seller    | Order group + seller orders             | Partial cancellation/shipment/refund               |
| Platform thu hoa hồng        | Commission rules + payable ledger       | Reconciliation, fee adjustment                     |
| Seller tự giao hàng          | Shipment ownership / seller events      | Webhook/integration, stale delivery status         |
| Platform giữ stock hộ seller | Warehouse location + stock ownership    | Allocation, transfers, physical vs book inventory  |
| B2B bán theo hợp đồng        | Organization, buyer roles, quotes       | Approval workflow, payment terms, credit limits    |
| Cho hàng đã qua sử dụng      | Listing-specific condition and identity | Moderation, trust, dispute, no forced SKU matching |

Một anti-pattern thường gặp là quyết định công nghệ trước: “Dùng Kafka và 12 microservices cho giống sàn lớn.” Bạn có thể gặp 12 service nhưng **chưa phân biệt seller order với buyer order**. Đó không phải vấn đề scale; đó là domain sai.

## 2.11. B2B: khi người mua không còn là một cá nhân

B2C thường có customer đăng nhập và tự quyết định checkout. Trong B2B, công ty mua hàng có thể có người tạo yêu cầu, quản lý duyệt, nhân viên tài chính xác nhận hóa đơn và điều khoản thanh toán sau 30 ngày.



Sơ đồ 10 · SVG offline ·

Những domain bổ sung có thể là:

- organization, organization\_members, buyer\_roles.
- quotes, quote\_lines, price\_lists, volume\_tiers.
- purchase\_orders, approval\_steps, credit\_terms.
- invoices, receivables, payment\_allocations.

B2B không mặc định phải có tất cả; mô hình mỗi doanh nghiệp khác nhau. Điểm cốt lõi: **đơn vị mua là một tổ chức có quy trình phê duyệt, không đơn thuần là user với giỏ hàng lớn hơn.**

## 2.12. Failure scenarios: chuyển đổi business model thất bại như thế nào?

| Failure scenario                                | Root cause thường gặp        | Điều cần bảo vệ            |
|-------------------------------------------------|------------------------------|----------------------------|
| Seller A sửa offer của Seller B                 | API chỉ kiểm tra role SELLER | Ownership/tenant isolation |
| Một seller hủy làm cả đơn group thành CANCELLED | Gộp trạng thái cha-con sai   | Aggregation policy         |

| Failure scenario                           | Root cause thường gặp                    | Điều cần bảo vệ                         |
|--------------------------------------------|------------------------------------------|-----------------------------------------|
| Seller đổi giá làm đơn cũ đổi theo         | Order item chỉ join giá hiện tại         | Price snapshot                          |
| Refund một item làm hoàn toàn bộ checkout  | Refund chỉ gắn group                     | Refund granularity theo item/amount     |
| Platform chuyển seller hai lần             | Retry thiếu idempotency                  | Transfer dedup + reconciliation         |
| Seller nhận tiền dù kiện chưa đủ điều kiện | Dùng PAID làm trigger payout duy nhất    | Settlement eligibility                  |
| Một offer được bán vượt số lượng           | Inventory gộp theo canonical product     | Seller/offer/warehouse stock invariants |
| Seller bị khóa vẫn gọi API bán hàng        | Chỉ kiểm tra token không kiểm tra status | Seller lifecycle authZ                  |
| Đồng bộ SKU trùng tên làm bán nhầm hàng    | Matching bằng text hoặc slug             | Stable product/variant identifiers      |

## 2.13. Engineering Lab — Nâng cấp Hoàng Phone thành marketplace

### Bài toán

Bạn có một Spring Boot modular monolith từ chương 01. Nhiệm vụ: cho hai seller đăng bán cùng một điện thoại, khách checkout một lần, từng seller giao riêng và platform thu commission 10% (chỉ để thực hành, bỏ qua thuế/phí khác).

#### Acceptance criteria:

1. Seller A không đọc/sửa order hay offer của Seller B.
2. Hai offer cùng variant có hai mức giá và inventory riêng.
3. Checkout tạo đúng một order group và hai seller orders khi giỏ có hai seller.
4. Order item giữ giá snapshot dù offer sau đó đổi giá.
5. Seller A **SHIPPED**, Seller B **PENDING** không tự biến order group thành **DELIVERED**.
6. Payment được xử lý bằng idempotency key; không tạo tác động tài chính trùng khi nhận lại webhook.
7. Refund một seller order không hoàn nhầm seller order còn lại.
8. Seller phải đủ điều kiện hoạt động mới được công khai offer.

### Lab A — Ma trận trách nhiệm

Điền cho từng vai trò: customer, seller, platform operator, payment provider, carrier: ai **được ghi giá**, ai **giữ stock**, ai **xác nhận giao hàng**, ai **xử lý refund**. Viết rõ những phần tùy hợp đồng.

### Lab B — Thiết kế database

Từ các bảng `products`, `orders`, `order_items` của chương 01, thiết kế migration sang `seller_offers`, `seller_orders`, `seller_inventory`. Không làm mất order history hoặc thay giá đơn cũ.

## Lab C — Viết API authorization tests

```
Given Seller A owns offer 101
When Seller B PATCH /seller/offers/101
Then 403 (or 404 masking existence, per API policy)
And original offer stays unchanged
```

## Lab D — Simulate partial cancellation

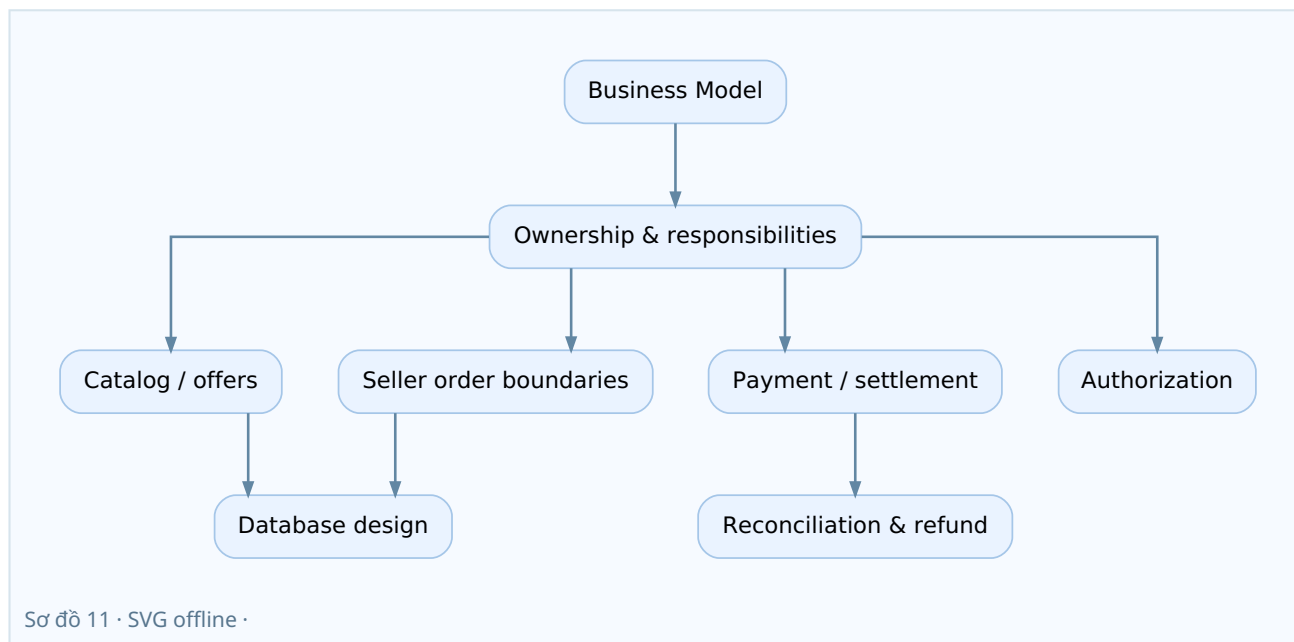
Seller A hủy do hết hàng sau khi Seller B đã đóng gói. Hãy mô tả state transition, inventory release, refund amount và những việc không được phép rollback ngược với Seller B.

## Lab E — Commission arithmetic

Cart A=2.000.000, B=1.000.000, platform fee=10%, voucher platform-funded=100.000. Tính buyer paid, seller payable, platform amount trước payment fee/tax/shipping. Sau đó đổi voucher thành seller-funded để quan sát trách nhiệm thay đổi.

**Đáp án tham khảo Lab E (platform-funded):** Buyer paid 2.900.000; seller payables 1.800.000 + 900.000 = 2.700.000; phần platform còn 200.000 trước các chi phí khác. Với seller-funded voucher, kết quả phụ thuộc **quy tắc phân bổ voucher theo item/seller và thứ tự tính commission**; phải xác định policy trước khi tính.

## Knowledge Map



## Key Takeaways

1. **B2B/B2C/C2C, D2C và 1P/3P** là các trục phân loại khác nhau. Đừng chọn schema dựa trên một nhãn marketing.
2. Marketplace là bài toán **đa chủ thể và ownership**, không chỉ “website có nhiều sản phẩm”.

3. Product, variant, offer và inventory cần được phân biệt; cùng SKU có thể có nhiều seller offer và tồn kho khác nhau.
  4. Một checkout có thể tạo một order group và nhiều seller order; giao hàng, refund, payout có thể diễn ra không đồng thời.
  5. PAID không có nghĩa DELIVERED; DELIVERED không có nghĩa tiền đã PAYOUT\_COMPLETED.
  6. Seller authorization và dữ liệu chéo tenant là **business invariant**, không phải chi tiết UI.
  7. Hãy làm đúng domain trong modular monolith trước; microservices, message broker và distributed transactions chỉ xuất hiện khi bài toán cần.
- 

## Tài liệu tham khảo và giới hạn áp dụng

---

- [S1] Stripe Connect — nền tảng/marketplace, các payment flows và payouts: <https://stripe.com/connect>
- [S2] Stripe Docs — Separate charges and transfers: <https://docs.stripe.com/connect/separate-charges-and-transfers>
- [S3] Stripe — Marketplace payment processing guide (updated 2026-05-05): <https://stripe.com/resources/more/marketplace-payment-processing-apis>
- [S4] AWS — Guidance for Third-Party Marketplace on AWS: <https://docs.aws.amazon.com/solutions/third-party-marketplace-on-aws/>

Tài liệu bên trên được dùng để kiểm chứng **khái niệm marketplace, seller fulfillment, luồng tiền nhiều bên**. Ví dụ schema, amount, state machine và quyết định domain trong chương là mô phỏng để học thiết kế, không phải cam kết rằng một provider hay một quốc gia hỗ trợ tất cả các luồng trong ví dụ.

**Chương tiếp theo:** Chapter 03 — Domain Modeling: Product, SKU, Offer, Inventory, Cart, Order, Payment và ranh giới trách nhiệm giữa chúng.

Inside E-commerce · Volume 1 · Chapter 02 · Bản đọc offline. Các sơ đồ SVG lưu kèm thư mục assets/chapter-02.

# INSIDE E-COMMERCE

## Volume 1 — Understanding E-commerce

### Chapter 03 — Domain Modeling: Từ nghiệp vụ đến mô hình dữ liệu và backend

**Câu hỏi dẫn đường:** Tại sao một bảng `products(id, name, price, stock)` và `orders(id, user_id, product_id, status)` có thể chạy trong demo nhưng trở nên khó sửa khi thêm màu sắc, nhiều seller, nhiều kho, voucher, thanh toán lại và hoàn hàng một phần?

**Phạm vi:** Chúng ta tiếp tục dự án giả định **Hoàng Phone → Hoàng Marketplace** của chương 01–02. Ví dụ dùng PostgreSQL và Java/Spring Boot, không tuyên bố đây là schema duy nhất. Quy tắc thuế, hoàn tiền và thanh toán thực tế phụ thuộc mô hình vận hành. Những đoạn code được ghi “minh họa” không phải repository production hoàn chỉnh.

### Sau chương này, bạn có thể

- Chuyển phát biểu kinh doanh thành **ubiquitous language, use cases, rules và invariants** trước khi vẽ bảng.
- Phân biệt **Product, Product Variant/SKU, Seller Offer, Stock, Reservation, Cart, Order Group, Seller Order, Order Item, Payment Attempt**.
- Nhận ra **Entity, Value Object, Aggregate, Aggregate Root, Bounded Context và Domain Event**; biết khi nào không cần áp dụng DDD đầy đủ.
- Xác định ranh giới transaction: dữ liệu nào cần nhất quán tức thì; dữ liệu nào có thể đồng bộ qua sự kiện.
- Viết schema PostgreSQL mẫu có PK/FK/UNIQUE/CHECK, snapshot giá, bảng reservation và chống trùng request.
- Tổ chức một Spring Boot **modular monolith** không biến mọi bảng thành REST CRUD public.
- Nhận biết lỗ hổng trong mô hình qua test scenario và câu hỏi của một system designer.

### 3.1. Bad case: bắt đầu bằng bảng, không bắt đầu bằng nghiệp vụ

Một developer nhận yêu cầu “xây website bán điện thoại”, rồi viết ngay:

```
CREATE TABLE products (
  id BIGINT PRIMARY KEY,
  name TEXT NOT NULL,
  price NUMERIC(18, 2) NOT NULL,
```

```

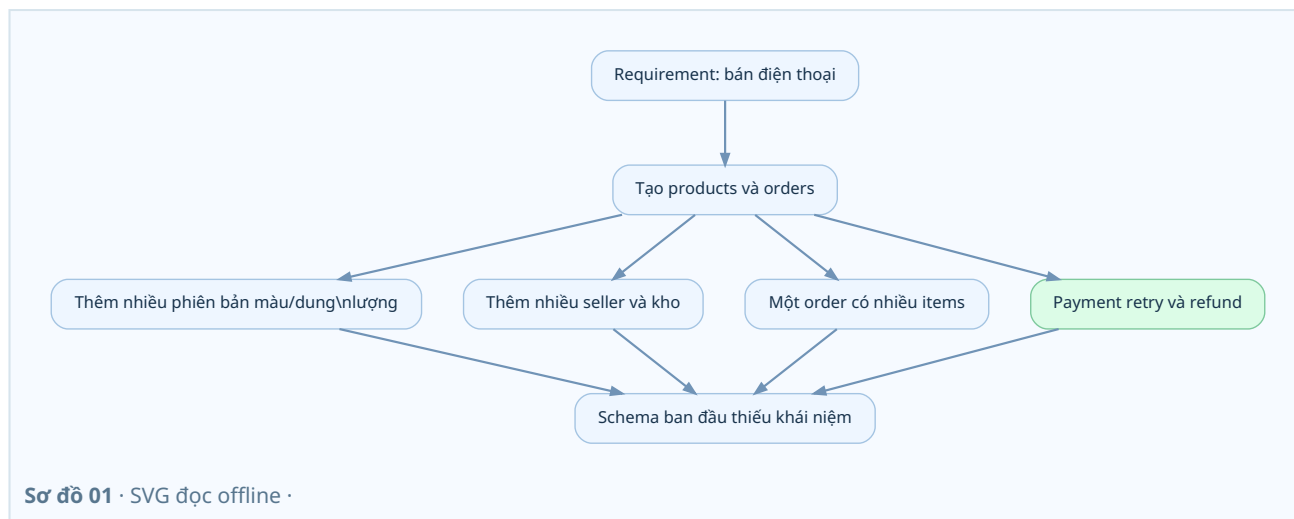
stock INTEGER NOT NULL
);

CREATE TABLE orders (
  id BIGINT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  product_id BIGINT NOT NULL,
  status TEXT NOT NULL
);

```

Với demo “mỗi đơn mua một sản phẩm, một người bán, một kho, không khuyến mãi”, nó có thể đủ để bắt đầu. Nhưng **database đang ngầm chứa một tập giả định nghiệp vụ**:

1. `products.price`: chỉ có một mức giá cho mọi seller và mọi kênh bán.
2. `products.stock`: chỉ có một con số tồn kho, không phân biệt kho hay hàng đã giữ.
3. `orders.product_id`: mỗi đơn chỉ có đúng một mặt hàng.
4. `orders.status`: một trạng thái đủ mô tả thanh toán, đóng gói, vận chuyển và hoàn hàng.
5. Không có dữ liệu thời điểm mua; giá đổi sẽ khiến ta khó giải thích tổng tiền của đơn cũ.



**Nỗi đau không phải SQL viết sai. Nỗi đau là ta đã thiết kế dữ liệu khi chưa biết nó phải đại diện cho hiện thực nào.**

## Một thực hành tốt hơn

Trước mỗi entity, hỏi: “Ai ra quyết định? Quy tắc nào không được phá vỡ? Đối tượng này sống bao lâu? Có thay đổi độc lập với đối tượng khác không?”

| Phát biểu từ business                            | Khái niệm cần xác định     | Câu hỏi thiết kế                                       |
|--------------------------------------------------|----------------------------|--------------------------------------------------------|
| “Cùng điện thoại, mỗi màu có số lượng khác nhau” | Variant / SKU              | Cái gì định danh từng phiên bản bán được?              |
| “Seller A và B bán cùng một máy, giá khác nhau”  | Offer                      | Giá thuộc catalog chung hay lời chào bán của seller?   |
| “Hàng đã giữ 15 phút”                            | Reservation                | Giữ bao nhiêu, đến lúc nào, ai sở hữu việc giải phóng? |
| “Giỏ có hàng của hai seller”                     | Order Group / Seller Order | Ai thực hiện từng phần đơn hàng?                       |
| “Khách đã trả tiền nhưng chưa giao hàng”         | Payment / Fulfillment      | Vì sao không thể dùng một status duy nhất?             |

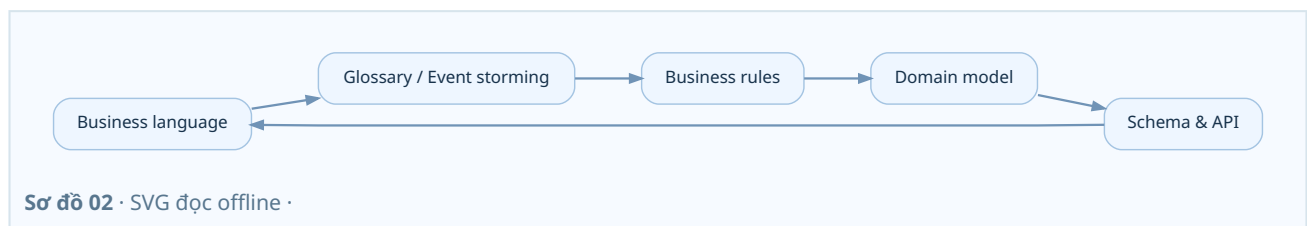
## 3.2. Ubiquitous Language — thống nhất từ vựng trước khi code

Trong Domain-Driven Design, **ubiquitous language** là ngôn ngữ chung được dùng nhất quán giữa business và engineering trong một phạm vi nghiệp vụ. Nó không đồng nghĩa “dịch mọi tên sang tiếng Anh”, mà là làm rõ nghĩa của từ.

Ví dụ business nói: “Còn 10 máy”. Engineer phải hỏi: 10 máy **thực tế trong kho**, 10 máy **được phép bán**, hay 10 máy **đã trừ lượng giữ hàng**?

| Từ trong dự án  | Định nghĩa vận hành trong ví dụ                                             |
|-----------------|-----------------------------------------------------------------------------|
| Product         | Thông tin chung về một dòng hàng được nhận diện trong catalog               |
| Variant / SKU   | Biến thể định danh đơn vị hàng có thể bán: màu + dung lượng                 |
| Seller Offer    | Lời chào bán của một seller cho một variant: giá và điều kiện bán           |
| On-hand Stock   | Số lượng vật lý ghi nhận trong kho theo định nghĩa kế toán kho của hệ thống |
| Reserved Stock  | Số lượng đang được giữ cho yêu cầu mua hợp lệ                               |
| Available Stock | Khả năng tiếp tục bán được suy ra từ các số liệu tồn kho                    |
| Cart Item       | Ý định mua; thường chưa giữ tồn kho                                         |
| Order Item      | Dòng hàng trong giao dịch đã được chốt, có dữ liệu snapshot                 |
| Payment Attempt | Một lần thử thực hiện thanh toán cho đơn/nhóm đơn                           |

**Lưu ý thực tế:** Nhiều doanh nghiệp dùng “SKU” để chỉ mã hàng do seller tự đặt, trong khi một marketplace có thể dùng mã variant chung riêng với seller SKU. Ta phải ghi rõ hai namespace, không tranh luận rằng chỉ có một định nghĩa SKU duy nhất.



### Kỹ thuật hỏi để khám phá domain

Với mọi khái niệm, hỏi năm câu: **nó là gì; ai sở hữu; khi nào sinh ra; khi nào thay đổi; khi nào chấm dứt**. Ví dụ `Reservation` sinh ra lúc checkout được chấp nhận, tồn tại trong trạng thái `ACTIVE`, sau đó chuyển `CONFIRMED`, `RELEASED` hoặc `EXPIRED` theo quy tắc. Có một `reservation_id` riêng sẽ giúp trace và chống hoàn hàng hai lần.

## 3.3. Entity khác Value Object ra sao?

### Entity: thứ cần theo dõi danh tính

Hai đơn hàng có cùng customer, số tiền và sản phẩm vẫn là **hai đơn khác nhau** vì có `order_id` khác nhau. Danh tính và vòng đời là trọng tâm. Tương tự, hai `Payment Attempt` cùng số tiền vẫn khác nhau.

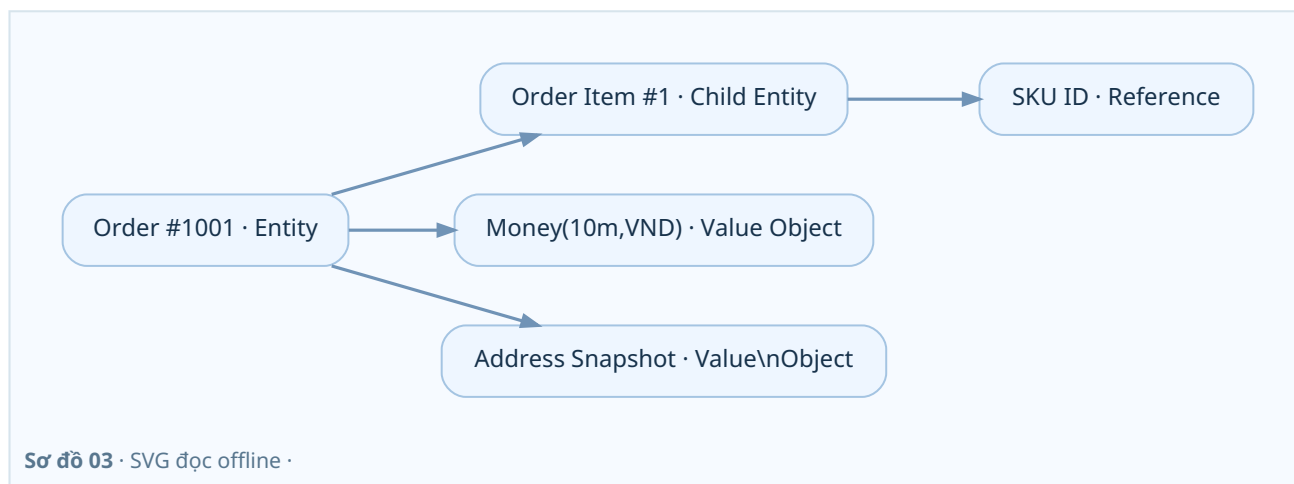
## Value Object: thứ được định nghĩa bằng giá trị

`Money(10000000, "VND")` và `Money(10000000, "VND")` có thể được coi là cùng một giá trị.

`ShippingAddressSnapshot` thường được lưu như giá trị trong order vì sau khi khách thay đổi số địa chỉ, địa chỉ giao hàng của order đã chốt không được tự thay đổi.

| Khái niệm                    | Thường mô hình hóa                         | Lý do                                                |
|------------------------------|--------------------------------------------|------------------------------------------------------|
| Order                        | Entity                                     | Có identity, trạng thái, lịch sử                     |
| Order Item                   | Entity con của Order Aggregate trong ví dụ | Cần định danh dòng hàng để hủy/hoàn một phần         |
| Payment Attempt              | Entity                                     | Có vòng đời và mã giao dịch riêng                    |
| Money                        | Value Object                               | Giá trị + đơn vị tiền; arithmetic phải đúng currency |
| Address trong Order          | Value Object snapshot                      | Không tự thay đổi khi profile đổi                    |
| Address Book Entry của khách | Có thể là Entity                           | Nếu sửa/xóa/quản lý theo ID qua thời gian            |

**Không đánh đồng “Entity trong DDD” với `@Entity` JPA.** `@Entity` là mapping cho persistence; một domain Value Object có thể được lưu nhiều cột, và một domain Entity có thể cần nhiều bảng để biểu diễn. Tài liệu Microsoft nhấn mạnh identity của Entity và đặc tính không có danh tính độc lập của Value Object [R1, R2].



## Money: tránh `double`

Số tiền phải được tính bằng kiểu có độ chính xác phù hợp (`BigDecimal` trong Java và `NUMERIC` trong PostgreSQL) hoặc đơn vị tiền nguyên nhỏ nhất theo chính sách. `double` là floating point nhị phân, không phù hợp để lưu kết quả thanh toán chính xác [R4]. `Money` phải mang theo currency; không được cộng VND với USD mà bỏ qua quy đổi và tỷ giá.

Ví dụ Value Object tối giản:

```
public record Money(BigDecimal amount, String currency) {
    public Money {
        Objects.requireNonNull(amount);
        Objects.requireNonNull(currency);
        if (currency.isBlank()) throw new IllegalArgumentException("currency");
    }
}
```

```

public Money add(Money other) {
    if (!currency.equals(other.currency())) {
        throw new IllegalArgumentException("Currency mismatch");
    }
    return new Money(amount.add(other.amount()), currency);
}
}

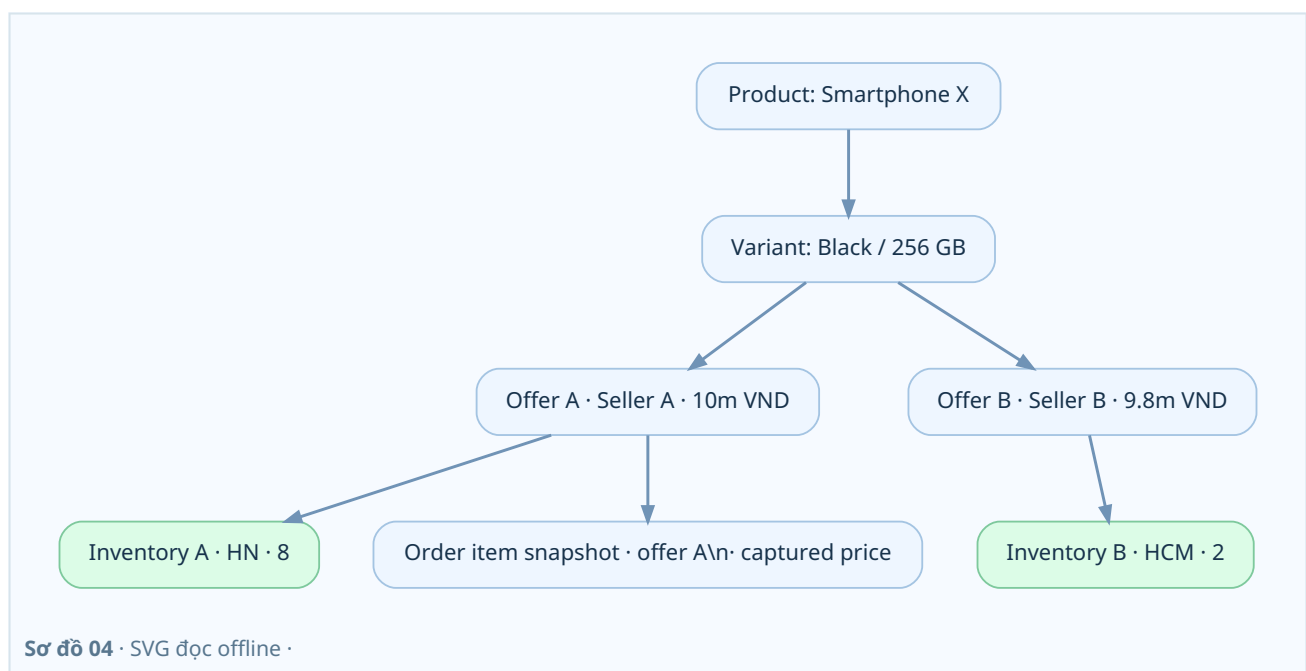
```

Code là minh họa ý tưởng; production còn phải chuẩn hóa currency, scale, rounding và giới hạn số tiền.

### 3.4. Tách Product / Variant / Offer / Inventory / Order Snapshot

Đây là bản nâng cấp mô hình từ chương 02. Hãy tưởng tượng mẫu Smartphone X được bán bởi hai seller:

| Thành phần  | Giá trị ví dụ                      |
|-------------|------------------------------------|
| Product     | Smartphone X                       |
| Variant     | Black / 256GB                      |
| Offer A     | Seller A, 10.000.000 VND           |
| Offer B     | Seller B, 9.800.000 VND            |
| Inventory A | Seller A ở kho HN: 8 còn bán được  |
| Inventory B | Seller B ở kho HCM: 2 còn bán được |



**Vì sao price không nhất thiết đặt trong products?** Cùng một định danh catalog có thể xuất hiện trong nhiều lời chào bán khác nhau. Giá có thể phụ thuộc seller, thời điểm, đối tượng mua, số lượng hay chương trình giảm giá. Mô hình chi tiết của Pricing Engine sẽ được khai thác sâu hơn ở chương 04.

**Vì sao inventory không đơn giản là products.stock?** Một offer có thể cung cấp từ nhiều kho. Một kho có thể có số hàng on-hand nhưng một phần đã reserved. Tồn kho thuộc **đơn vị có thể bán và nguồn cung** chứ không thuộc mô tả sản phẩm chung.

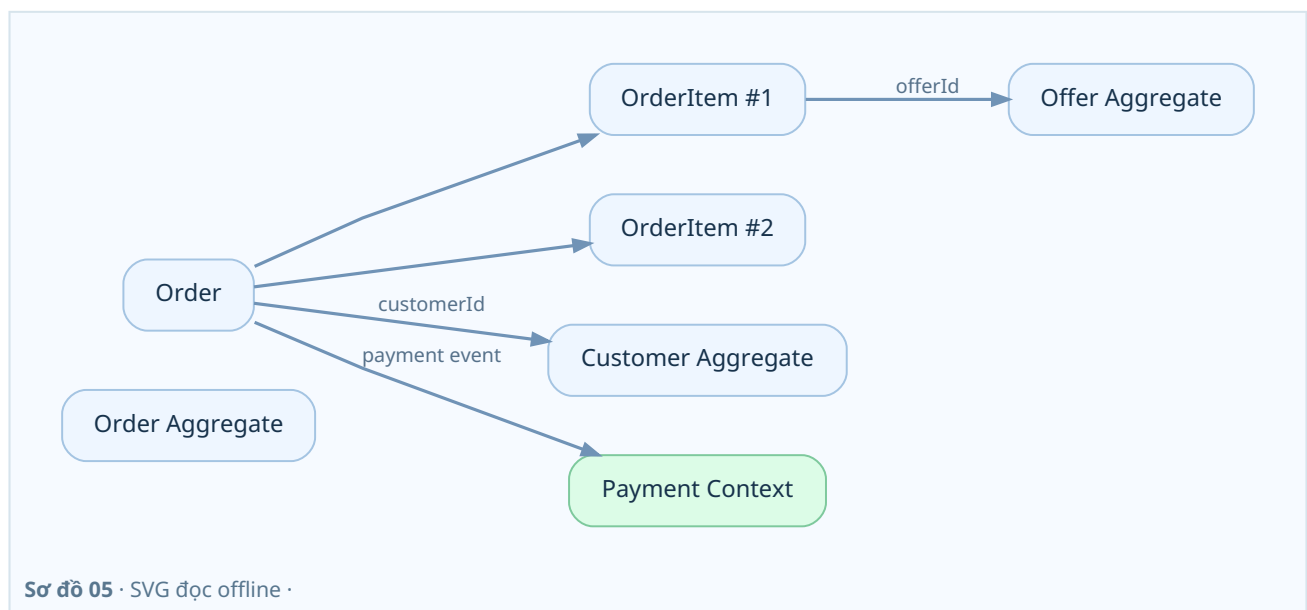
**Vì sao Order Item lưu snapshot?** `offer_id` chỉ là tham chiếu về đối tượng gốc. Order Item phải ghi `sku_code_snapshot`, `title_snapshot`, `unit_price`, `currency`, `discount/tax basis` phù hợp và số lượng để tái dựng giao dịch lịch sử. Không lấy trực tiếp `offers.price` hiện tại để tính lại đơn cũ.

Với sản phẩm đồ cũ độc nhất, có thể không cần canonical product matching. Listing có thể đại diện trực tiếp cho hàng bán; mô hình phải phục vụ business, không bắt business chạy theo sơ đồ mẫu.

### 3.5. Quan hệ giữa các domain: không phải mọi mũi tên là foreign key

Có ít nhất bốn loại quan hệ cần phân biệt:

1. **Composition trong aggregate:** Order quản lý OrderItem.
2. **Identity reference giữa aggregates:** OrderItem lưu `offerId` để biết mua lời chào bán nào, nhưng dữ liệu hợp đồng đã chốt nằm trong snapshot.
3. **Database foreign key:** đảm bảo ID liên quan tồn tại nếu cùng DB và phù hợp lifecycle; không thay được mọi business rule.
4. **Integration relationship:** Order thông báo Payment/Fulfillment qua event/API; không nhất thiết dùng FK hoặc chung database.



#### Bad case: JPA graph xuyên toàn bộ hệ thống

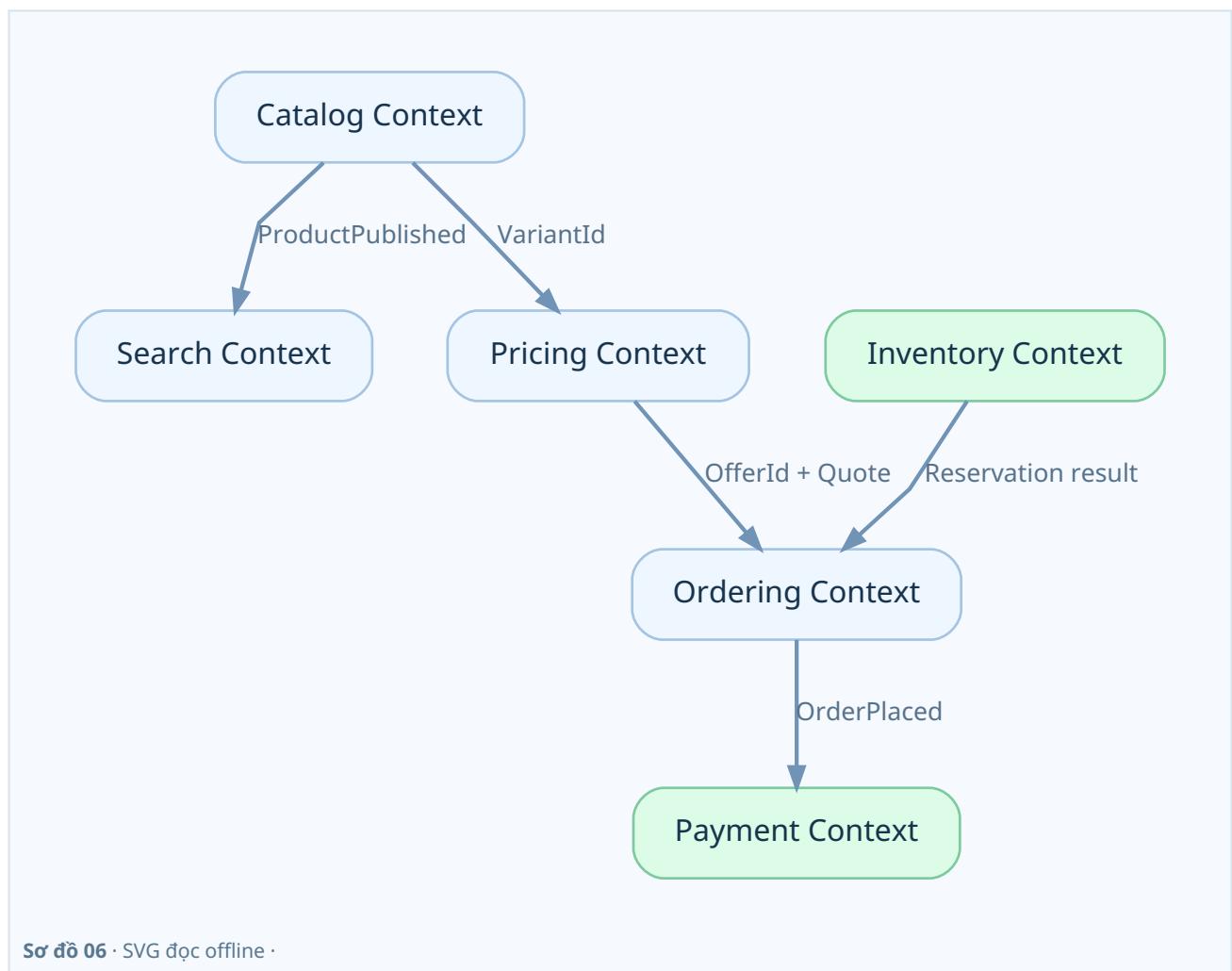
Nếu Order có @ManyToOne Customer, mỗi item @ManyToOne Offer, Offer @ManyToOne Product, Product @OneToOne Reviews và API serialize nguyên entity, bạn có thể gặp query N+1, vòng lặp JSON, lazy loading ngoài transaction và coupling giữa module.

Thay vào đó, request/response API nên dùng DTO rõ mục đích; aggregate liên hệ qua ID khi thích hợp; truy vấn đọc có thể join/projection riêng, không bị buộc phải đi qua domain entity graph.

### 3.6. Bounded Context: cùng một từ có thể mang nhiều ý nghĩa

Trong một E-commerce, từ `Product` xuất hiện ở nhiều nơi nhưng không phải chỗ nào cũng cần cùng một model:

| Context   | Cần biết gì về sản phẩm?                                  |
|-----------|-----------------------------------------------------------|
| Catalog   | Tên, mô tả, ảnh, thuộc tính, category                     |
| Pricing   | Đối tượng định giá, currency, chính sách và phiên bản giá |
| Inventory | Mã hàng/offer và nguồn cung theo kho                      |
| Ordering  | Snapshot mặt hàng mà khách đã mua                         |
| Search    | Document được tối ưu tìm kiếm, có thể chậm hơn Catalog    |



**Bounded Context không đồng nghĩa microservice.** Trong dự án đầu tiên, năm context vẫn có thể nằm trong một Spring Boot và một database; boundaries thể hiện ở package/module, API nội bộ, quyền ghi và quy tắc phụ thuộc. Tài liệu Microsoft nêu việc tách model theo context, và xem microservice là một lựa chọn triển khai boundary chứ không phải định nghĩa của domain [R1, R2].

## Context ownership matrix

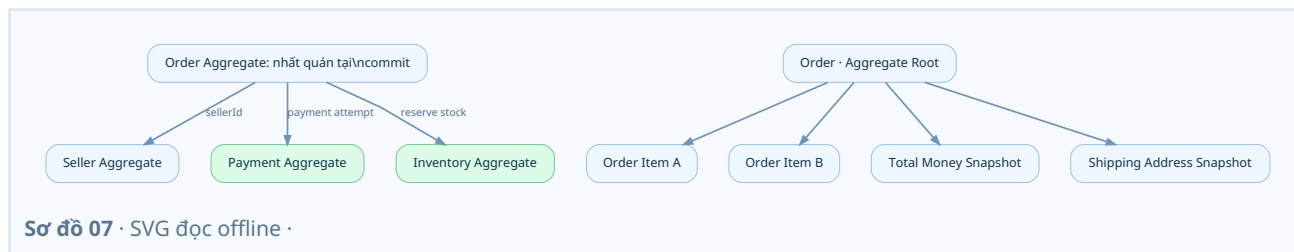
| Dữ liệu / quyết định         | Owner trong thiết kế ví dụ                      |
|------------------------------|-------------------------------------------------|
| Product description          | Catalog                                         |
| Seller offer price           | Pricing / Offer management theo quyết định team |
| On-hand và reserved quantity | Inventory                                       |
| Confirmed order total        | Ordering                                        |
| Provider transaction status  | Payments                                        |
| Parcel / shipping label      | Fulfillment                                     |

**Quy tắc:** cùng một dữ liệu có thể được sao chép phục vụ READ nhưng không nên có nhiều nơi tùy ý chỉnh cùng một sự thật nghiệp vụ.

## 3.7. Aggregate và Aggregate Root: từ “cụm bảng” sang “ranh giới tính đúng đắn”

Một Aggregate là nhóm đối tượng nghiệp vụ phải giữ **một tập invariant nhất quán** khi được thay đổi. Aggregate Root là cửa vào cho các thao tác cập nhật nhóm đó. Nó có thể chỉ có **một entity**, không bắt buộc là 10 bảng. Đây là cách tài liệu tactical DDD mô tả transactional consistency boundaries [R1, R2].

Ví dụ với `Order`:



### Các invariant ví dụ trong Order:

- Không tạo order không có item nếu business không cho phép.
- `quantity > 0` cho mỗi dòng hàng.
- Các item và tổng tiền là dữ liệu snapshot nhất quán theo chính sách tính tiền.
- Không chuyển `CANCELLED` → `PAID` chỉ bằng một lệnh tùy ý; mọi state transition phải được định nghĩa.

**Inventory aggregate** bảo vệ bất biến riêng: `reserved_quantity <= on_hand_quantity` và không được trừ/giải phóng vượt số lượng. Lưu ý, CHECK trên stock row không tự bảo đảm tổng `quantity` ở bảng reservations luôn bằng `reserved_quantity`; các bước tạo, xác nhận và giải phóng reservation phải cập nhật đồng bộ theo transaction. **Payment aggregate** bảo vệ chống xử lý trùng provider transaction và tính hợp lệ của payment state.

### Có phải một `@Transactional` = một Aggregate?

**Không.** Aggregate là ranh giới mô hình nghiệp vụ. Database transaction là công cụ persistence để giữ tính nhất quán. Một use case có thể cần cập nhật nhiều aggregates trong cùng một database transaction, chẳng hạn tạo Order và đặt Reservation, nếu phù hợp độ phức tạp và lock behavior. Điều đó không bắt hai

aggregates phải gộp thành một object graph. Khi đã tách database/service, ta không còn mặc định một local ACID transaction bao phủ được mọi nơi.

### 3.8. Business Invariants: chuyển từ câu nói thành luật có thể kiểm thử

Nhiều developer viết `validate()` rải rác trong controller, rồi tưởng rằng đã bảo vệ dữ liệu. Câu hỏi đúng là: **điều kiện nào phải đúng kể cả khi 20 Pod và 100 Worker gọi đồng thời?**

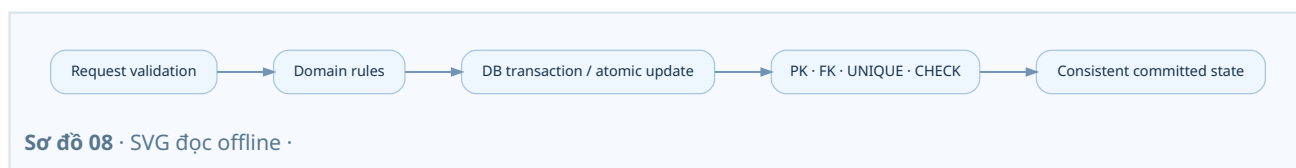
| Invariant                                        | Cách bảo vệ khả thi                        | Ranh giới        |
|--------------------------------------------------|--------------------------------------------|------------------|
| Offer price không âm                             | Domain validation + DB CHECK               | Offer            |
| Stock reserved không vượt on-hand                | Atomic DB update + CHECK                   | Inventory row    |
| Order item quantity dương                        | Domain validation + DB CHECK               | Order            |
| Client retry không tạo đơn lần hai               | Idempotency key + UNIQUE + transaction     | Ordering         |
| Order total không dùng giá hiện hành để tính lại | Price snapshot + server-side calculation   | Ordering         |
| Một reservation không được release hai lần       | Conditional state transition + transaction | Inventory        |
| Seller không sửa offer seller khác               | Authenticated actor + ownership check      | Offer management |

#### Check constraint đủ để ngăn overselling?

Với hai cột trên **cùng một dòng stock**, CHECK là lớp chặn cuối cùng hữu ích:

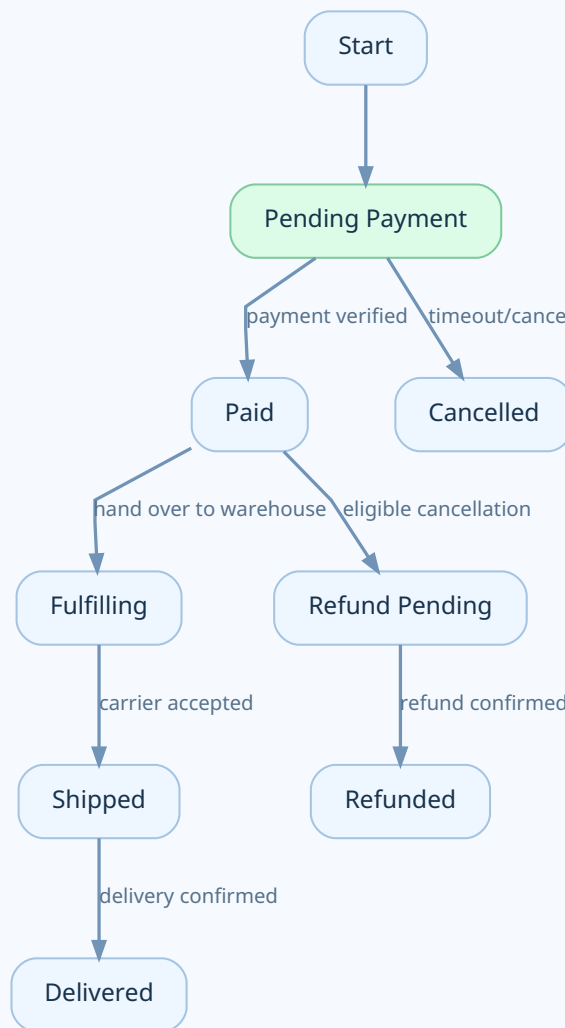
```
CHECK (on_hand_quantity >= 0),  
CHECK (reserved_quantity >= 0),  
CHECK (reserved_quantity <= on_hand_quantity)
```

Nhưng nếu dữ liệu tổng nằm rải ở nhiều dòng hay nhiều bảng, PostgreSQL `CHECK` không được dùng như cơ chế đảm bảo điều kiện xuyên bảng. Khi cần, dùng transaction, FK/UNIQUE, atomic update, trigger đã kiểm chứng hoặc thiết kế lại trạng thái trung tâm [R3].



### 3.9. State Machine: `status` là hợp đồng nghiệp vụ, không phải chuỗi tùy ý

Nếu mọi module cùng gọi `order.setStatus("SUCCESS")`, mỗi team sẽ hiểu khác nhau: đã tạo đơn, đã trả tiền hay đã giao xong? Hãy dùng từ có nghĩa và mô hình trạng thái rõ.



Sơ đồ 09 · SVG đọc offline ·

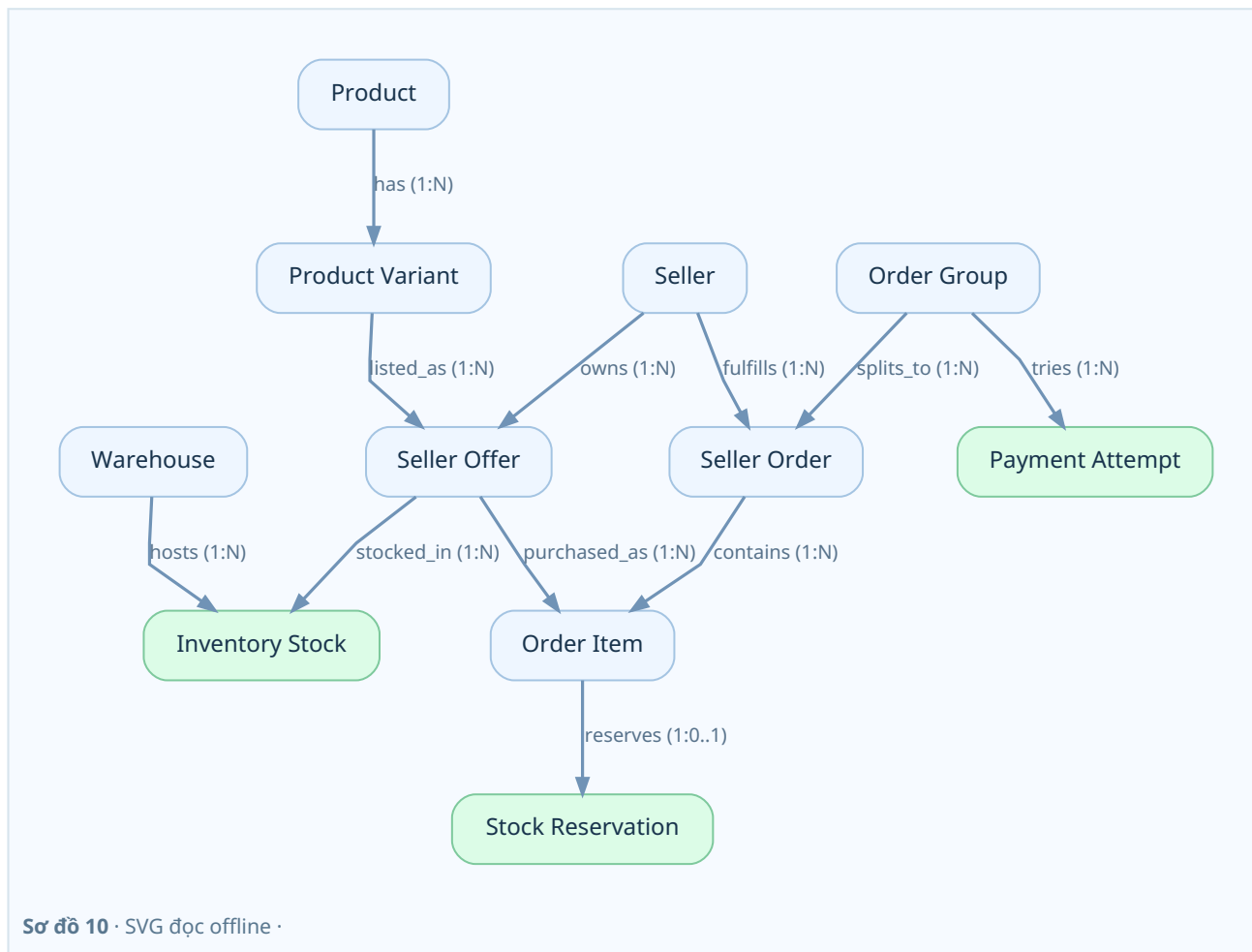
Sơ đồ trên **chỉ minh họa** một order flow; không đại diện mọi chính sách hủy, hoàn một phần, COD hoặc shipment nhiều kiện. Với marketplace, nên tách `order_group`, `seller_order`, `payment_attempt`, `shipment`, `refund` thành state machine có trách nhiệm riêng.

### Bad case: kết hợp trạng thái không hợp lệ

Ví dụ `order.payment_status = PAID`, `order.fulfillment_status = NOT_STARTED` là trạng thái **có thể hoàn toàn hợp lệ**. Nếu dùng một cột `status`, bạn dễ đánh mất thông tin quan trọng.

## 3.10. Thiết kế Database Schema tham khảo cho Hoàng Marketplace

Chúng ta chọn mô hình học tập: **một order group có tối đa một seller order cho mỗi seller; mỗi order item có tối đa một reservation trong một kho; tất cả trong một PostgreSQL**. Đây là giới hạn có chủ đích để schema hiểu được. Khi một item chia nhiều kho hoặc nhiều shipment, cần mở rộng reservation/allocation tương ứng.



### 3.10.1. Bộ bảng tối thiểu và quyền sở hữu

| Bảng               | ID quan trọng                            | Điểm thiết kế                        |
|--------------------|------------------------------------------|--------------------------------------|
| products           | product_id                               | Canonical catalog identity           |
| product_variants   | variant_id, sku_code                     | Một biến thể thuộc một product       |
| sellers            | seller_id                                | Chủ sở hữu offer/order phần seller   |
| seller_offers      | offer_id, seller_id, variant_id          | Giá của lời chào bán                 |
| warehouses         | warehouse_id, seller_id                  | Kho thuộc seller trong ví dụ         |
| inventory_stock    | offer_id + warehouse_id                  | Một stock row cho nguồn cung         |
| order_groups       | order_group_id, customer_id              | Một checkout của khách               |
| seller_orders      | seller_order_id, group_id, seller_id     | Một seller chịu trách nhiệm phần đơn |
| order_items        | order_item_id, seller_order_id, offer_id | Snapshot dòng hàng                   |
| stock_reservations | reservation_id, order_item_id            | Giữ hàng có lifecycle                |
| payment_attempts   | payment_attempt_id, group_id             | Mỗi lần thử thanh toán               |

### 3.10.2. SQL có thể kiểm tra trong PostgreSQL

File SQL đầy đủ đi kèm trong `labs/chapter-03/schema.sql`. Trích đoạn quan trọng:

```

CREATE TABLE seller_offers (
  offer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  seller_id BIGINT NOT NULL REFERENCES sellers(seller_id),
  variant_id BIGINT NOT NULL REFERENCES product_variants(variant_id),
  unit_price NUMERIC(18, 2) NOT NULL CHECK (unit_price >= 0),
  currency CHAR(3) NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('DRAFT','ACTIVE','PAUSED')),
  UNIQUE (offer_id, seller_id)
);

CREATE TABLE inventory_stock (
  offer_id BIGINT NOT NULL,
  seller_id BIGINT NOT NULL,
  warehouse_id BIGINT NOT NULL,
  on_hand_quantity INTEGER NOT NULL CHECK (on_hand_quantity >= 0),
  reserved_quantity INTEGER NOT NULL DEFAULT 0 CHECK (reserved_quantity >= 0),
  PRIMARY KEY (offer_id, warehouse_id),
  FOREIGN KEY (offer_id, seller_id)
    REFERENCES seller_offers(offer_id, seller_id),
  FOREIGN KEY (warehouse_id, seller_id)
    REFERENCES warehouses(warehouse_id, seller_id),
  CHECK (reserved_quantity <= on_hand_quantity)
);

```

Composite FK bảo vệ để tồn kho của Seller A không bị đặt vào kho Seller B trong mô hình giả định. Việc seller được phép dùng fulfillment warehouse chung sẽ yêu cầu sửa quy tắc sở hữu này; không nên giữ cứng khi business thay đổi.

### 3.10.3. Giá chốt trong Order Item

```

CREATE TABLE order_items (
  order_item_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  seller_order_id BIGINT NOT NULL REFERENCES seller_orders(seller_order_id),
  offer_id BIGINT NOT NULL REFERENCES seller_offers(offer_id),
  sku_code_snapshot TEXT NOT NULL,
  title_snapshot TEXT NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price_snapshot NUMERIC(18,2) NOT NULL
    CHECK (unit_price_snapshot >= 0),
  currency CHAR(3) NOT NULL
);

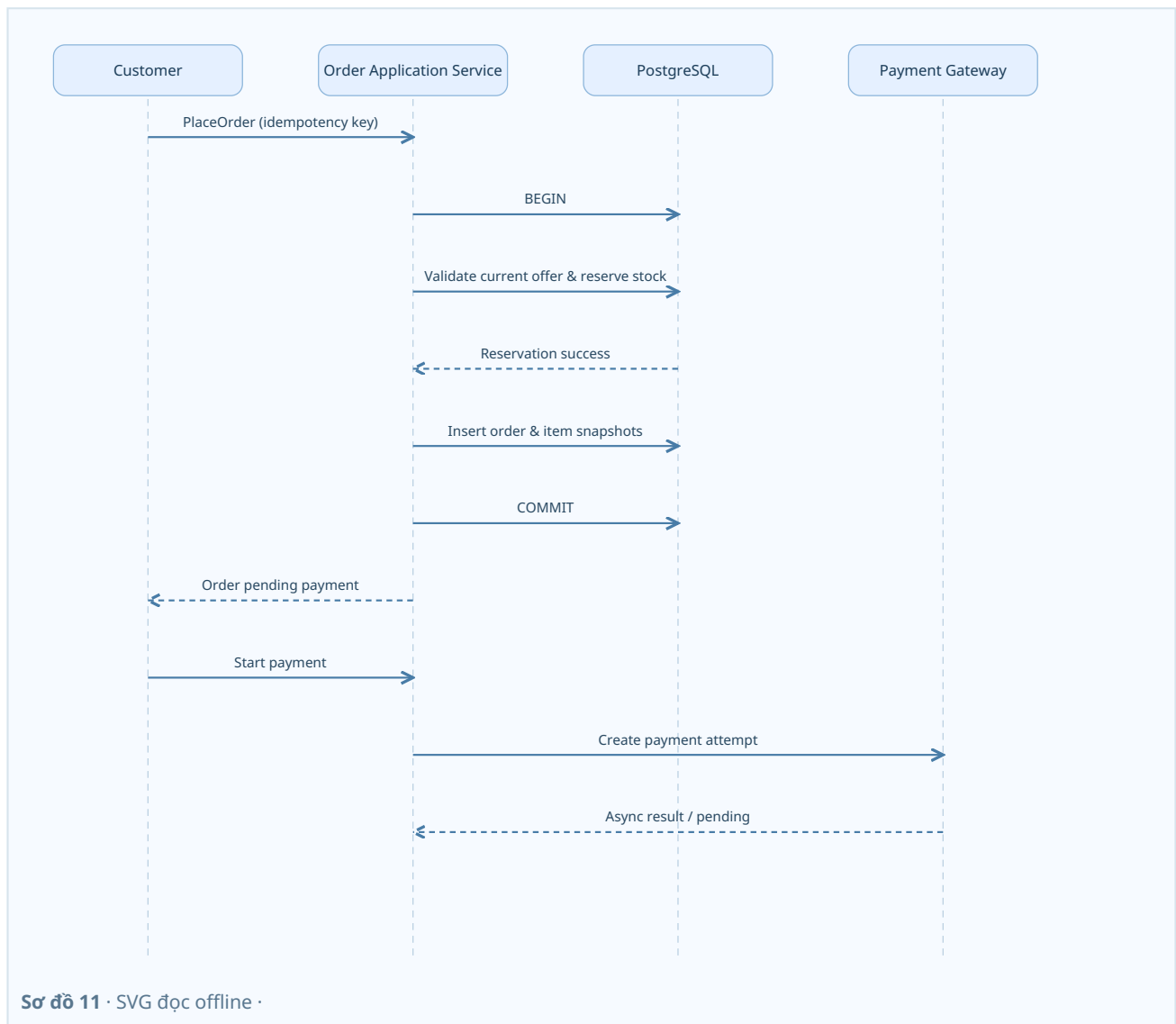
```

**Lưu ý consistency:** chỉ có FK `offer_id` vẫn chưa đảm bảo offer và seller order thuộc cùng seller. Ở schema đầy đủ, `seller_id` trên `order_items` được xác minh qua composite FK đến cả offer và seller order. Đừng coi việc join đúng ở một API là bảo vệ toàn hệ thống.

## 3.11. Transaction Boundary trong use case `PlaceOrder`

### Ranh giới nguyên tử nếu Order và Inventory cùng DB

Từ request, server resolve Offer và current quote; xác thực seller/availability; tạo Order; giữ stock; commit. Các thay đổi liên quan cần được quản lý bằng một local transaction phù hợp. **Không gọi HTTP Payment Gateway khi đang giữ row lock trong transaction.**



Một cách trừ **available** để tăng **reserved** an toàn tại PostgreSQL:

```

UPDATE inventory_stock
SET reserved_quantity = reserved_quantity + :quantity
WHERE offer_id = :offer_id
  AND warehouse_id = :warehouse_id
  AND on_hand_quantity - reserved_quantity >= :quantity
RETURNING offer_id, on_hand_quantity, reserved_quantity;

```

Nếu không có dòng trả về, không được coi là đã giữ thành công. PostgreSQL xử lý UPDATE đồng thời với row locking và kiểm tra điều kiện thích hợp khi tiếp tục, nhưng toàn bộ use case vẫn phải thiết kế retry/timeout và constraint đúng [R5]. Khi reservation thực sự được tiêu thụ để xuất hàng theo chính sách ví dụ, `on_hand_quantity` và `reserved_quantity` phải cùng giảm đúng `quantity` trong một transaction kèm chuyển trạng thái reservation. Nếu đơn bị hủy trước khi xuất hàng, chỉ `reserved_quantity` giảm khi giải phóng thành công; `on_hand_quantity` không đổi.

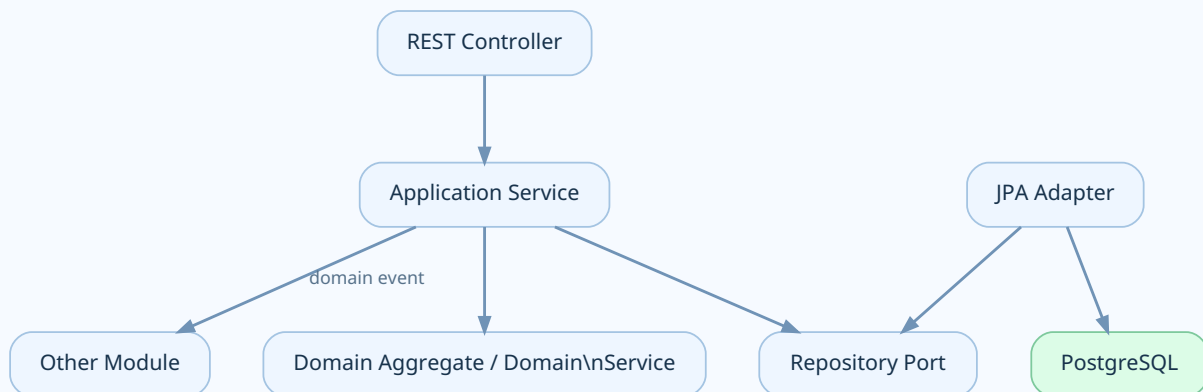
## Khi nào không nên gộp vào một Aggregate?

Order và Stock có lifecycle khác nhau. Một Order cần giữ stock cho từng line, nhưng stock có thể bị điều chỉnh bởi nhập kho, hỏng hàng, xuất kho và đơn khác. Gộp tất cả thành một `Order` object cực lớn không làm transaction “an toàn hơn”; nó tạo coupling và có thể tăng contention.

## 3.12. Modular Monolith trong Spring Boot

Chọn phân chia package theo **business capability**, không chia toàn dự án thành các package toàn cục `controller`, `service`, `repository`, `entity` khiến code của mỗi use case rải khắp nơi.

```
com.hoang.marketplace/
├── catalog/
│   ├── api/
│   ├── application/
│   ├── domain/
│   └── infrastructure/
├── offers/
├── inventory/
├── ordering/
│   ├── api/PlaceOrderController.java
│   ├── application/PlaceOrderService.java
│   ├── domain/Order.java
│   ├── domain/OrderItem.java
│   ├── domain/Money.java
│   └── infrastructure/OrderJpaRepository.java
├── payments/
└── fulfillment/
```



Sơ đồ 12 · SVG đọc offline ·

| Layer               | Nên làm                                        | Không nên làm                                            |
|---------------------|------------------------------------------------|----------------------------------------------------------|
| API / Controller    | Parse request, auth boundary, map DTO/ status  | Tự trừ stock và sửa state machine                        |
| Application Service | Điều phối use case, transaction, idempotency   | Nhồi toàn bộ invariant vào <code>if</code> rải rác       |
| Domain              | Business rule, state transitions, Value Object | Phụ thuộc HTTP, controller, Spring MVC                   |
| Infrastructure      | SQL, JPA, broker, external provider            | Quyết định ai được phép mua dựa trên input chưa xác thực |

DDD **không ép tất cả module phải có rich domain model**. Catalog CRUD thuần túy có thể đơn giản hơn; Inventory và Ordering thường chứa nhiều invariant và hành vi cần mô hình hóa rõ. Tài liệu Microsoft cũng lưu ý DDD pattern đặc biệt hữu ích với domain phức tạp, còn CRUD đơn giản không cần bị over-engineer [R1].

## Java Domain ví dụ: không cho ai tùy ý đổi trạng thái

```
public final class Order {
    private final long id;
    private OrderStatus status;

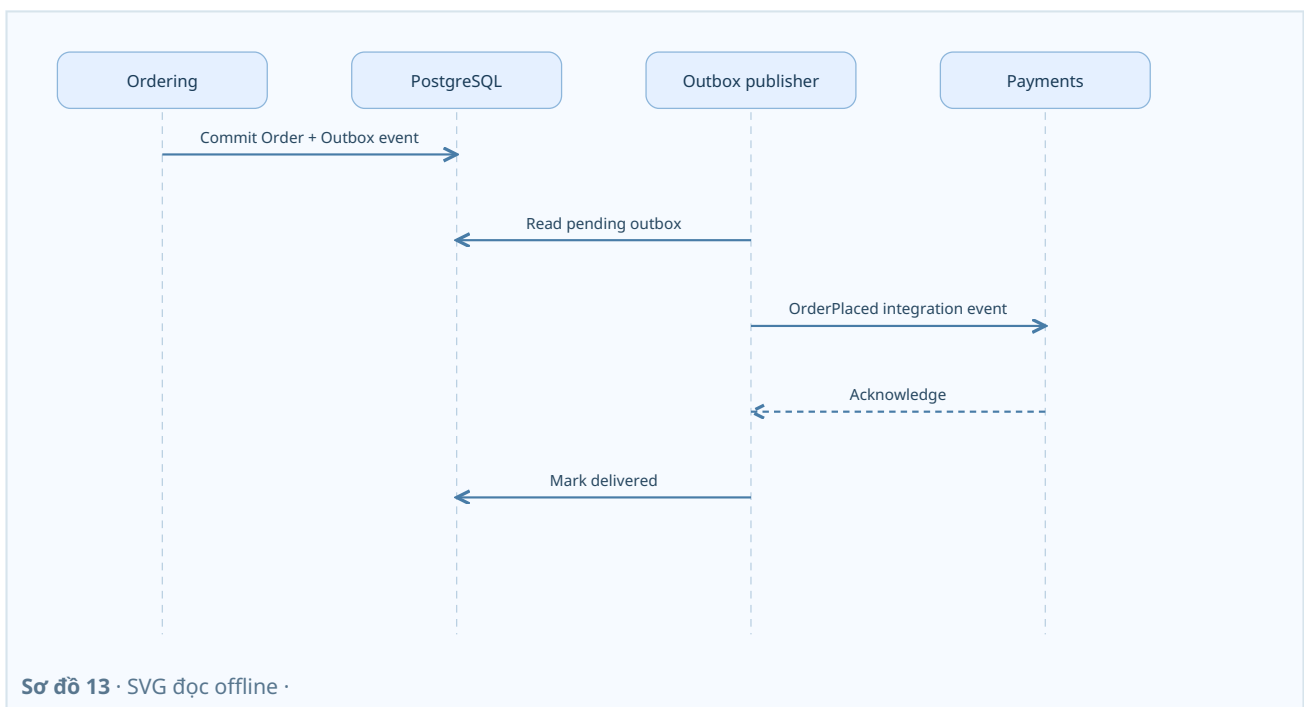
    public void markPaid() {
        if (status != OrderStatus.PENDING_PAYMENT) {
            throw new IllegalStateException("Invalid transition");
        }
        status = OrderStatus.PAID;
    }

    public void cancelBeforePayment() {
        if (status != OrderStatus.PENDING_PAYMENT) {
            throw new IllegalStateException("Cannot cancel here");
        }
        status = OrderStatus.CANCELLED;
    }
}
```

Ví dụ này chỉ mô tả invariant trạng thái. Hệ thống thực tế cần map trạng thái Payment verified, xử lý đồng thời, nhiều trạng thái hủy/hoàn tiền, lưu vết và persistence.

## 3.13. Domain Events: thứ gì đã xảy ra, không phải một lệnh SQL

`OrderPlaced`, `StockReserved`, `PaymentVerified`, `ReservationExpired` là các sự kiện có nghĩa nghiệp vụ. `row_inserted`, `table_updated` chỉ nói về persistence.



Với các module cùng process và database, bạn có thể xử lý một số domain event nội bộ trong use case. Nếu dùng broker hoặc service bên ngoài, phải nghĩ đến **DB committed nhưng event không gửi được**.

Transactional Outbox là một hướng giải quyết: ghi sự kiện trong cùng DB transaction, rồi xuất bản bất đồng bộ. Consumer vẫn cần idempotent vì event có thể được gửi lại.

Đừng làm toàn bộ event-driven ngay chương này; chúng ta dùng Domain Event để diễn tả một sự thật nghiệp vụ, còn reliability kỹ thuật sẽ học sâu ở Volume 5–6.

### 3.14. Failure scenarios: domain model tốt giúp phát hiện lỗi trước khi scale

| Failure scenario                                   | Thiếu trong mô hình              | Cách phát hiện / phòng vệ                                                   |
|----------------------------------------------------|----------------------------------|-----------------------------------------------------------------------------|
| Khách đổi địa chỉ profile sau khi đặt              | Address snapshot                 | Test order address không đổi                                                |
| Seller B sửa offer của Seller A                    | Owner boundary                   | AuthZ test khác seller → 403                                                |
| Order cũ đổi tổng tiền khi offer đổi giá           | Order item price snapshot        | Test update offer không đổi order                                           |
| Hai checkout giữ cùng stock cuối                   | Atomic reservation               | Concurrency integration test                                                |
| Client retry sau timeout tạo order thứ hai         | Idempotency                      | UNIQUE(customer_id, key) + replay result                                    |
| Worker giải phóng một reservation hai lần          | Reservation lifecycle            | Conditional <code>ACTIVE → RELEASED</code> và update stock cùng transaction |
| Offer thuộc seller A nhưng order item gán seller B | Cross-table ownership constraint | Composite FK hoặc domain verification + integration test                    |
| Một seller order hủy nhưng group báo hoàn tất      | Group state derived incorrectly  | Test partial-failure aggregation                                            |

#### Quiz phân biệt lỗi về model và lỗi về infrastructure

- Một cột `stock` không phân biệt kho: lỗi mô hình.
- PostgreSQL hết connection: vấn đề vận hành/capacity, có thể bị khuếch đại bởi schema/query tệ.
- Payment callback gửi hai lần tạo hai lần hoàn tiền: model/idempotency + xử lý concurrency.
- Search index cập nhật chậm vài giây nhưng order check lại current price: có thể là eventual consistency được phép theo business.

### 3.15. Engineering Lab — từ schema ngây thơ đến domain có thể kiểm chứng

#### Điều kiện đầu vào

- Hoàng Marketplace: hai seller bán cùng một variant; mỗi seller có một kho.
- Giỏ có thể chứa hàng của hai seller.
- Checkout tạo order group + seller order + order items.
- Giá đã chốt phải không đổi khi offer thay đổi.
- Một reservation ở một kho có thể được release tối đa một lần.

## Lab A — Vẽ context map

Tạo sơ đồ Catalog, Offers, Inventory, Ordering, Payments, Fulfillment. Với mỗi mũi tên, ghi rõ đây là API query, command, event hay identity reference. Không vẽ tất cả thành quan hệ FK.

## Lab B — Chạy SQL mẫu

File `labs/chapter-03/schema.sql` chứa schema và các constraint; file `queries.sql` có câu UPDATE đặt giữ hàng, release và truy vấn snapshot. Chạy thử một lỗi `reserved_quantity > on_hand_quantity` và xác nhận CHECK từ chối.

## Lab C — Concurrency test

Seed stock: `on_hand = 1`, `reserved = 0`. Cho hai transaction đồng thời chạy UPDATE giữ 1. Kỳ vọng: chỉ một request nhận một dòng `RETURNING`, request còn lại nhận zero rows, sau commit stock không âm và không overselling.

## Lab D — State transition test

Viết test cho `PENDING_PAYMENT → PAID` hợp lệ; `CANCELLED → PAID` bị từ chối nếu chưa có quy trình phục hồi đặc biệt. Đừng chỉ test HTTP response; test invariant trong domain.

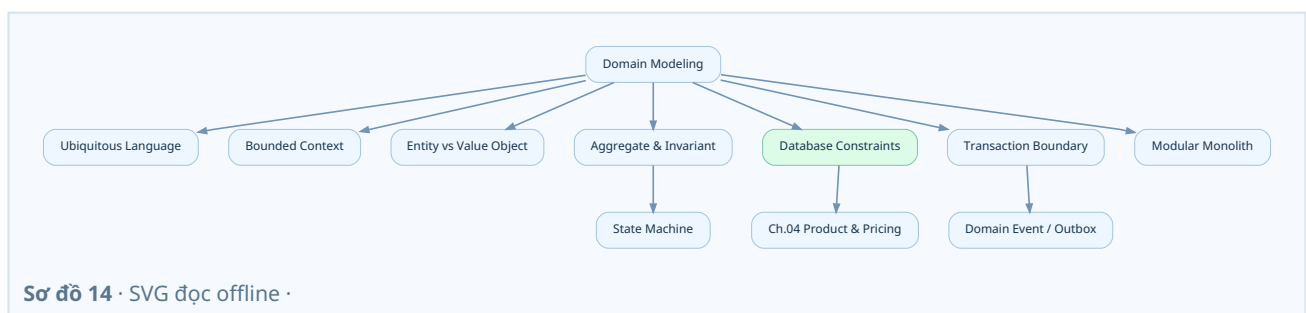
## Lab E — Architecture review

Trả lời bằng lời, không cần code: Tại sao Order lưu `offer_id` và cả `unit_price_snapshot`? Vì sao Payment Attempt tách khỏi Order? Khi nào một `Address` là Value Object, khi nào trở thành Entity? Nếu sau này seller dùng kho fulfillment chung, constraint nào trong schema hiện tại phải thay đổi?

## Checklist hoàn thành chương

- [ ] Tôi phân biệt Product, Variant, Offer, Inventory và Order snapshot.
- [ ] Tôi biết Entity không phải lúc nào cũng bằng bảng hoặc `@Entity`.
- [ ] Tôi xác định được ít nhất bốn business invariants có thể viết test.
- [ ] Tôi biết Aggregate là consistency boundary, không phải tên khác của microservice.
- [ ] Tôi biết khi nào dùng ID reference, FK và event.
- [ ] Tôi hiểu local DB transaction không bao phủ tự động payment provider.
- [ ] Tôi có thể giải thích vì sao dữ liệu seller cần owner boundary.

## Knowledge Map



## Key Takeaways

---

1. **Business rules → domain model → persistence/API**, không đảo ngược và để bảng quyết định nghiệp vụ.
2. Product, Variant, Offer, Inventory, Order Item Snapshot là những khái niệm khác nhau; trộn chúng khiến giá, kho, lịch sử đơn hàng sai.
3. Entity cần identity; Value Object được nhận diện bằng giá trị. `@Entity` JPA chỉ là cơ chế persistence.
4. Bounded Context là ranh giới về ngôn ngữ và quyền quyết định, không bắt buộc tách deployment.
5. Aggregate bảo vệ invariant; không mặc định một Aggregate = một bảng hoặc một `@Transactional`.
6. Dùng DB constraints cùng domain logic/transaction để bảo vệ tính đúng đắn dưới concurrency.
7. Giao dịch tài chính, tồn kho và trạng thái đơn có lifecycle riêng; không nhồi vào một `status` đa nghĩa.
8. Schema tốt phải có cả tính đúng đắn **và** kế hoạch phát triển khi business thay đổi.

## Tài liệu tham khảo và giới hạn áp dụng

---

- **[R1]** Microsoft Learn, *Designing a microservice domain model*: <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/microservice-domain-model>
- **[R2]** Microsoft Learn, *Use tactical DDD to design microservices*: <https://learn.microsoft.com/en-au/azure/architecture/microservices/model/tactical-ddd>
- **[R3]** PostgreSQL Documentation, *Constraints*: <https://www.postgresql.org/docs/current/ddl-constraints.html>
- **[R4]** PostgreSQL Documentation, *Numeric Types*: <https://www.postgresql.org/docs/current/datatype-numeric.html>
- **[R5]** PostgreSQL Documentation, *Transaction Isolation*: <https://www.postgresql.org/docs/current/transaction-iso.html>
- **[R6]** Spring Data JPA, *Reference Documentation*: <https://docs.spring.io/spring-data/jpa/reference/>

Các sơ đồ và số liệu Hoàng Marketplace trong chương là mô hình học tập giả định. Cần thống nhất đúng nghiệp vụ trước khi áp dụng vào một dự án thật; schema và code phải được kiểm thử với phiên bản PostgreSQL/Spring đang triển khai.

**Tiếp theo — Chapter 04: Product & Pricing.** Chúng ta sẽ khai thác product lifecycle, variant modeling, giá của Offer, lịch sử giá, voucher, quote, giá được chốt và các trade-off khi cache giá sản phẩm.

# INSIDE E-COMMERCE

## Volume 1 — Understanding E-commerce

### Chapter 04 — Product & Pricing Engineering: Vì sao giá không chỉ là một cột price?

**Câu hỏi dẫn đường:** Khách thấy điện thoại giá 20 triệu, admin giảm còn 19 triệu, voucher giảm thêm 500 nghìn, phí giao hàng thay đổi, rồi khách nhấn thanh toán đúng lúc chương trình hết hạn. Hệ thống phải hiển thị, tính và ghi nhận giá nào?

**Phạm vi:** Tiếp tục ví dụ giả định **Hoàng Phone → Hoàng Marketplace** ở chương 01–03. Các khoản tiền, thuế và ưu đãi trong chương là ví dụ thiết kế, không phải tư vấn pháp lý hoặc công thức áp dụng cho mọi quốc gia. Các chiến lược như “giữ giá X phút”, “áp voucher trước phí ship” là **quy tắc business phải khai báo**, không phải quy luật chung của mọi E-commerce.

### Sau chương này, bạn có thể

- Phân biệt **Product, Variant/SKU, Seller Offer, List Price, Selling Price, Compare-at Price, Cost, Price Quote** và **Order Price Snapshot**.
- Thiết kế pricing pipeline kiểm tra điều kiện khuyến mãi, voucher, phí vận chuyển và thuế có **thứ tự, phiên bản, cách làm tròn xác định**.
- Giải thích vì sao cart total chỉ là **ước tính** và vì sao checkout phải tính lại bằng dữ liệu có thẩm quyền.
- Xử lý cuộc đua giữa cập nhật giá, cache cũ, voucher giới hạn lượt, Flash Sale và request checkout bị gửi lại.
- Thiết kế dữ liệu PostgreSQL và Java `BigDecimal` cho tiền tệ không sử dụng sai `double`.
- Tạo **Price Quote** có thời hạn hoặc chính sách revalidation; ghi **Order Snapshot** giải thích được giao dịch cũ.
- Quan sát bottleneck pricing, test invariants và nêu trade-off khi tách Pricing Service.

### 4.1. Bad case: lưu mọi thứ trong `products.price`

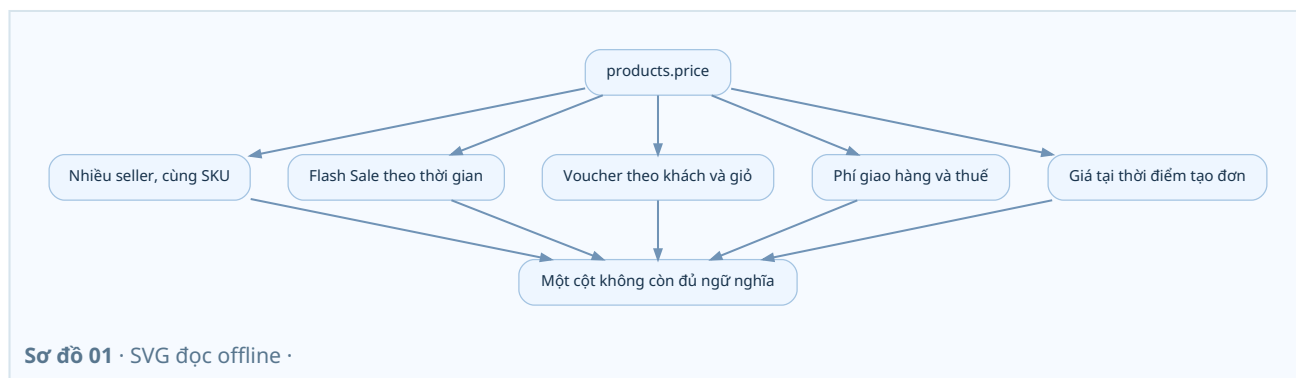
Phiên bản demo của Hoàng Phone chỉ có một giá và một seller:

```
CREATE TABLE products (  
  id BIGINT PRIMARY KEY,  
  name TEXT NOT NULL,  
  price NUMERIC(18, 2) NOT NULL  
);
```

Frontend nhận `product.price`, nhân số lượng, trừ voucher từ request rồi gửi `totalAmount` về backend. Backend tạo đơn theo số tiền đó.

**Vấn đề không phải cột `price` luôn sai. Vấn đề là ta đã ép quá nhiều khái niệm khác nhau vào một con số và để client quyết định tiền phải thu.**

Nếu thêm Marketplace, cùng một SKU có thể được Seller A chào bán 20 triệu và Seller B chào bán 19,7 triệu. Nếu thêm Flash Sale, giá còn phụ thuộc thời điểm và điều kiện. Nếu thêm voucher, giá cuối cùng còn phụ thuộc khách hàng, số lượng, giỏ hàng, địa chỉ nhận, cách giao hàng và quy tắc kết hợp ưu đãi.



## Nỗi đau được bộc lộ qua bốn câu hỏi

1. Cùng một mã hàng, hai seller khác nhau có được đặt hai giá khác nhau không?
2. Khách đã đặt đơn với giá 20 triệu, admin sửa catalog còn 18 triệu: đơn cũ phải hiện 20 hay 18?
3. Khách sửa `totalAmount=1000` trong HTTP request: backend có tin không?
4. Giá Flash Sale hết hạn giữa lúc khách đang điền địa chỉ: hệ thống áp giá nào và thông báo ra sao?

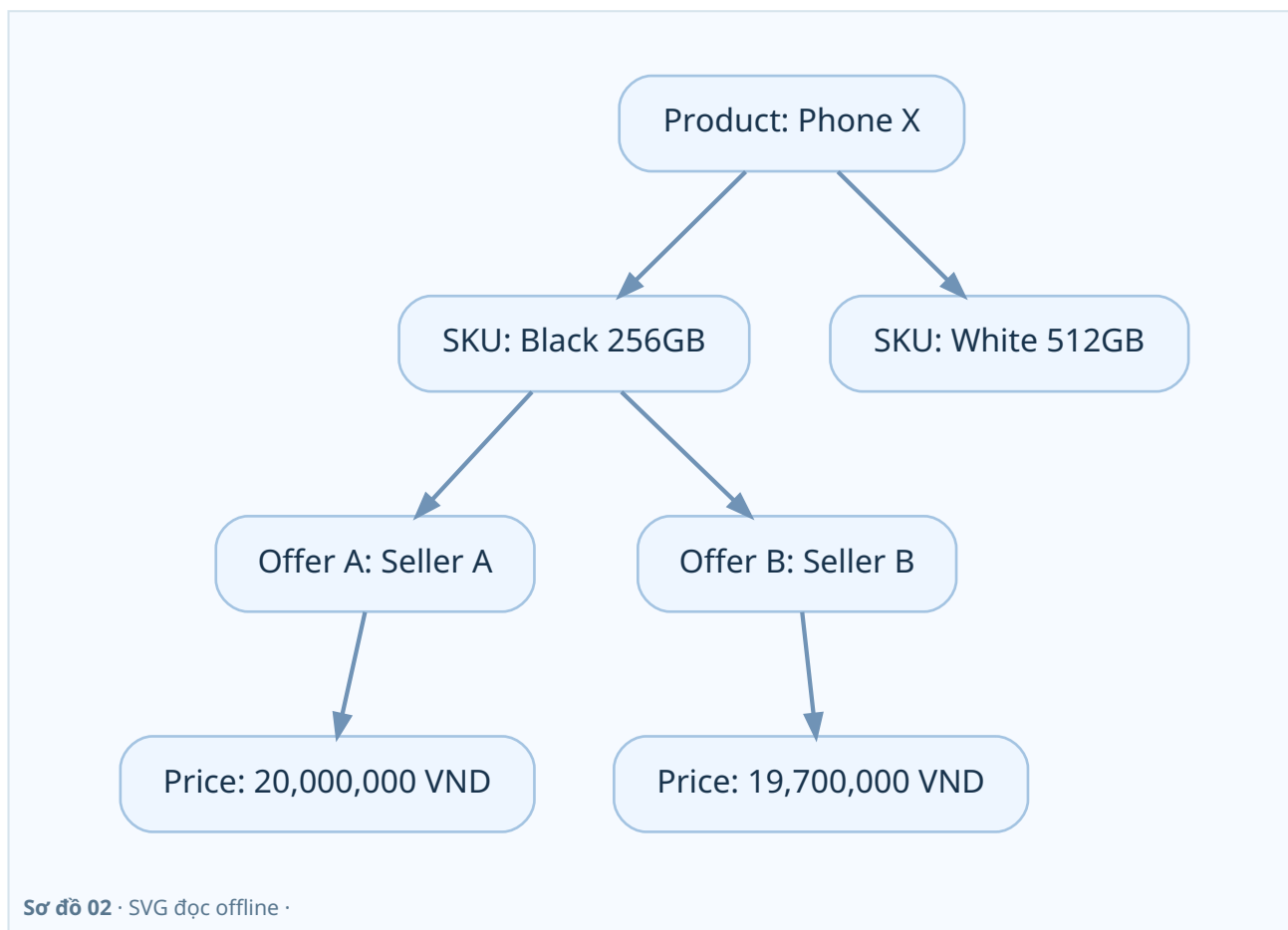
Ta cần định nghĩa lại **giá thuộc ngữ cảnh nào và có hiệu lực vào thời điểm nào**.

## 4.2. Product, SKU và Seller Offer: ai thực sự sở hữu giá?

Trong ví dụ Marketplace, ba khái niệm dễ nhầm là:

| Khái niệm     | Trách nhiệm                                    | Ví dụ                                  |
|---------------|------------------------------------------------|----------------------------------------|
| Product       | Thông tin mặt hàng chung                       | Điện thoại X                           |
| Variant / SKU | Cấu hình hàng có thể bán                       | X đen, 256 GB                          |
| Seller Offer  | Lời chào bán của seller cho SKU theo điều kiện | Seller A bán X đen 256 GB giá 20 triệu |

Cùng một variant có thể có nhiều Offer. Nếu hệ thống bán trực tiếp, giá có thể gắn vào SKU/price list của cửa hàng; nếu Marketplace, Seller Offer thường là vị trí tự nhiên để biểu đạt seller, kênh bán, điều kiện và mức giá.



**Khái niệm ‘giá thuộc SKU hay Offer’ là quyết định domain**, không phải yêu cầu công nghệ. Cần dựa vào cách vận hành: ai được sửa giá, ai chịu doanh thu, mức giá có thay đổi theo kênh/bên bán hay không.

## Những mức giá phải tách tên

| Thuật ngữ            | Ý nghĩa                                             | Có nhất thiết là giá thanh toán? |
|----------------------|-----------------------------------------------------|----------------------------------|
| List / Regular Price | Giá bán thông thường theo chính sách                | Không                            |
| Selling Price        | Giá bán đang có hiệu lực trước các ưu đãi bổ sung   | Chưa chắc                        |
| Compare-at Price     | Giá đối chiếu phục vụ hiển thị khi phù hợp quy định | Không                            |
| Cost / COGS          | Chi phí vốn phục vụ nội bộ                          | Không                            |
| Unit Price in Quote  | Đơn giá hệ thống xác nhận cho một yêu cầu cụ thể    | Tham gia tính tổng               |
| Line Total           | Tổng dòng hàng sau phân bổ giảm giá tương ứng       | Một phần tổng đơn                |
| Order Snapshot       | Giá đã chốt và phép tính tại thời điểm đặt đơn      | Căn cứ giải thích giao dịch      |

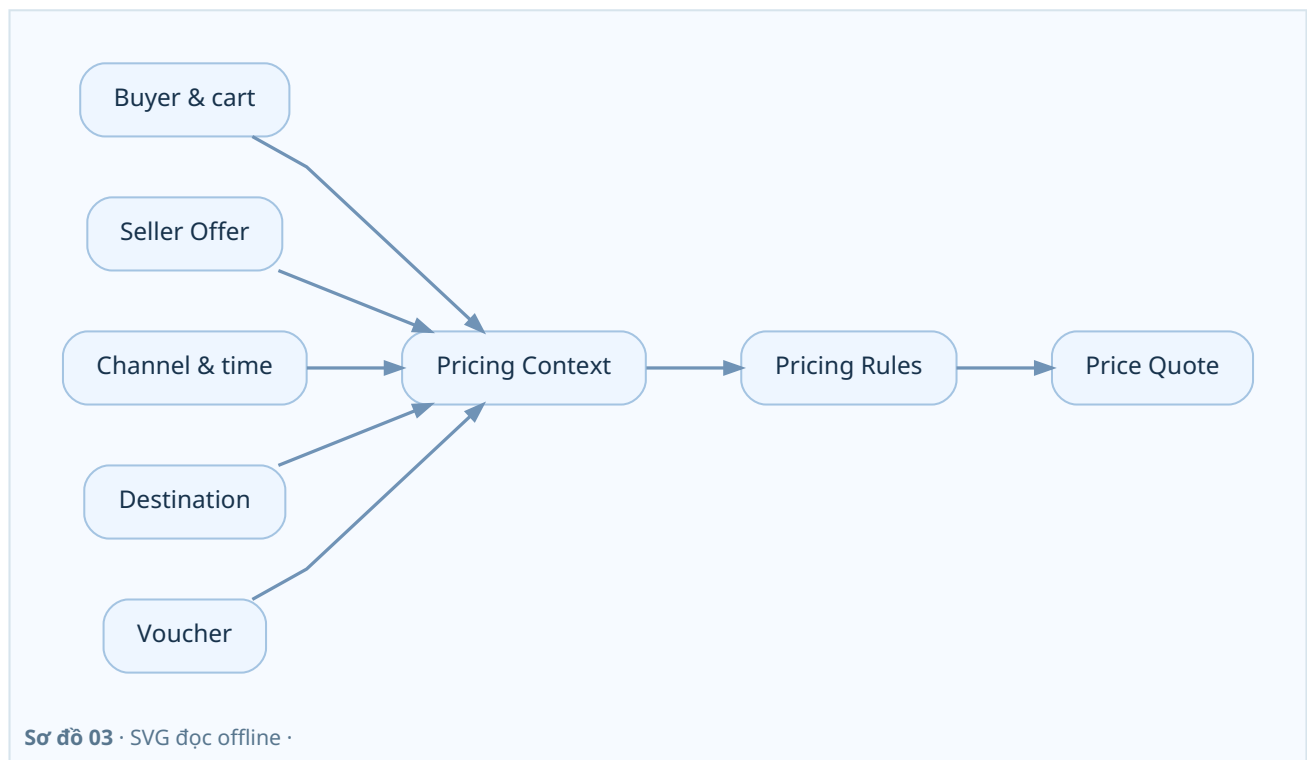
**Compare-at Price không mặc nhiên chứng minh khách đã từng mua ở giá đó.** Cách thể hiện ‘giảm bao nhiêu %’ còn phải tuân thủ chính sách kinh doanh và quy định quảng cáo/giá tại thị trường triển khai. Các nền tảng commerce thực tế cũng tách `price` và `compare-at price` theo variant; tham khảo Shopify Help Center trong mục Tài liệu tham khảo cuối chương.

### 4.3. Pricing Context — vì sao cùng SKU có nhiều giá hợp lệ?

Đừng viết hàm `getPrice(productId)` rồi cố truyền thêm tham số mỗi khi business đổi yêu cầu. Hãy hình dung toàn bộ ngữ cảnh cần thiết để **định giá một lời đề nghị mua hàng**:

```
{
  "buyerId": 101,
  "sellerOfferId": 701,
  "skuId": "PHONE-X-BLACK-256",
  "quantity": 2,
  "channel": "WEB",
  "currency": "VND",
  "deliveryZone": "HANOI",
  "promotionCodes": ["WELCOME500"],
  "quotedAt": "2026-09-23T13:45:00+07:00"
}
```

`buyerId` có thể chỉ dùng để tra cứu phân khúc/điều kiện ưu đãi, không cần lộ dữ liệu nhạy cảm trong cache key. `quotedAt` cần được lấy từ thời gian tin cậy phía server; **không tin thời gian do client tự gửi**. Địa chỉ chính xác được cung cấp ở checkout, còn khu vực giao hàng có thể dùng cho báo giá sơ bộ.



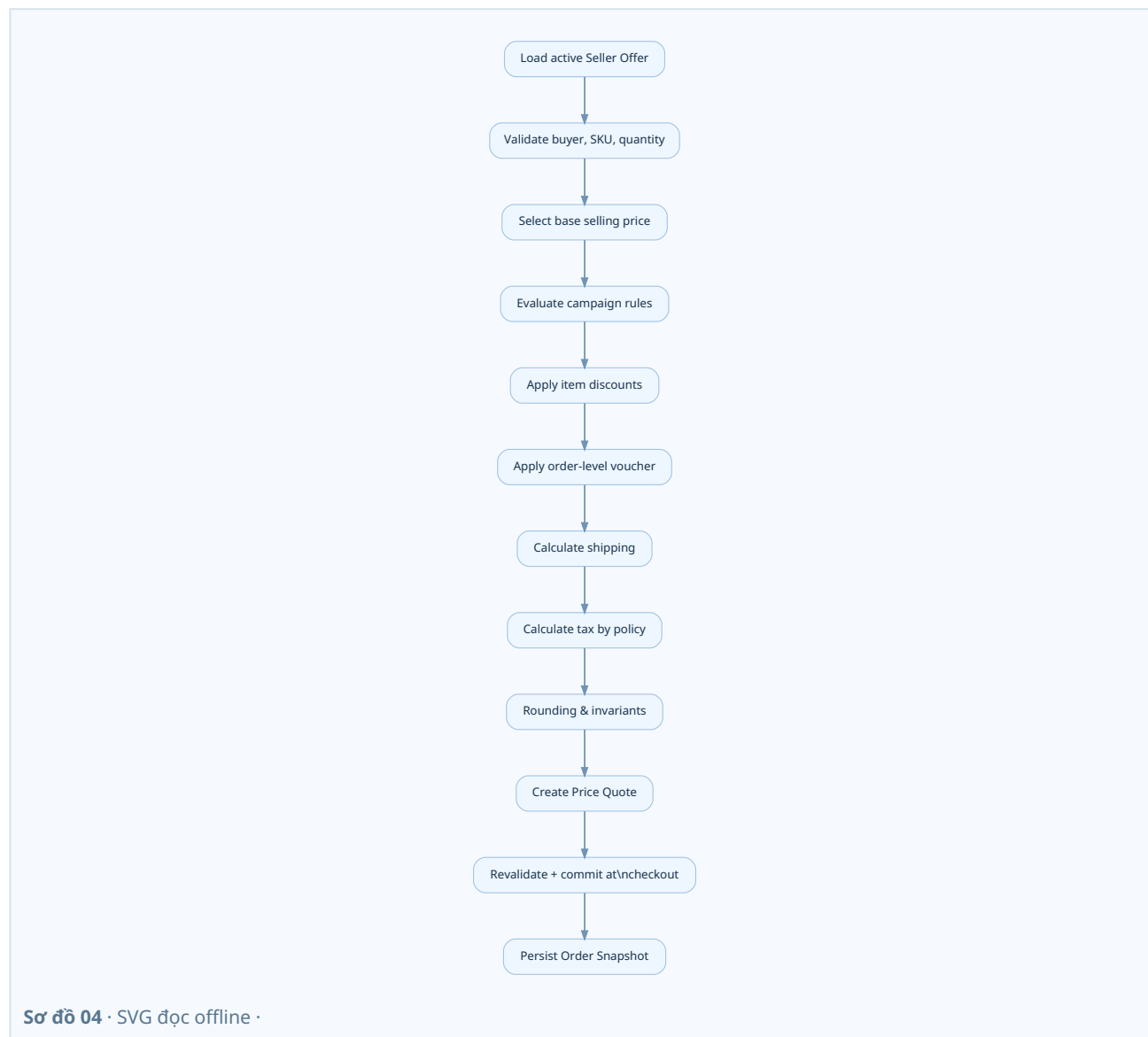
#### Phân biệt Quote và Order

- **Product display price:** mức giá đủ hữu ích để duyệt catalog; có thể cache theo ngữ cảnh đã xác định.
- **Cart estimate:** giá tạm tính; phải chấp nhận thay đổi về phí ship/ưu đãi khi checkout.
- **Checkout quote:** kết quả tính chính thức hơn, có thời hạn và điều kiện sử dụng rõ ràng.
- **Order snapshot:** các giá trị giao dịch được chốt sau khi tạo đơn thành công; không tham chiếu sống sang catalog để tái tính đơn cũ.

Một Quote **không tự động giữ tồn kho hay voucher**. Muốn đảm bảo mức giá/ưu đãi đến thời điểm chốt đơn, bạn cần triển khai thêm cơ chế commit/reservation cho các tài nguyên tương ứng, và quy tắc hết hạn rõ ràng.

## 4.4. Pricing Pipeline: từ giá chào bán đến số tiền cần thanh toán

Một mô hình minh họa cho Hoàng Phone:



**Không có một thứ tự cộng/trừ duy nhất đúng với tất cả thị trường hoặc chiến dịch.** Đặc biệt thuế có thể là giá bao gồm thuế, giá chưa thuế, thuế tính trên giá sau giảm, các voucher có thể được phân bổ về từng dòng hàng. Từng chiến dịch phải mô tả rõ cơ sở tính, đối tượng giảm, thứ tự, khả năng kết hợp và làm tròn.

### Một phép tính trọn vẹn (giả định chính sách cụ thể)

Đơn mua 2 chiếc tai nghe, mỗi chiếc 500.000 VND:

| Bước                   | Phép tính              | Kết quả   |
|------------------------|------------------------|-----------|
| Gross item subtotal    | $2 \times 500.000$     | 1.000.000 |
| Item campaign discount | -10% của item subtotal | -100.000  |

| Bước                                      | Phép tính           | Kết quả            |
|-------------------------------------------|---------------------|--------------------|
| Subtotal after item discount              | 1.000.000 – 100.000 | 900.000            |
| Order voucher                             | –50.000             | –50.000            |
| Merchandise total                         | 900.000 – 50.000    | 850.000            |
| Shipping fee                              | +30.000             | 880.000            |
| Tax (ví dụ này giả định đã nằm trong giá) | +0                  | <b>880.000 VND</b> |

Lưu ý: đây là ví dụ **giá đã bao gồm mọi thuế áp dụng** nên không cộng thuế ở cuối. Không suy diễn công thức này thành cách tính thuế mặc định cho mọi hàng hóa.

## Không cho tổng âm

Nếu voucher 1 triệu áp lên giỏ đủ điều kiện trị giá 850.000 mà chính sách không cho phát sinh credit, discount được **cap** tại 850.000, hoặc voucher bị từ chối nếu quy tắc quy định mức mua tối thiểu. Việc chọn cách xử lý thuộc rule engine, không phải `Math.max(0, total)` và ở bước cuối.

## 4.5. Promotion và Voucher: hai khái niệm không nên nhập làm một

Một campaign tự động có thể áp dụng khi điều kiện thỏa mãn; voucher thường yêu cầu code hoặc được phát cho một khách nhất định. Nhưng business hoàn toàn có thể gọi “voucher tự động”, vì vậy hãy ghi rõ định nghĩa trong glossary của dự án.

**Promotion Rule** nên trả lời được:

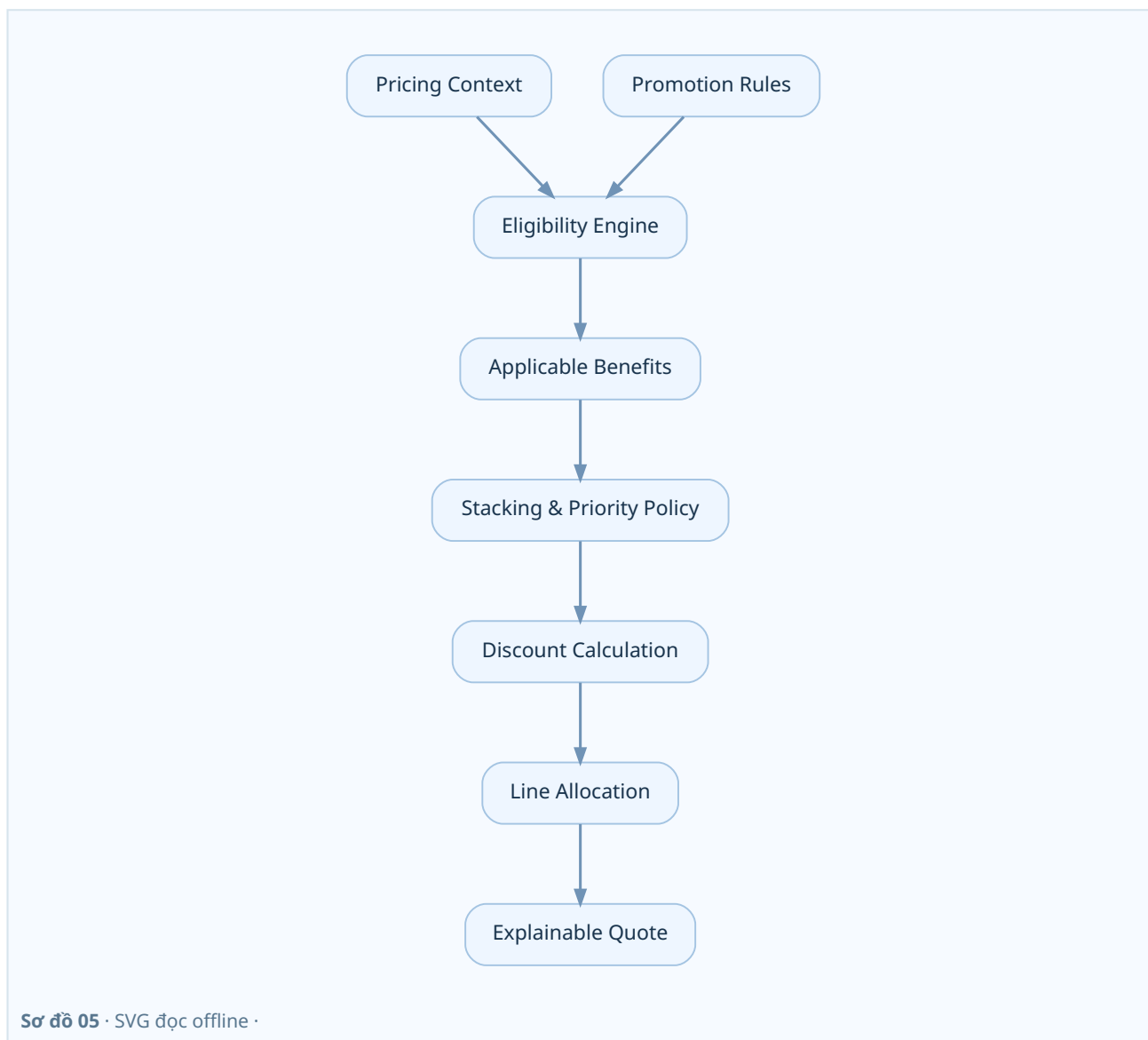
- **eligible**: ai, seller nào, SKU nào, kênh nào, vùng giao nào được áp dụng?
- **validity**: `starts_at <= server_now < ends_at` hay lịch theo timezone nào?
- **benefit**: giảm % hay tiền cố định, có mức tối đa không, có miễn phí ship không?
- **stackability**: có kết hợp voucher, campaign khác, Flash Sale hay giá dành cho thành viên không?
- **budget**: tổng ngân sách, số lượt sử dụng, giới hạn mỗi user và giới hạn toàn campaign là bao nhiêu?
- **allocation**: discount cấp đơn phân về từng item/seller như thế nào khi hoàn một phần?

### Bad case: logic ưu đãi rải rác trong controller

```
if (voucher.equals("WELCOME500")) total -= 500_000;
if (isFlashSale) total = total * 0.8;
if (isVip) total -= 100_000;
```

Ba dòng code này không nói rõ thứ tự, giá trị tối thiểu, giới hạn giảm, số tiền còn lại, điều kiện seller, cách làm tròn, quyền sửa mã từ request, và liệu khách VIP có được dùng đồng thời với Flash Sale không.

Thay vì “if chain” không có metadata, tách việc kiểm tra điều kiện và việc áp dụng:



## Stacking policy ví dụ

| Campaign       | Voucher       | Chính sách giả định | Kết quả                             |
|----------------|---------------|---------------------|-------------------------------------|
| Flash Sale 20% | WELCOME50K    | Không stack         | Chọn một theo điều kiện business    |
| Category 10%   | Free Shipping | Stack được          | Giảm hàng + giảm ship               |
| Seller A 5%    | Seller B 5%   | Áp theo seller      | Mỗi benefit chỉ trên item tương ứng |

Từ “chọn một” không nhất thiết nghĩa là chọn giảm giá lớn nhất: có nơi ưu tiên campaign cao hơn, nơi khác chọn lợi ích cao hơn, nơi khác buộc người dùng lựa chọn. Đây là phần cần thống nhất với business và đưa vào test.

## 4.6. Money: đừng dùng `double` để biểu diễn tiền

`double` lưu số thực nhị phân gần đúng; không thể biểu diễn chính xác nhiều số thập phân như 0.1. Những phép cộng/trừ nhỏ khi tích lũy qua hàng nghìn dòng có thể gây sai lệch khi đối soát. Với Java, dùng

`BigDecimal` có scale/rounding rõ ràng, hoặc dùng số nguyên đơn vị tiền nhỏ nhất nếu hệ thống thống nhất quy ước từ đầu.

```
import java.math.BigDecimal;
import java.math.RoundingMode;

BigDecimal price = new BigDecimal("500000");
BigDecimal quantity = BigDecimal.valueOf(2);
BigDecimal gross = price.multiply(quantity);
BigDecimal tenPercent = gross
    .multiply(new BigDecimal("0.10"))
    .setScale(0, RoundingMode.HALF_UP);
BigDecimal net = gross.subtract(tenPercent);

// Tránh new BigDecimal(0.1): constructor nhận double đã gần đúng.
```

## Value Object nên biết currency

```
public record Money(BigDecimal amount, String currency) {
    public Money {
        if (amount == null || currency == null || currency.isBlank()) {
            throw new IllegalArgumentException("Money must have amount and currency");
        }
    }

    public Money add(Money other) {
        if (!currency.equals(other.currency())) {
            throw new IllegalArgumentException("Currency mismatch");
        }
        return new Money(amount.add(other.amount()), currency);
    }
}
```

Đây là **mẫu minh họa**; production còn phải quy định scale được chấp nhận của từng currency, cách chuẩn hóa mã tiền, rounding, serialisation, overflow/range và conversion rate nếu bán xuyên biên giới.

## PostgreSQL: `NUMERIC` thay vì `floating point`

```
selling_amount NUMERIC(18, 2) NOT NULL
CHECK (selling_amount >= 0),
currency_code CHAR(3) NOT NULL
```

PostgreSQL khuyến nghị kiểu `numeric` cho các giá trị cần chính xác như tiền. `NUMERIC(18,2)` không tự động là lựa chọn tốt nhất cho tất cả currency: VND thường có thể chọn `NUMERIC(18,0)` theo chính sách không dùng phần thập phân, trong khi một số currency cần số chữ số lẻ khác. Nếu dữ liệu đa tiền tệ, nên quyết định một quy tắc chuẩn hóa hợp lệ và nhất quán cho từng đồng tiền, hoặc lưu đơn vị nhỏ nhất khi phù hợp.

## Vấn đề làm tròn: giảm 10.000 VND cho ba dòng hàng

Tính số tiền giảm theo tỷ trọng với line gross lần lượt 100.000, 100.000, 100.000:

- Tỷ lệ mỗi dòng:  $10.000 \div 3 = 3.333,333... \text{ VND}$ .
- Nếu làm tròn từng dòng:  $3.333 \times 3 = 9.999 \rightarrow$  thiếu 1 VND.
- Cách phân bổ deterministic minh họa:  $3.334 + 3.333 + 3.333 = 10.000$ .

Quy tắc phân bổ phải bảo đảm:

```
sum(allocated_discounts) == order_discount
sum(line_payables) + shipping + tax_adjustment == payable_total
```

Với marketplace, lưu allocation theo seller/line để refund một phần và settlement không phải đoán lại từ tỷ lệ hiện tại. Các khoản tax/shipping nếu được hoàn hay không phải được xử lý theo quy tắc riêng.

## 4.7. Giá tại ba thời điểm: display → quote → snapshot

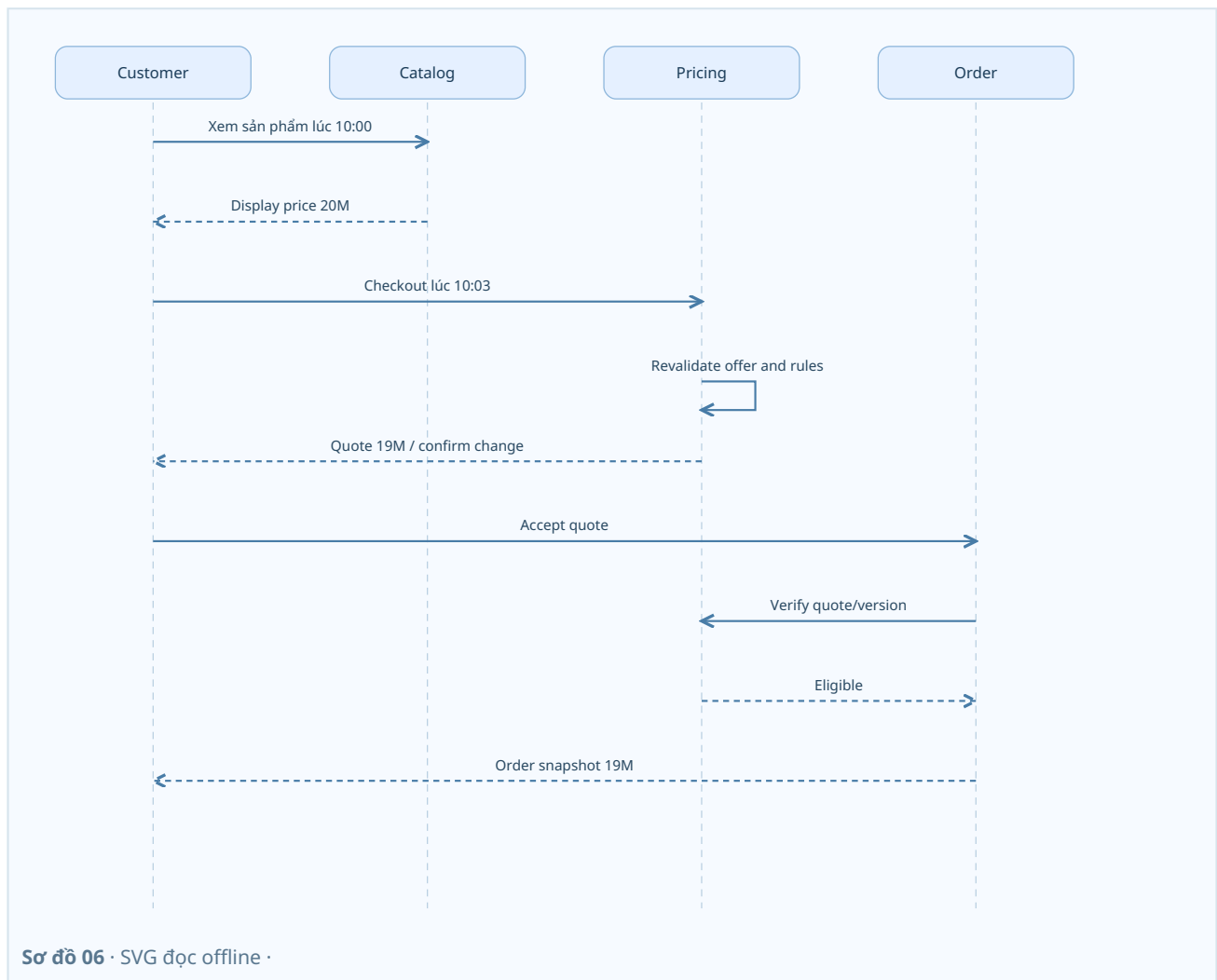
Giả sử Hoàng Phone đang bán một điện thoại:

| Thời điểm | Sự kiện                 | Giá                          |
|-----------|-------------------------|------------------------------|
| 10:00     | Khách mở trang sản phẩm | 20.000.000                   |
| 10:02     | Admin giảm giá          | 19.000.000                   |
| 10:03     | Khách nhấn checkout     | Cần xử lý theo chính sách    |
| 10:04     | Đơn được xác nhận       | Giá trong snapshot được chốt |

Có ít nhất hai mô hình hợp lệ:

**Model A — Reprice at checkout:** giá hiển thị mang tính tham khảo; checkout trả quote mới 19 triệu; khách cần xác nhận thay đổi. Dễ triển khai hơn, nhưng UX phải minh bạch.

**Model B — Time-bounded price guarantee:** hệ thống phát quote 20 triệu hợp lệ tới thời điểm nhất định. Cần lưu và xác minh quote, đồng thời xử lý trường hợp chiến dịch/ngân sách/voucher/tồn kho hết. Quote đã hết hạn phải được tính lại; quote chưa hết hạn không mặc nhiên bảo đảm tồn kho.



Một **Price Quote** có thể gồm:

```

{
  "quoteId": "QT-20260923-001",
  "buyerId": 101,
  "currency": "VND",
  "sellerOfferId": 701,
  "priceVersion": 42,
  "promotionVersion": 8,
  "lineSubtotal": "20000000",
  "discountTotal": "1000000",
  "shippingFee": "30000",
  "taxIncluded": true,
  "payableTotal": "19030000",
  "expiresAt": "2026-09-23T14:00:00+07:00"
}
  
```

`priceVersion` và `promotionVersion` giúp truy vết **rule nào đã tạo ra quote**, không đồng nghĩa với tự động thực thi ràng buộc hết hạn. `quoteId` phải liên kết với dữ liệu lưu phía server hoặc có chữ ký chống sửa đổi, ràng buộc buyer và cart fingerprint khi sử dụng. Nếu cần giữ quyền dùng voucher, có thể thêm **promotion reservation** được commit cùng order theo cơ chế nhất quán đã chọn.

## 4.8. Checkout Revalidation — tuyệt đối không tin tổng tiền từ frontend

### Bad case

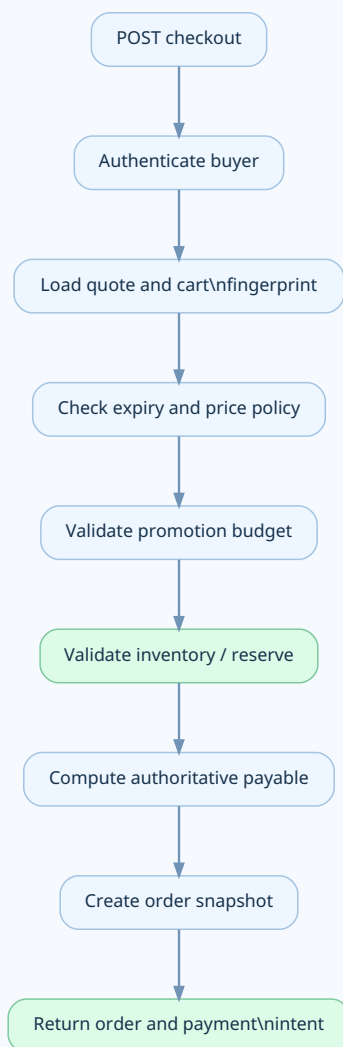
```
POST /orders
Content-Type: application/json

{"skuId":"PHONE-X","quantity":1,"totalAmount":1000}
```

Nếu backend lấy `totalAmount` trực tiếp để tạo order và charge payment, chỉ cần sửa request là giá đã sai.

### Thiết kế tốt hơn

```
Client gửi: offerId, quantity, address/method, voucher, quoteId, idempotencyKey
Server lấy: giá và rules có thẩm quyền, tình trạng voucher, quota, shipping, tax
Server tự tính: line totals, discounts, payable, snapshot
```



Sơ đồ 07 · SVG đọc offline ·

Nếu giá thay đổi ngoài policy cho phép, trả lỗi nghiệp vụ có cấu trúc như `PRICE_CHANGED` và quote mới để client quyết định, **không tự âm thầm charge giá khác đã được khách đồng ý**.

## Cửa sổ TOCTOU: kiểm tra đúng nhưng ghi sai

TOCTOU = *Time Of Check To Time Of Use*.

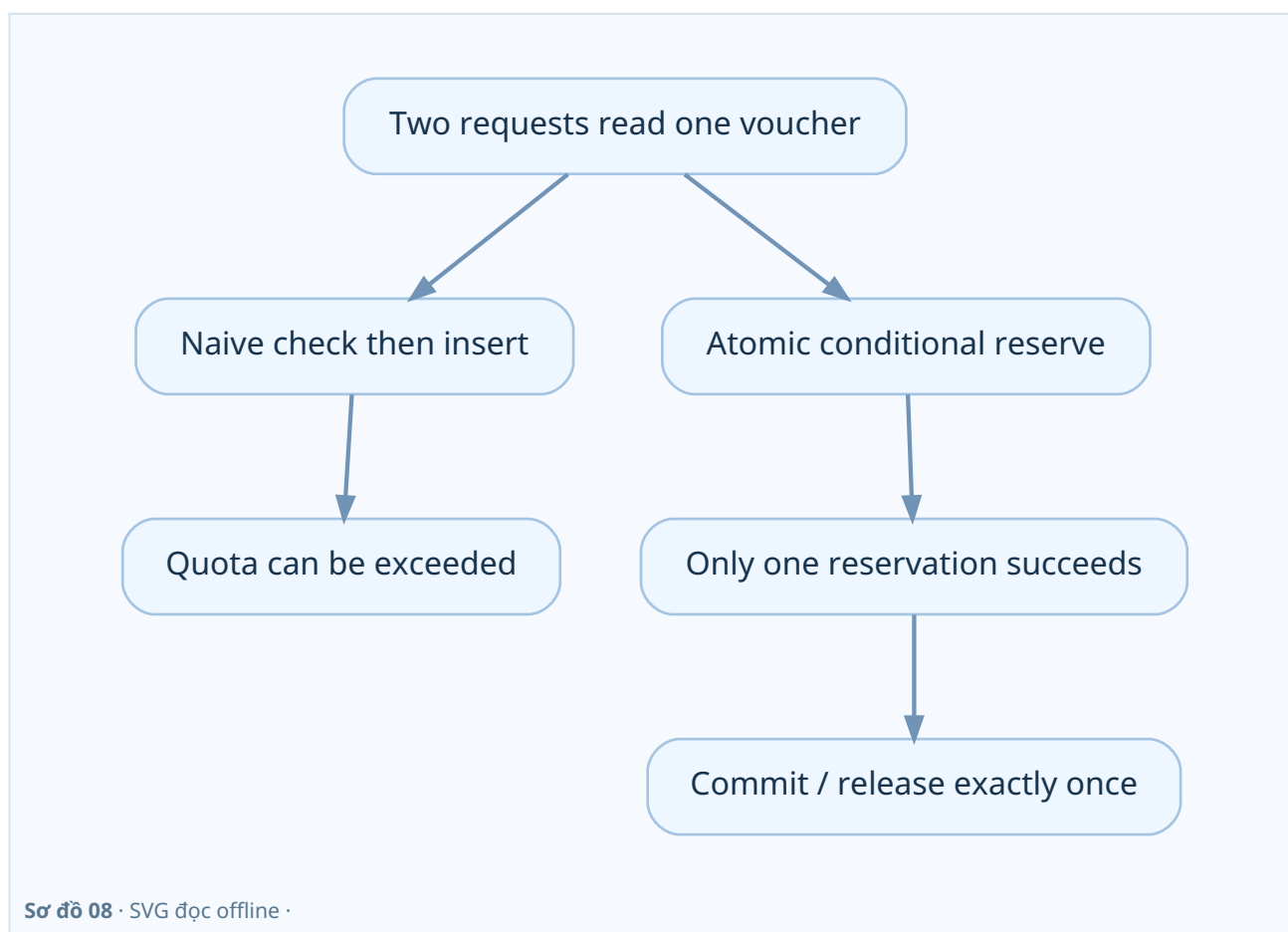
```
A: Check voucher remaining = 1
B: Check voucher remaining = 1
A: Create order and use voucher
B: Create order and use voucher
```

Dù cả hai lần kiểm tra đều chính xác ở thời điểm đọc, tổng cộng đã dùng voucher hai lần. Đây không phải bug của Redis hay Kubernetes; đây là lỗi **kiểm tra và tiêu thụ quyền lợi không được thực hiện nguyên tử**.

Một cách bảo vệ quota PostgreSQL:

```
UPDATE promotion_budget
SET reserved_uses = reserved_uses + 1
WHERE promotion_id = :promotion_id
AND reserved_uses + consumed_uses < usage_limit;
```

Chỉ tiếp tục khi `affected_rows = 1`. Để chống lặp theo mỗi user cần thêm khóa duy nhất/idempotency trên `promotion_redemption`, không chỉ global counter. `reserved_uses` phải được giải phóng chính xác một lần khi reservation hết hạn/hủy; `consumed_uses` phản ánh rule “dùng thành công” đã được business định nghĩa.



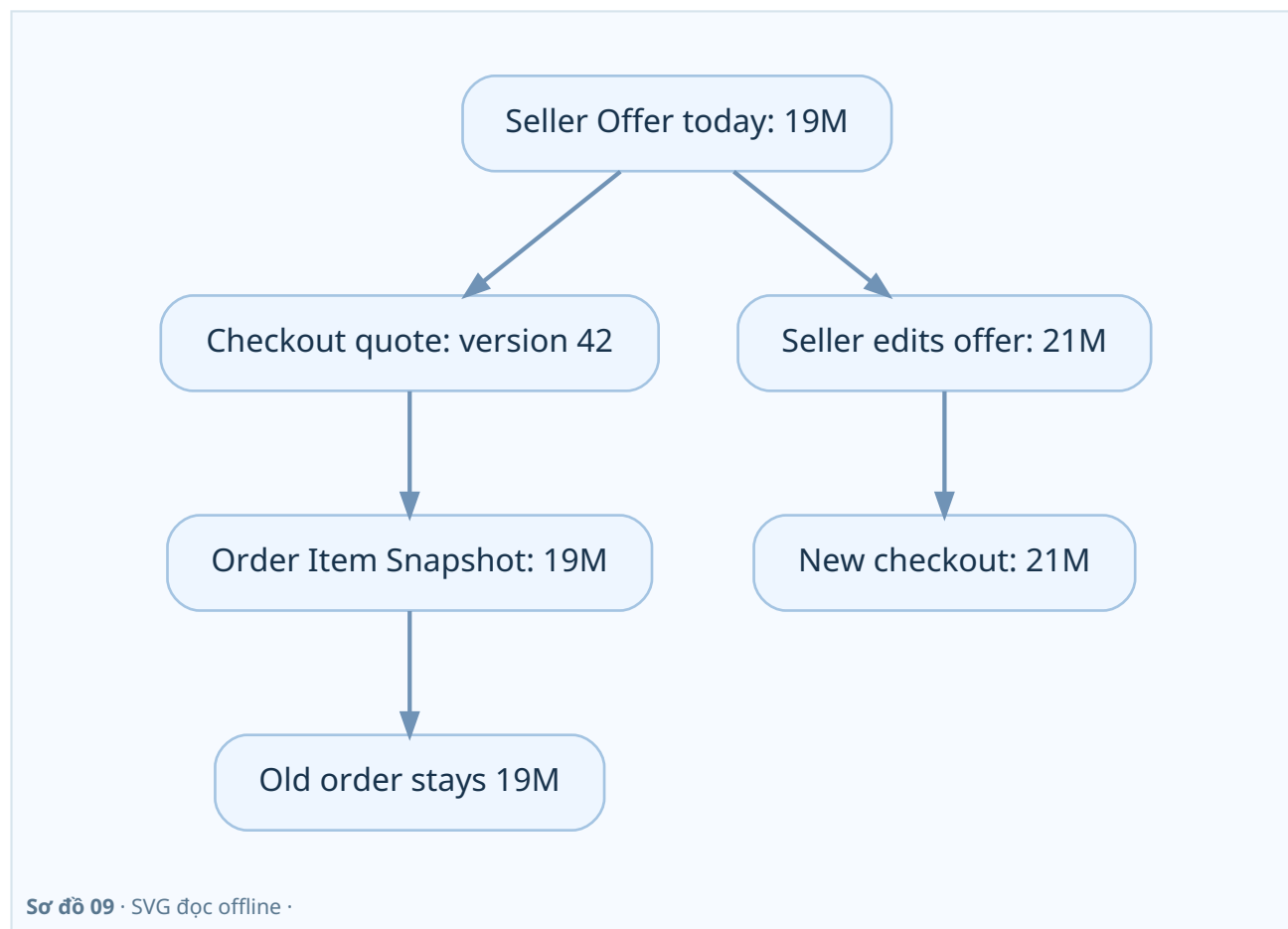
## 4.9. Order Snapshot: một hóa đơn cũ phải tự giải thích được

Một lỗi khá phổ biến: `order_items` chỉ lưu `sku_id` và `quantity`, còn màn hình lịch sử đơn luôn JOIN sang bảng giá hiện tại.

Sau hai tháng, seller xóa offer hoặc đổi giá. Hóa đơn cũ đột nhiên hiện số tiền mới, hoặc không còn dữ liệu để render.

Giải pháp: lưu snapshot những dữ liệu cần thiết tại thời điểm giao dịch; tham chiếu ID vẫn có ích để liên kết catalog, nhưng **không dùng nó để tái tính giá đã chốt**.

| Dữ liệu ở Order Item                                                         | Lý do                            |
|------------------------------------------------------------------------------|----------------------------------|
| <code>sku_id</code> , <code>seller_offer_id</code>                           | Liên kết nguồn hàng              |
| <code>product_name_snapshot</code> , <code>variant_snapshot</code>           | Hiển thị lịch sử khi catalog đổi |
| <code>unit_price_snapshot</code>                                             | Đơn giá tại thời điểm mua        |
| <code>quantity</code>                                                        | Số lượng đã xác nhận             |
| <code>gross_line_amount</code>                                               | Giá trước điều chỉnh của dòng    |
| <code>allocated_item_discount</code> , <code>allocated_order_discount</code> | Giải thích và refund từng dòng   |
| <code>net_line_amount</code>                                                 | Giá trị dòng sau giảm            |
| <code>currency_code</code>                                                   | Đồng tiền giao dịch              |
| <code>price_version</code> , <code>applied_rule_ids</code>                   | Audit và giải thích nguồn giá    |



Trong Marketplace, cần lưu **discount allocation theo Seller Order và Order Item**, cùng các khoản fee/commission/seller subsidy do ai chịu, để seller settlement và hoàn hàng một phần có thể đối soát. Không nên cố suy ra lịch sử từ cấu hình khuyến mãi đã được sửa hoặc xóa.

### Giá lịch sử khác “đóng băng mọi thứ”

Bạn vẫn có thể cập nhật một số thông tin vận hành như tracking number, ghi chú nội bộ và trạng thái giao hàng. Điều cần bất biến hoặc được điều chỉnh qua nghiệp vụ rõ ràng là **các giá trị đã thỏa thuận và vết thay đổi tài chính**, không phải toàn bộ record `orders`.

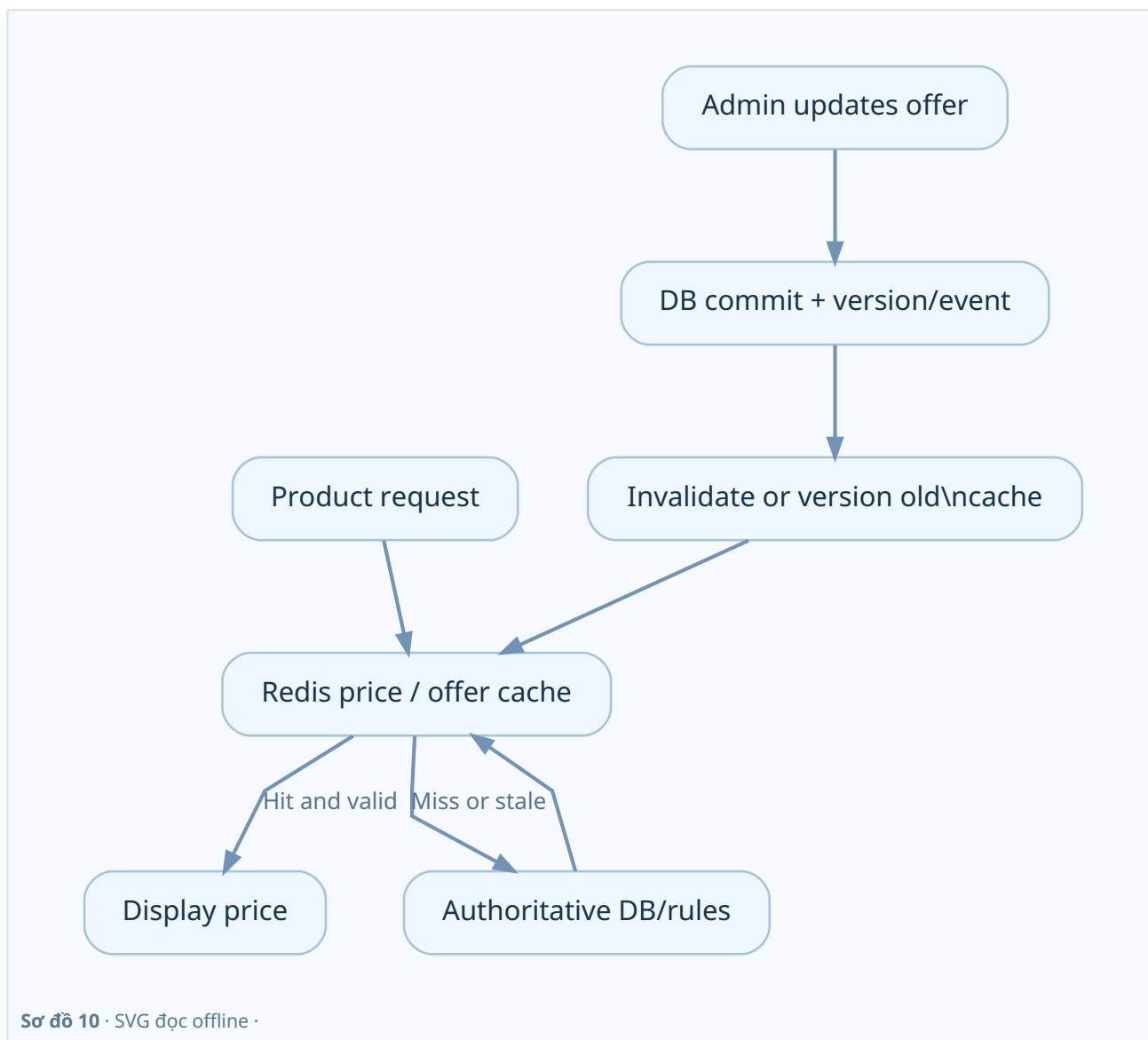
## 4.10. Cache giá: tăng READ throughput nhưng tạo bài toán consistency

Product page là một candidate điển hình để cache. Nhưng cache nguyên `product:1001` chỉ chứa một giá có thể sai khi: một SKU có nhiều seller; Flash Sale đổi theo thời gian; khách thuộc price tier khác; currency/channel khác.

Định nghĩa cache key theo **những chiều thực sự làm thay đổi kết quả**:

```
catalog:product:{productId}:v{catalogVersion}
offer:{sellerOfferId}:v{priceVersion}:channel:{channel}
```

Không cache tùy tiện toàn bộ price quote được cá nhân hóa bằng một key dùng chung. Nếu một quote phụ thuộc buyer/voucher/địa chỉ, cần cache ở phạm vi hợp lý hoặc không cache; không ghi dữ liệu nhạy cảm thô vào key.



## Bốn failure case phải dự liệu

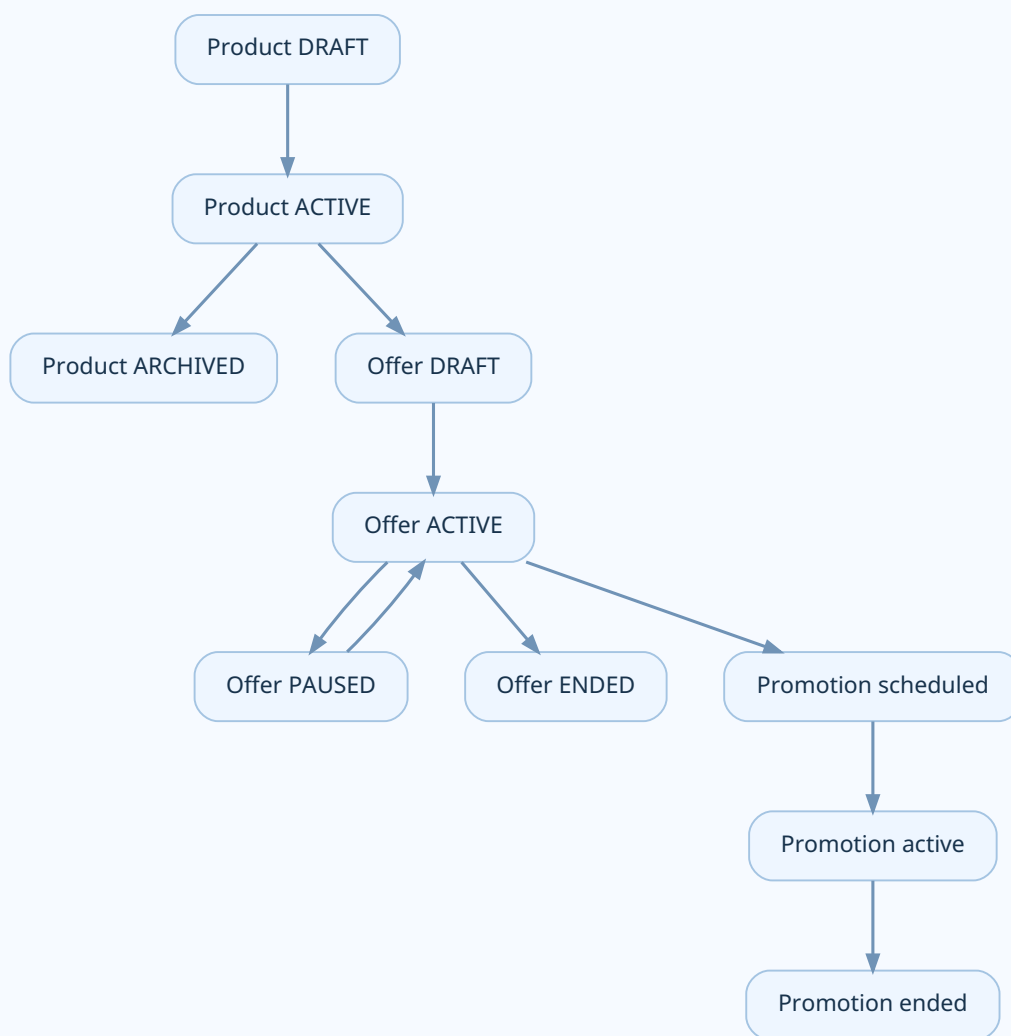
1. **Stale cache:** giá admin đã cập nhật nhưng cache vẫn còn TTL. Giải pháp có thể là versioned key, invalidate sau commit, TTL ngắn phù hợp hoặc nguồn cập nhật event; vẫn phải revalidate lúc checkout.
2. **Cache stampede:** giá hot vừa hết TTL, hàng nghìn request cùng đánh DB. Có thể dùng single-flight/per-key lock, stale-while-revalidate cho **display** khi business chấp nhận, hoặc prewarm. Không lấy stale estimate làm tiền charge.
3. **Delayed invalidation:** DB commit thành công nhưng event xóa cache thất bại. Có thể dùng transactional outbox để phát event đáng tin cậy hơn, TTL là giới hạn bảo vệ bổ sung.
4. **Clock boundary:** Flash Sale hết hạn nhưng key còn 5 phút TTL. TTL phải được giới hạn bởi thời điểm hết hiệu lực giá/chiến dịch nếu cache biểu diễn giá đang có hiệu lực.

## Phân biệt tốc độ và sự thật nghiệp vụ

Redis phục vụ hàng nghìn trang sản phẩm nhanh; PostgreSQL/rule authority cùng giao dịch checkout mới quyết định số tiền được chốt theo hợp đồng nghiệp vụ. Không phải mọi checkout đều cần bypass Redis: điều bắt buộc là **cơ chế xác nhận đúng rule/version/quota**, bất kể dữ liệu được đọc từ đâu.

## 4.11. Product Lifecycle và Price Lifecycle là hai state machine khác nhau

Một Product có thể tồn tại nhưng chưa được đăng bán; Offer có thể được lưu nhưng chưa có giá hợp lệ; chương trình giảm giá có thời điểm bắt đầu/kết thúc độc lập. Nếu gom tất cả vào `products.status`, bạn sẽ không biểu diễn được những trạng thái này.



Sơ đồ 11 · SVG đọc offline ·

Đây là sơ đồ **minh họa nhiều state machine liên quan**, không có nghĩa Product ACTIVE “tạo” Offer DRAFT qua một transition của cùng aggregate. Cần quy định permission và transitions độc lập theo domain.

### Price history và thời gian hiệu lực

Giá không chỉ có `updated_at`; có thể cần `valid_from` và `valid_to` để xác định mức giá tại thời điểm nào, đặc biệt khi đặt lịch khuyến mãi. Cần phân biệt:

- **Effective time:** mức giá có hiệu lực với giao dịch khi nào?
- **Recorded time:** hệ thống ghi nhận thay đổi khi nào?

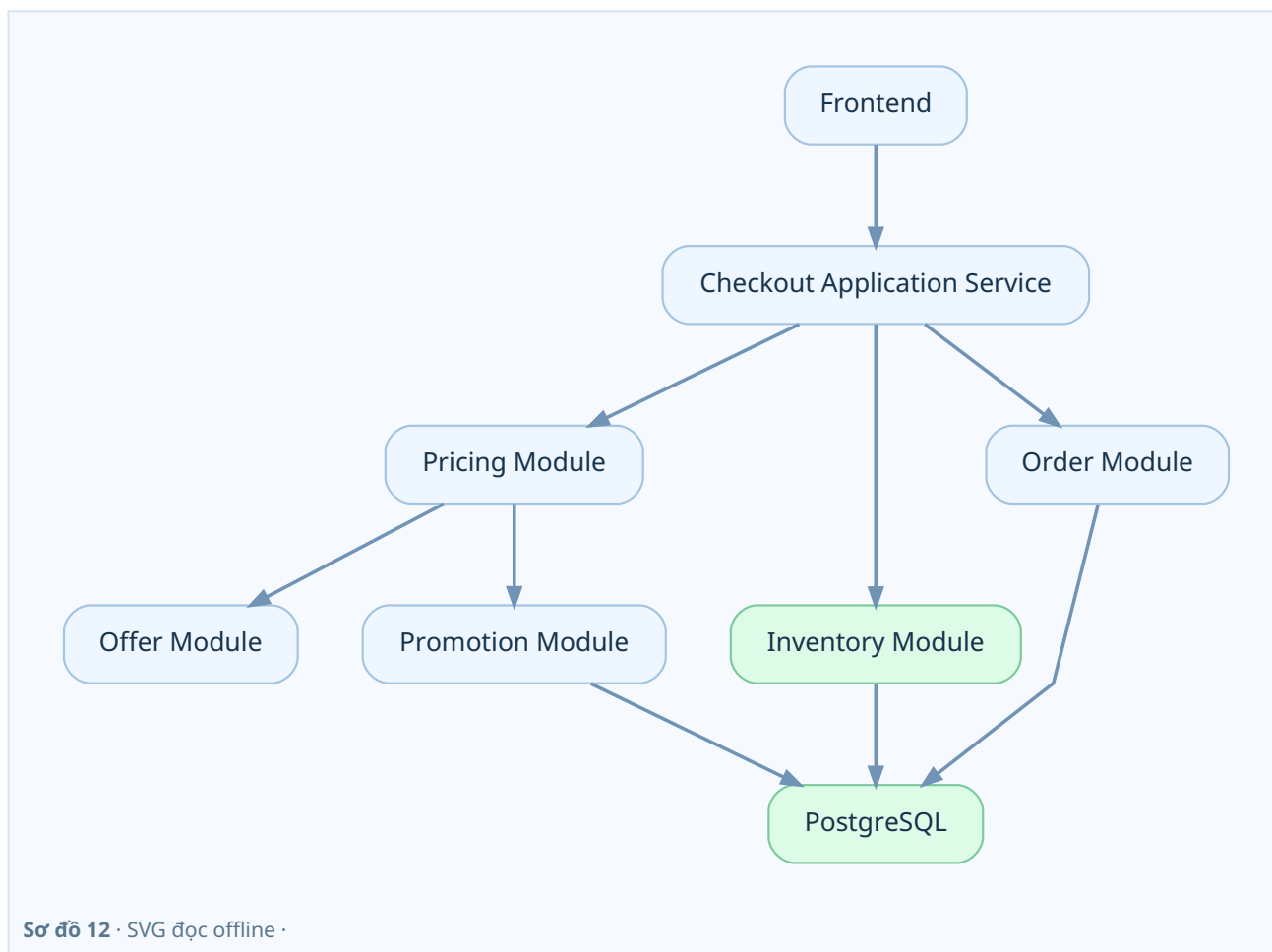
Hai khái niệm này giúp audit một thay đổi đặt lịch trước hoặc chỉnh sửa dữ liệu sai. Chỉ thêm hai cột thời gian chưa bảo đảm không có hai khoảng giá active chồng nhau; cần transaction và ràng buộc phù hợp (ví dụ exclusion constraint khi dùng PostgreSQL range).

## 4.12. Thiết kế ranh giới Pricing và Order trong Spring Boot

MVP không cần deploy riêng Pricing Service. Bắt đầu bằng **modular monolith**:

```
com.example.ecommerce
├── catalog/
│   ├── Product
│   └── Variant
├── offer/
│   ├── SellerOffer
│   └── OfferPriceHistory
├── pricing/
│   ├── PriceQuote
│   ├── PricingContext
│   ├── PricingPolicy
│   ├── DiscountAllocation
│   └── PricingApplicationService
├── promotion/
│   ├── PromotionRule
│   ├── PromotionBudget
│   └── Redemption
├── checkout/
│   └── CheckoutApplicationService
└── order/
    ├── Order
    └── OrderItemSnapshot
```

Kiến trúc trách nhiệm:



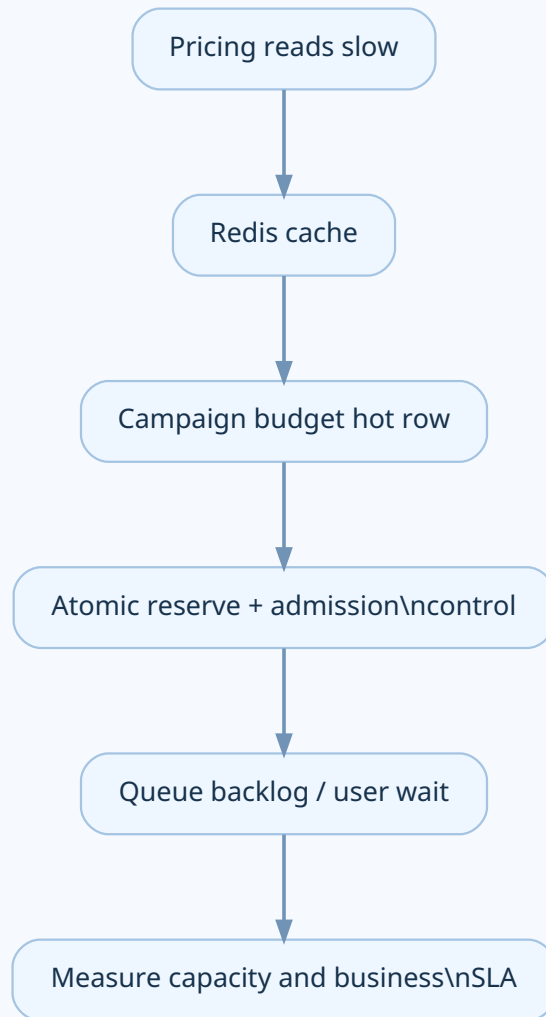
**Aggregate boundaries** không nhất thiết đi theo package hoặc table. Pricing có thể là domain service tính kết quả thuần, còn Order sở hữu snapshot và Inventory/Promotion sở hữu reservation. Chính xác transaction nào cùng commit tùy coupling và database ownership.

### Khi nào tách Pricing Service?

Tách riêng có thể hợp lý khi nhiều sản phẩm/kênh cùng dùng pricing engine; business cần deploy rule độc lập; team ownership đã rõ; tải CPU rule evaluation lớn hơn phần Order. Nhưng tách service tạo thêm network latency, versioning, cache consistency và distributed checkout flow. Chưa có nhu cầu thì package/module có ranh giới rõ là đủ.

## 4.13. Bottleneck Shifting: tối ưu một lớp, điểm nghẽn dịch sang lớp khác

Ở giai đoạn đầu, query lấy giá có thể chậm vì JOIN/rules quá phức tạp. Bạn cache offer. Read throughput tăng nhưng vào giờ Flash Sale, tất cả cùng dùng voucher 500 nghìn → **hot row** `promotion_budget`. Bạn tăng API Pods thì transaction chờ row lock nhiều hơn. Đưa yêu cầu vào queue sẽ giảm concurrent processing nhưng người mua phải đợi xác nhận.



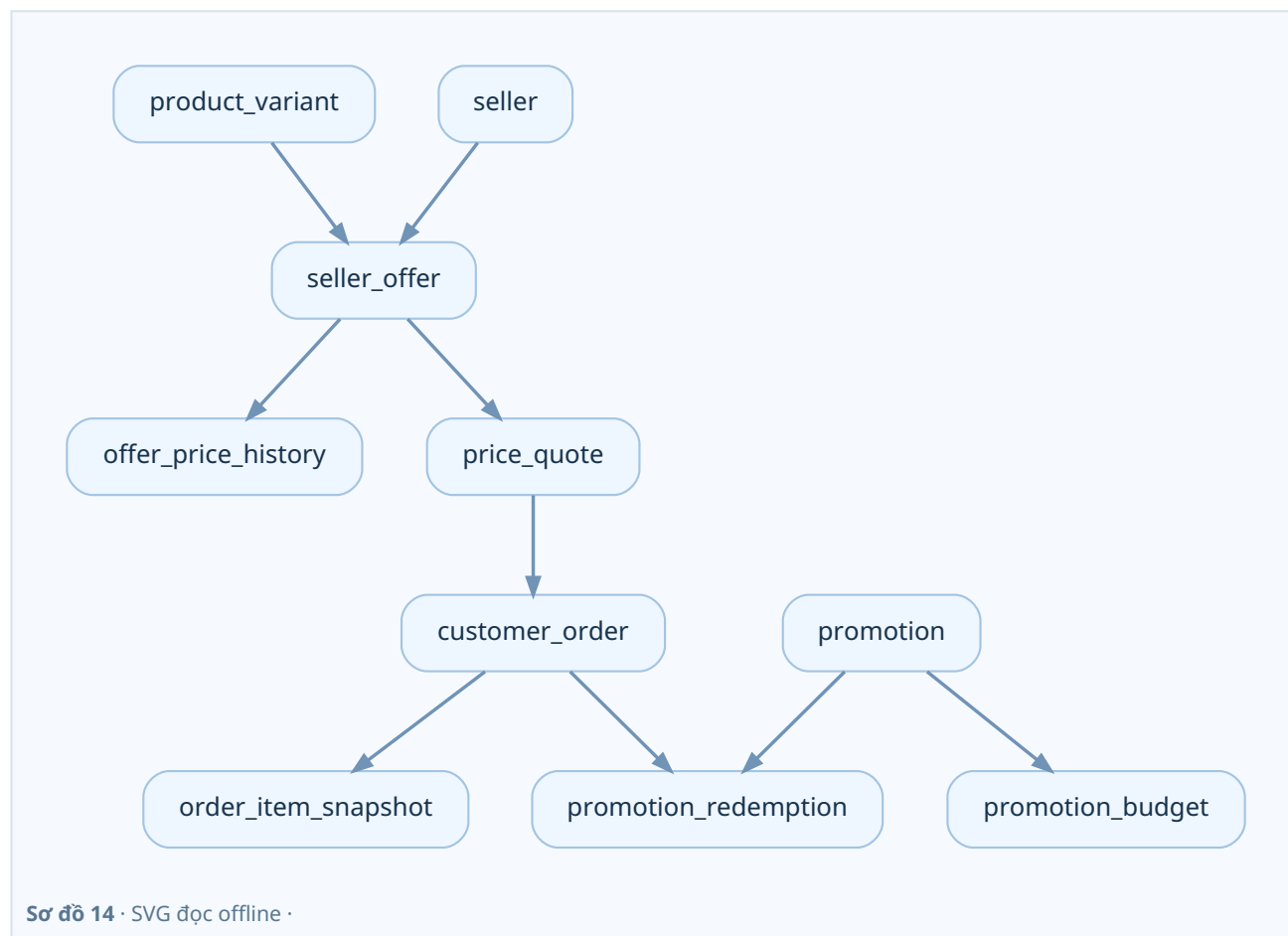
Sơ đồ 13 · SVG đọc offline ·

| Bottleneck                     | Triệu chứng quan sát                     | Cách tiếp cận                                                             |
|--------------------------------|------------------------------------------|---------------------------------------------------------------------------|
| Rule evaluation CPU            | p95 quote tăng khi nhiều campaign        | Pre-filter candidate rules, profiling, cache rule metadata                |
| DB connection pool             | Threads chờ lấy connection               | Pool sizing, giảm transaction duration, hạn mức concurrency               |
| Voucher hot row                | Lock wait cao ở một promotion            | Admission control, queue/partition workload, điều chỉnh thiết kế quota    |
| Pricing cache miss             | DB read đột biến sau TTL                 | Single-flight, prewarm, staggered TTL, version key                        |
| External tax/shipping quote    | Phụ thuộc bên ngoài timeout              | Timeout, fallback theo chính sách, hiển thị pending nếu không thể báo giá |
| Multi-seller order calculation | Nhiều item / seller, allocation phức tạp | Grouping, giảm query N+1, deterministic algorithm, đo CPU/DB              |

**Không thể tăng số Pod và hy vọng giải quyết mọi trường hợp.** Hạn mức ngân sách voucher, row lock và service đối tác là những tài nguyên chia sẻ không tự tăng khi thêm Spring Boot instances.

## 4.14. Database design tham khảo: ghi rõ lịch sử và tính toán

Không nên ép tất cả bảng dưới đây vào một transaction duy nhất bằng mọi giá. Đây là mẫu **modular monolith** dùng một PostgreSQL, sau đó mới cân nhắc tách ranh giới dữ liệu khi thực sự cần.



Các trường quan trọng nên được xem xét:

| Table                | Fields cốt lõi                                                         | Constraint nghiệp vụ                                |
|----------------------|------------------------------------------------------------------------|-----------------------------------------------------|
| seller_offer         | seller_id, sku_id, active_price, currency, price_version, status       | Giá không âm; phiên bản giá tăng có kiểm soát       |
| offer_price_history  | offer_id, amount, valid_from, valid_to, version                        | Khoảng hiệu lực theo policy; audit biến động        |
| price_quote          | buyer_id, quote_hash, payable_total, expires_at, status, price_version | Không dùng quote hết hạn/khác buyer hoặc cart       |
| promotion_budget     | usage_limit, reserved_uses, consumed_uses                              | Không vượt quota; counter không âm                  |
| promotion_redemption | promotion_id, buyer_id, order_id, request_key, status                  | Chống cùng request hưởng quyền lợi nhiều lần        |
| order_item_snapshot  | unit_price, discounts, net_amount, currency, price_version             | Không phụ thuộc giá hiện tại để giải thích order cũ |

**Đừng nhầm `order_id` duy nhất với `payment_attempt_id` duy nhất:** một order có thể có nhiều lần thử thanh toán. Pricing giữ vết số tiền của order; Payment xử lý dòng tiền và trạng thái giao dịch ở chương 05/ phần V.

## 4.15. Failure Scenarios và nguyên tắc xử lý

| Failure                                    | Hậu quả nếu thiết kế đơn giản        | Hướng xử lý                                                                               |
|--------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------|
| Admin cập nhật giá giữa render và checkout | Khách bị thu giá khác với kỳ vọng    | Reprice + yêu cầu xác nhận, hoặc giá được bảo đảm bằng quote hợp lệ                       |
| Redis chưa invalidate                      | Hiển thị giá cũ                      | TTL/version; checkout dùng rule/version có thẩm quyền                                     |
| Voucher cùng lúc hết lượt                  | Dùng vượt hạn mức                    | Atomic reservation + idempotency                                                          |
| Khách gửi lại checkout sau timeout         | Tạo hai đơn, sử dụng voucher hai lần | Idempotency key, lookup kết quả request trước                                             |
| Offer bị tạm dừng sau khi tạo cart         | Cart chứa item không thể bán         | Revalidate availability, trả lỗi nghiệp vụ                                                |
| Ship/tax service timeout                   | Tổng tiền không chắc chắn            | Không charge “đại”; timeout/retry, thông báo chưa đủ dữ liệu hoặc fallback được chấp nhận |
| Quote hết hạn khi khách confirm            | Checkout dùng giá cũ                 | Quote expiration policy, tạo quote mới                                                    |
| Một dòng hàng được hoàn trả                | Voucher toàn đơn bị phân bổ sai      | Lưu allocated discounts, định nghĩa refund policy                                         |
| Worker chết sau giữ quota                  | Voucher bị kẹt                       | Reservation expiry, recovery job, at-least-once idempotent                                |
| Multi-currency checkout                    | Cộng VND với USD không có chuyển đổi | Money currency guard, FX rate snapshot và settlement currency riêng                       |

### Three invariants kiểm tra xuyên suốt

```
I1. order.payable_total = sum(order_item.net_amount)
    + shipping + applicable_taxes - order_level_adjustments_not_yet_allocated

I2. promotion.reserved_uses + promotion.consumed_uses <= promotion.usage_limit
    (for limited-use promotions)

I3. Existing order snapshots do not change when active_offer_price changes.
```

I1 là **một biểu thức có điều kiện về mô hình phân bổ**: nếu toàn bộ voucher order đã được phân bổ xuống từng item thì `order_level_adjustments_not_yet_allocated = 0`. Không trừ cùng một discount hai lần. Một số phí/thuế có thể nằm ở item hoặc order level, phải ghi rõ nơi ghi nhận để phép cộng đúng.

## 4.16. Hands-on Lab — từ một cột giá tới Checkout Quote

Thư mục thực hành kèm theo ở `labs/chapter-04/` gồm schema PostgreSQL, các query có thể chạy và bộ test scenario. Mục tiêu không phải tạo production-grade promotion engine ngay lập tức mà tự kiểm chứng các invariants quan trọng.

## Lab A — Khởi tạo hai Seller Offer cho cùng SKU

- Tạo Seller A với giá 20.000.000 VND, Seller B với giá 19.700.000 VND.
- Xác nhận hai offer có `price_version` độc lập.
- Admin đổi giá Seller A lên 21.000.000, Seller B không thay đổi.
- Đọc lịch sử giá và kiểm tra đơn đã tạo vẫn có giá snapshot.

## Lab B — Voucher chỉ còn một lượt

Mở hai session/transaction PostgreSQL cùng cố reserve voucher một lượt. Kiểm chứng chỉ một transaction được tăng `reserved_uses`, transaction còn lại trả `affected_rows = 0` sau khi chờ. Ghi lại row lock wait khi dùng `pg_stat_activity` hoặc log trong môi trường thử nghiệm.

## Lab C — Cố tình phá checkout

Gửi payload giả định:

```
{"sellerOfferId":701,"quantity":1,"clientTotal":1000}
```

Backend phải phớt lờ `clientTotal` và tính lại bằng dữ liệu offer/rules của server. Nếu giá/quote thay đổi, trả kết quả nghiệp vụ để người dùng xác nhận chứ không charge theo tổng frontend.

## Lab D — Phân bổ và hoàn một phần

- Giỏ có ba dòng hàng, tổng voucher 10.000 VND.
- Kiểm tra tổng phân bổ chính xác là 10.000 VND.
- Hủy một dòng; không tính lại giá đơn cũ bằng campaign hiện tại.
- Xác định theo business policy xem phần voucher của item bị hủy được thu hồi, giữ hay tính lại, rồi hạch toán nhất quán.

## Lab E — Cache staleness drill

- Cache giá version 1.
- Admin đổi DB sang version 2.
- Cố tình giữ cache version 1 ở trang product.
- Gửi checkout: phải xác thực lại giá/rule và đưa ra quyết định theo chính sách đã chọn.

## 4.17. Chapter Review — tự trả lời như một system designer

1. Một SKU có ba seller và ba mức giá; bảng nào sở hữu giá và tại sao?
2. Vì sao cart total không nên là số tiền backend charge trực tiếp?
3. Hai request cùng dùng voucher còn một lượt: cần atomic ở đâu?
4. `BigDecimal` giải quyết vấn đề nào và **không** giải quyết vấn đề nào (ví dụ sai thứ tự áp voucher)?
5. Làm sao bảo đảm tổng giảm phân bổ không lệch 1 đơn vị tiền?
6. Giá đã chốt trong Order có được cập nhật khi admin đổi Offer không?
7. Cache key thiếu seller hoặc channel có thể gây lỗi gì?
8. Khi nào Price Quote có thể bảo đảm giá trong 5 phút, và điều kiện bổ sung nào cần cho voucher/stock?
9. Thêm 10 Pricing Pod có giúp gì khi voucher hot row đang bị lock không?
10. Nếu checkout gồm hai seller và một voucher toàn giỏ, tại sao cần lưu allocation?

## Key Takeaways

- **Giá là kết quả của một quyết định nghiệp vụ theo ngữ cảnh**, không đơn giản là một thuộc tính tĩnh của Product.
- Product, SKU và Seller Offer có ranh giới riêng; mô hình marketplace thường khiến giá thuộc về lời chào bán của seller.
- Display price, cart estimate, checkout quote và order snapshot khác nhau về vai trò và mức độ cam kết.
- Pricing pipeline cần thứ tự rule, stacking, rounding và discount allocation được xác định.
- Tiền tệ phải có biểu diễn chính xác và quy tắc xử lý từng currency.
- Checkout không tin tổng tiền client; cần xác nhận lại giá, điều kiện voucher, quota và tồn kho theo chính sách.
- Atomic voucher reservation xử lý race condition; nó vẫn có thể tạo hot row và lock contention.
- Redis tăng tốc READ nhưng yêu cầu chiến lược invalidation, time boundary và revalidation rõ ràng.
- Order snapshot là bằng chứng nghiệp vụ cho lịch sử giao dịch, refund và đối soát.

## Kết nối chương tiếp theo

**Chapter 05 — Order Lifecycle:** Sau khi đã biết “đơn hàng phải chốt giá gì”, câu hỏi kế tiếp là “đơn hàng được phép đi qua những trạng thái nào, và hệ thống phải làm gì khi thanh toán, giữ hàng, vận chuyển hoặc hoàn trả gặp lỗi?”. Chương 05 sẽ biến checkout thành một **state machine có transition rules và failure recovery**.

## Tài liệu tham khảo

1. PostgreSQL Documentation — Numeric Types: <https://www.postgresql.org/docs/current/datatype-numeric.html>
2. PostgreSQL Documentation — Transactions: <https://www.postgresql.org/docs/current/tutorial-transactions.html>
3. PostgreSQL Documentation — Explicit Locking: <https://www.postgresql.org/docs/current/explicit-locking.html>
4. Java `BigDecimal`: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/math/BigDecimal.html>
5. Shopify Help — Setting sale prices: <https://help.shopify.com/en/manual/products/details/product-pricing/sale-pricing>
6. Shopify Help — Product prices: <https://help.shopify.com/en/manual/products/details/product-pricing>

**Biên tập:** Số tiền, traffic và chính sách discount trong chương là ví dụ minh họa có ghi giả định. Các đoạn schema/Java là mã học tập, không hàm ý đã tích hợp đầy đủ tax, FX, invoicing, ledger, fraud, migration hoặc quốc gia cụ thể. Lab phải được chạy trên môi trường thử nghiệm trước khi dùng làm nền tảng production.

# INSIDE E-COMMERCE

## Volume 1 — Understanding E-commerce

### Chapter 05 — Order Lifecycle: Từ nút Mua ngay đến hoàn tất, hủy và hoàn tiền

**Câu hỏi dẫn đường:** Một khách mua chiếc điện thoại cuối cùng. Hệ thống giữ hàng và tạo đơn, cổng thanh toán trả timeout, khách nhấn hủy, nhưng webhook báo trả tiền thành công đến sau đó. Liệu một cột `orders.status` có mô tả và giải quyết chính xác mọi thứ?

**Phạm vi:** Tiếp tục cửa hàng giả định **Hoàng Phone → Hoàng Marketplace** của chương 01–04. Trạng thái, SLA giữ hàng và chính sách hoàn tiền dưới đây là ví dụ thiết kế, không phải quy tắc bất biến áp dụng cho mọi nhà bán hàng hay nhà cung cấp thanh toán. `PaymentIntent` của một PSP không đồng nghĩa với trạng thái đơn hàng nội bộ của doanh nghiệp.

## Sau chương này, bạn sẽ có khả năng

- Vẽ state machine và bảng chuyển trạng thái có **guard/action rõ ràng**, thay cho các lệnh `setStatus(...)` tự do.
- Phân biệt **Order, Payment, Reservation, Fulfillment, Shipment và Refund** theo vòng đời độc lập.
- Thiết kế **atomic checkout** với order snapshot, inventory reservation và outbox trong một database transaction khi chúng cùng datastore.
- Kiểm soát hai tác vụ cạnh tranh: cancel vs payment callback; cancel vs shipping; expiry vs payment success.
- Xử lý timeout không rõ kết quả, duplicate webhook, out-of-order event, retry và worker crash.
- Phân biệt hủy trước giao với return/refund sau giao; xử lý một phần khi đơn có nhiều dòng hoặc nhiều seller.
- Tìm bottleneck ở hot order row, worker/queue backlog, PSP quota và reconciliation.
- Có SQL Lab để tự tái hiện các lỗi và kiểm tra invariant cốt lõi.

## 5.1. Bad case: biến mọi thứ thành một `orders.status`

Phiên bản demo đầu tiên thường viết như sau:

```
CREATE TABLE orders (  
  id BIGINT PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  status VARCHAR(30) NOT NULL,
```

```
total_amount NUMERIC(18, 2) NOT NULL
);
```

```
order.setStatus("SUCCESS");
repository.save(order);
```

Nhưng **SUCCESS** nghĩa là gì? Đã tạo đơn, đã trừ tiền, đã thu tiền, đã giao hàng hay đã hết hạn đổi trả? Nếu webhook thanh toán gửi lại **SUCCESS** sau khi nhân viên vừa cập nhật **CANCELLED**, thằng nào là sự thật? Nếu người mua trả lại một trong ba món, trạng thái đơn nên là **SUCCESS**, **REFUNDED** hay **PARTIALLY\_RETURNED**?

Đây không phải lỗi riêng của JPA. **Lỗi nằm ở domain modeling: ép nhiều sự kiện và vòng đời độc lập thành một biến trạng thái nghèo ngữ nghĩa.**



**Feynman analogy:** Phiếu gọi món ở quán không thể chỉ ghi “XONG”, bởi bếp nấu xong, thu ngân đã thu tiền và người giao hàng đã mang đồ đi là ba sự việc khác nhau. E-commerce cũng vậy.

## Ba nhầm lẫn gây lỗi lớn

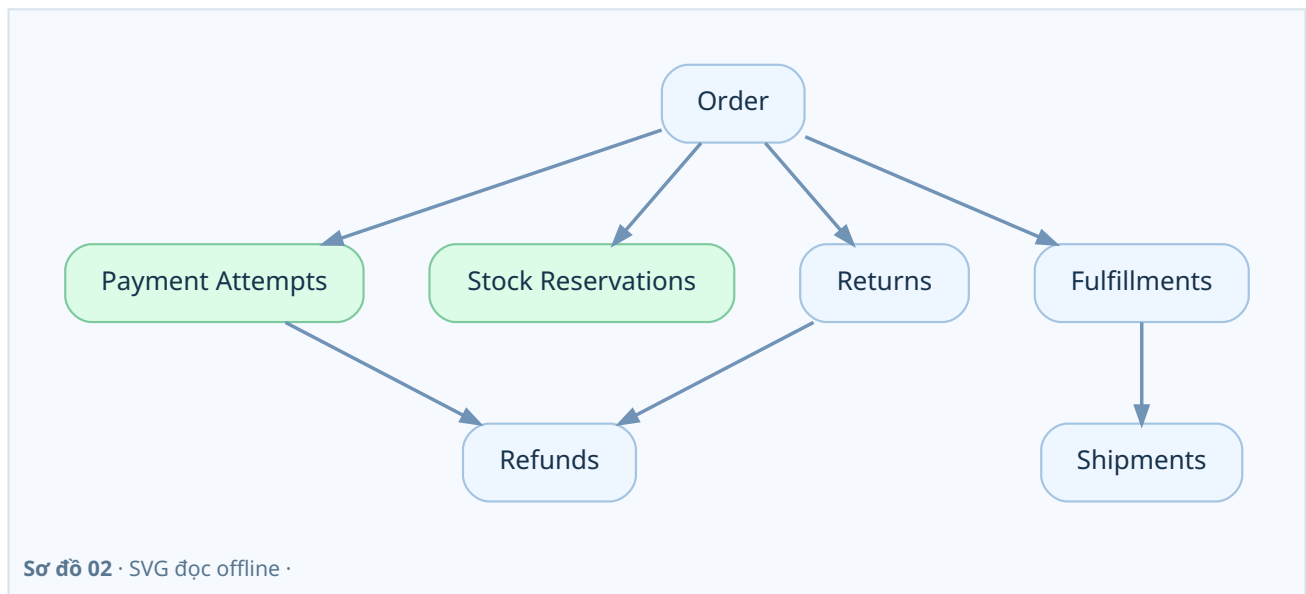
1. **State  $\neq$  Event:** **PAID** là trạng thái; **PaymentConfirmed** là điều vừa xảy ra. Một event có thể dẫn đến thay đổi nhiều projection/trạng thái.
2. **Command  $\neq$  Fact:** **CancelOrder** là yêu cầu; **OrderCancelled** chỉ được ghi sau khi cancel thực sự thành công theo rule.
3. **HTTP response  $\neq$  Business outcome:** Timeout không chứng minh giao dịch thanh toán hay tạo đơn đã thất bại.

## 5.2. Tách sáu lifecycle liên quan nhưng không đồng nhất

Một hệ thống đủ thực dụng không nhất thiết tạo sáu microservice, nhưng nên nhận diện sáu domain entity/process khác nhau:

| Đối tượng                 | Câu hỏi sở hữu                                                          | Ví dụ trạng thái                      |
|---------------------------|-------------------------------------------------------------------------|---------------------------------------|
| Order                     | Khách và cửa hàng đã cam kết mua bán tới bước nào?                      | PENDING_PAYMENT, CONFIRMED, CANCELLED |
| Payment Attempt / Payment | Có khoản thanh toán được khởi tạo, đang xử lý, thành công hay thất bại? | PENDING, SUCCEEDED, FAILED, UNKNOWN   |

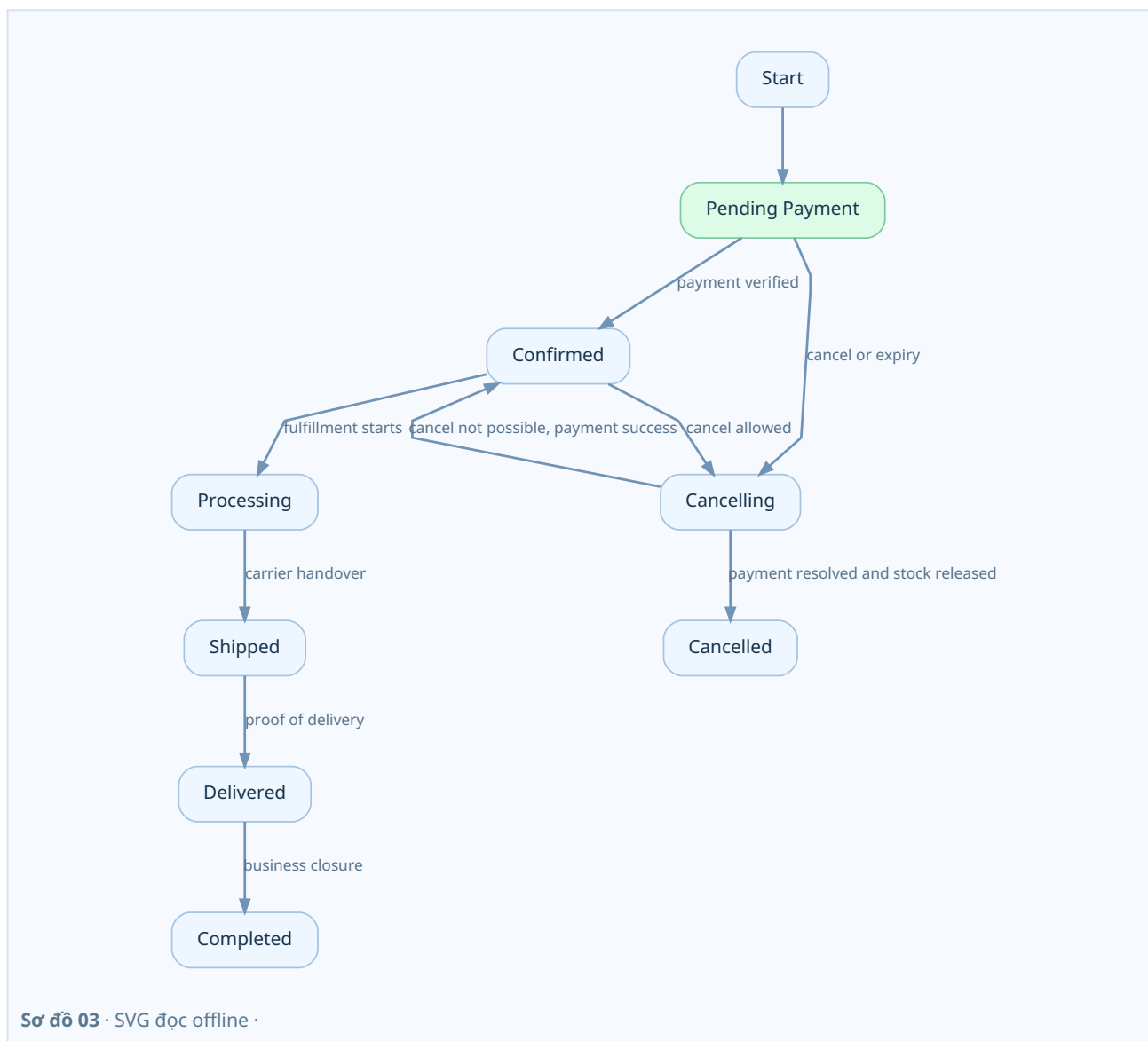
| Đối tượng             | Câu hỏi sở hữu                                   | Ví dụ trạng thái                         |
|-----------------------|--------------------------------------------------|------------------------------------------|
| Inventory Reservation | Bao nhiêu SKU đã giữ cho đơn, tới lúc nào?       | ACTIVE, CONSUMED, RELEASED, EXPIRED      |
| Fulfillment           | Kho đã phân bổ, nhặt, đóng gói tới đâu?          | NEW, PICKING, PACKED, HANDOVER           |
| Shipment              | Đơn vị vận chuyển đã nhận và giao ra sao?        | CREATED, IN_TRANSIT, DELIVERED, LOST     |
| Return / Refund       | Hàng có được nhận trả và tiền có hoàn hay không? | REQUESTED, APPROVED, SUCCEEDED, REJECTED |



Mối liên hệ là business reference, không đồng nghĩa bắt buộc một bảng cho mỗi ô. Trong Marketplace một **checkout group** còn có thể tách thành nhiều **seller order** và nhiều shipment. Đừng để UI chỉ hiển thị “Đơn đã giao” che mất chi tiết các gói hàng khác nhau.

### 5.3. Order lifecycle là một State Machine có điều kiện

Ta chọn lifecycle tối giản cho **một đơn prepaid, một seller**, payment thu tiền sau khi reserve thành công:



Đây là **mô hình minh họa**, không phải tập trạng thái chuẩn quốc tế. **COMPLETED** là mốc business closure do doanh nghiệp quy định; không đồng nghĩa hết hạn bảo hành hay cấm refund. **CANCELLING** là trạng thái trung gian cần thiết khi cancellation phụ thuộc vào gateway/kho/shipper không thể xử lý tức thì.

### Transition table: contract thực sự của backend

| Từ              | Command/Event          | Điều kiện (guard)                                       | Sang       | Side effect nghiệp vụ                                     |
|-----------------|------------------------|---------------------------------------------------------|------------|-----------------------------------------------------------|
| PENDING_PAYMENT | PaymentVerified        | Đúng attempt, đúng amount/currency, chưa cancel thực sự | CONFIRMED  | Đánh dấu giữ hàng đã gắn với đơn; yêu cầu fulfillment     |
| PENDING_PAYMENT | CancelRequested        | Trong cửa sổ hủy                                        | CANCELLING | Chặn tạo payment attempt mới; yêu cầu xác minh/cancel PSP |
| CANCELLING      | PaymentSettledAsUnpaid | Không còn khoản có thể hoàn tất bất ngờ                 | CANCELLED  | Release reservation đúng một lần                          |

| Từ         | Command/Event        | Điều kiện (guard)                       | Sang                                                                           | Side effect nghiệp vụ                                |
|------------|----------------------|-----------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------|
| CANCELLING | PaymentSucceededLate | Thanh toán thành công dù đã yêu cầu hủy | Theo chính sách: <b>CONFIRMED</b> hoặc tiếp tục <b>CANCELLING</b> để hoàn tiền | Không release bù; quyết định hoàn tiền/tiếp tục giao |
| CONFIRMED  | StartFulfillment     | Chưa có active cancel, đủ hàng          | <b>PROCESSING</b>                                                              | Tạo fulfillment                                      |
| PROCESSING | CarrierAccepted      | Handover hợp lệ                         | <b>SHIPPED</b>                                                                 | Ghi tracking                                         |
| SHIPPED    | ProofOfDelivery      | Gói hàng tương ứng xác nhận giao        | <b>DELIVERED</b>                                                               | Ghi POD; tiếp tục hậu mãi                            |

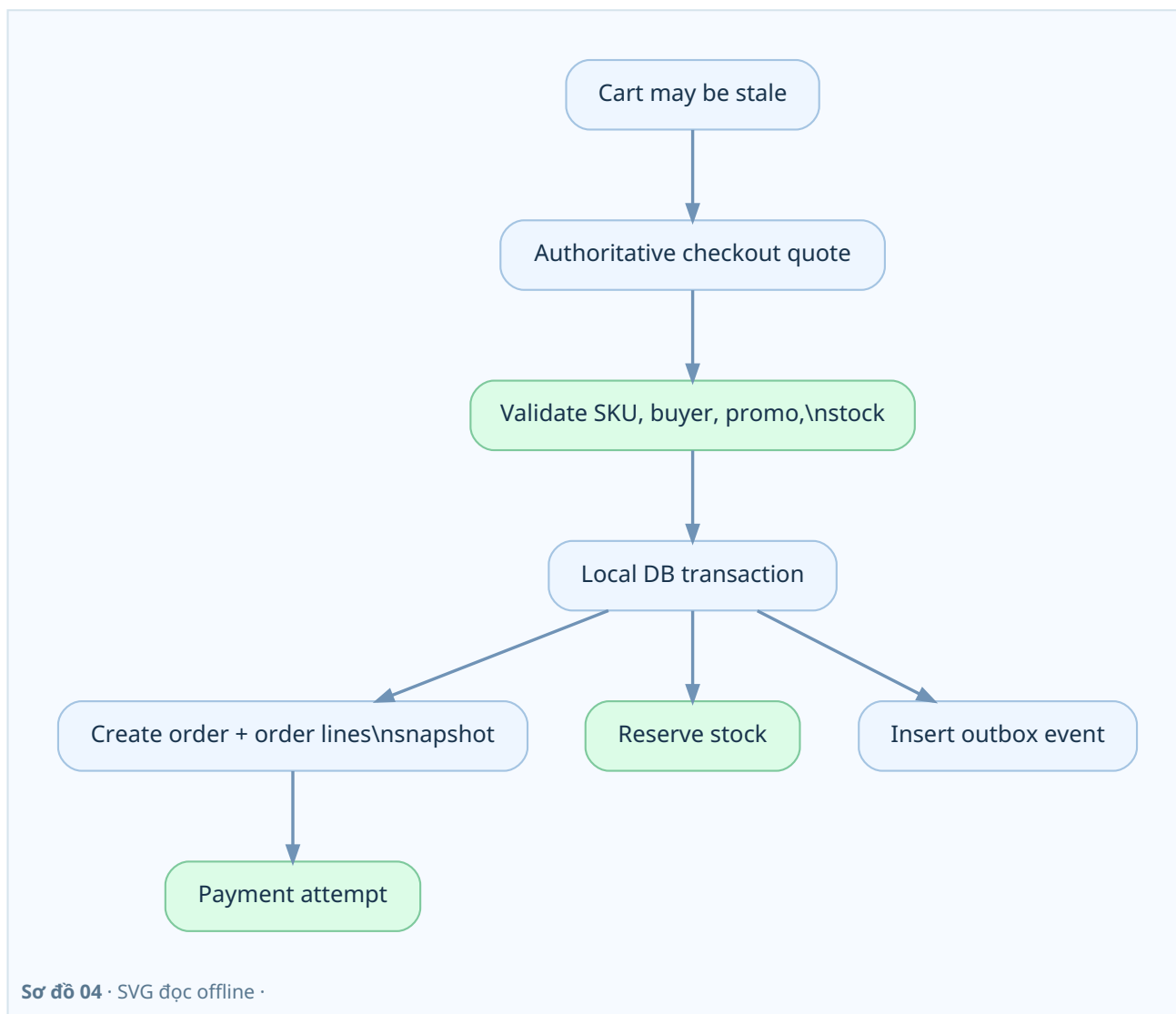
**Quan trọng:** Điều kiện `PaymentSettledAsUnpaid` phải dựa trên kết quả đủ tin cậy từ PSP/reconciliation, không phải suy ra từ việc request HTTP bị timeout.

## Invariant cần cố định trước khi viết code

- **CONFIRMED** trong prepaid flow cần payment evidence hợp lệ, hoặc một ngoại lệ thanh toán khác được model hóa rõ (COD, pay-later).
- Một reservation chỉ có **một kết cục terminal**: **CONSUMED**, **RELEASED** hoặc **EXPIRED** (nếu expiry là một biến thể release; không đồng thời).
- Số lượng `reserved_qty` trên kho không âm và không vượt `on_hand_qty` theo quy tắc kho đơn giản của lab.
- Một lệnh gửi lại cùng idempotency key với cùng nội dung trả cùng business result; cùng key mà nội dung khác phải từ chối.
- Không chuyển từ **CANCELLED** về **CONFIRMED** chỉ vì nhận event cũ. Late money phải thành dispute/refund/reconciliation case.
- Status snapshot có thể mất đồng bộ ngắn với projection nhưng event/audit log không được làm mất dấu các chuyển trạng thái đã commit.

## 5.4. Product → Price Quote → Order Snapshot: giữ bằng chứng giao dịch

Chương 04 đã tách `price` hiện tại khỏi `price snapshot`. Chapter 05 nối logic đó với lịch sử đơn:



Một dòng đơn hàng tối thiểu nên giữ: `sku_id`, `seller_id`, tên/biến thể để hiển thị lịch sử, `qty`, `unit_price`, discount allocation, tax/shipping allocation nếu domain yêu cầu, currency và policy version. Không query giá catalog hiện tại để tính lại đơn cũ.

**Vấn đề khó:** quote hết hạn giữa lúc khách bấm confirm. Contract cần nói rõ giá được cố định tại thời điểm nào: khi tạo quote, khi server chấp nhận checkout, hay chỉ sau khi thanh toán? Không có lựa chọn duy nhất cho mọi cửa hàng. Điều cần có là rule minh bạch, server enforce và snapshot có thể audit.

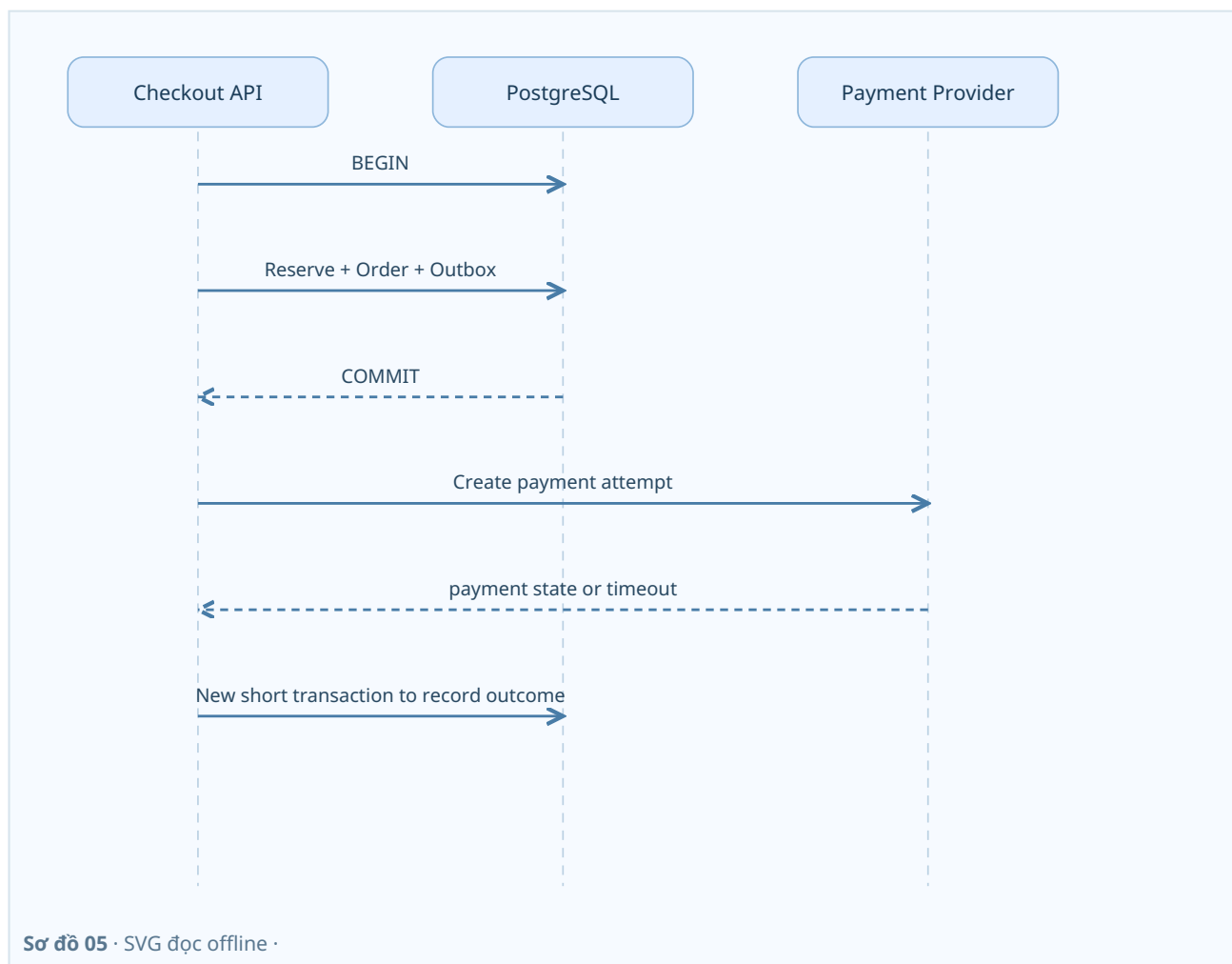
## 5.5. Transaction Boundary: cái gì thực hiện nguyên tử được, cái gì không?

Nếu order và inventory nằm cùng PostgreSQL, checkout có thể thực hiện nguyên tử trong **transaction** ngắn:

```
BEGIN;
-- 1. Idempotency row/unique key, validate price version.
-- 2. Conditionally reserve inventory.
-- 3. Insert orders and order_items snapshots.
-- 4. Insert stock_reservations.
```

```
-- 5. Insert outbox event OrderPlaced.  
COMMIT;
```

Nếu bước nào thất bại, rollback toàn bộ. Không để backend tạo order mà không reserve hàng, hoặc reserve hàng nhưng không tạo order do lỗi ở giữa.



**Không gọi PSP bên trong transaction đang giữ row lock kho/đơn.** PSP có thể timeout vài giây hoặc cần người dùng xác thực; transaction bị giữ sẽ tăng lock wait, connection occupancy và deadlock risk. PostgreSQL mô tả row lock tồn tại tới cuối transaction và cảnh báo tránh giữ transaction mở dài. Tham khảo PostgreSQL Explicit Locking ở cuối chương.

Khi Order và Inventory chuyển thành database riêng, local `@Transactional` trên Order Service không thể rollback kho tại service khác; cần workflow phân tán và compensation, được đi sâu ở Volume 6.

## 5.6. Business command không được là `PATCH /orders/{id}/status`

API ngây thơ:

```
PATCH /orders/123  
{ "status": "DELIVERED" }
```

Nếu khách hoặc client có thể tự chọn trạng thái, bạn đã trao quyền bỏ qua business guards. API nên mô tả **ý định nghiệp vụ**:

```
POST /orders/123/cancellations
Idempotency-Key: user-101-cancel-order-123

{"reason": "CHANGED_MIND"}
```

```
POST /fulfillments/ful-123/handover
Idempotency-Key: carrier-handover-xyz

{"trackingNumber": "TRK-001"}
```

```
POST /webhooks/payment
Stripe-Signature: ...

{...provider payload...}
```

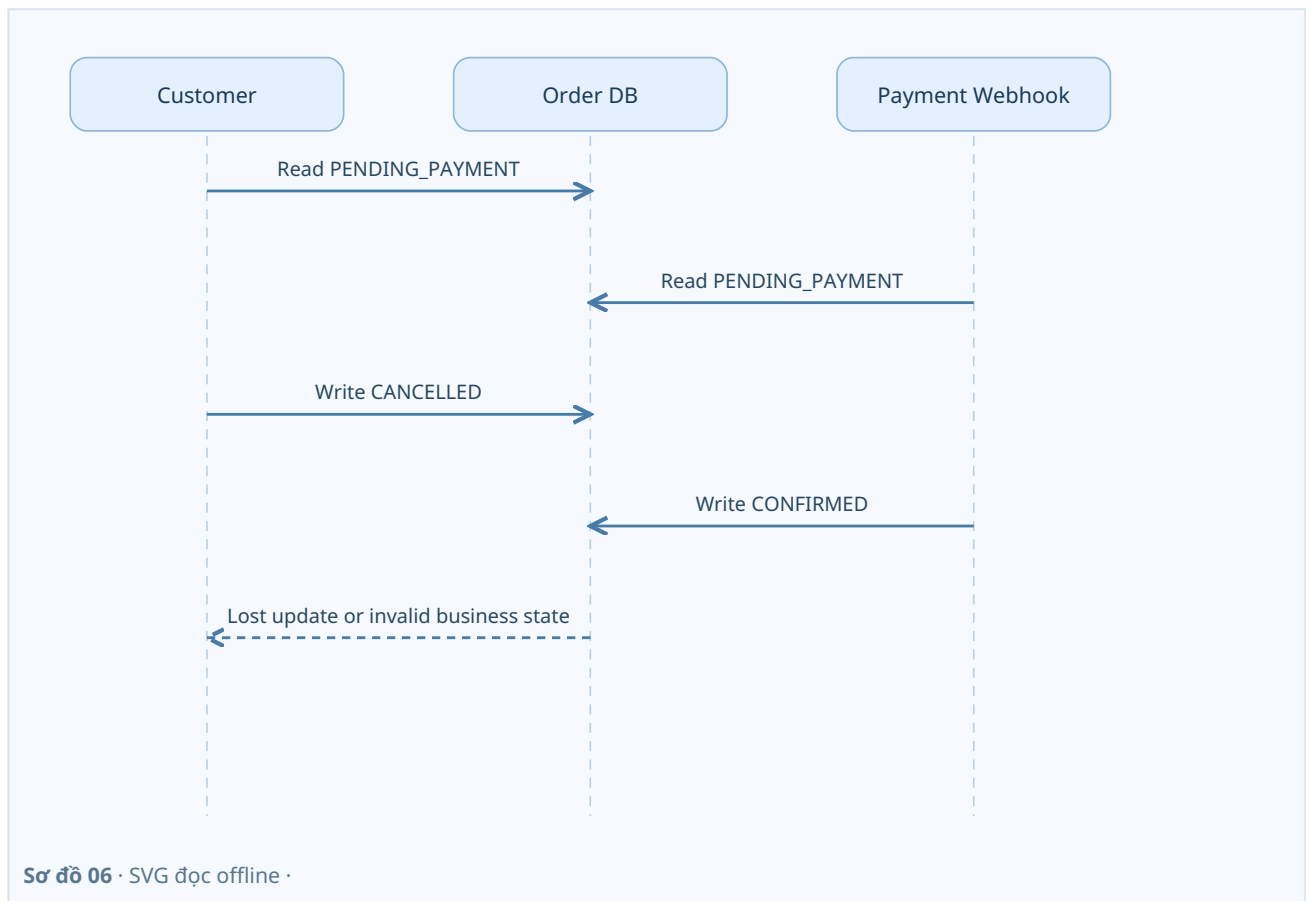
Chữ ký/cơ chế xác minh webhook là trách nhiệm riêng; không dùng request body tự khai `paymentStatus=PAID` để xác nhận tiền. Quyền actor phải được gắn vào command: customer được *yêu cầu hủy*, warehouse được *xác nhận đóng gói*, carrier/đầu tích hợp đáng tin được *cập nhật vận chuyển*.

## 5.7. Race Condition 1: Cancel Order và Payment Succeeded cùng đến

---

Tình huống hay xuất hiện:

- T0: Order `PENDING_PAYMENT`; PSP đang xử lý.
- T1: Khách nhấn Cancel và request A đọc trạng thái.
- T2: Webhook `payment.succeeded` và request B cũng đọc trạng thái.
- T3: A set `CANCELLED`; B set `CONFIRMED`; transaction nào commit cuối cùng thắng nếu không có bảo vệ.



Cách bảo vệ tầng DB: **compare-and-set (CAS)** theo `status`, hoặc optimistic locking bằng `version`, kết hợp guard nghiệp vụ. Ví dụ:

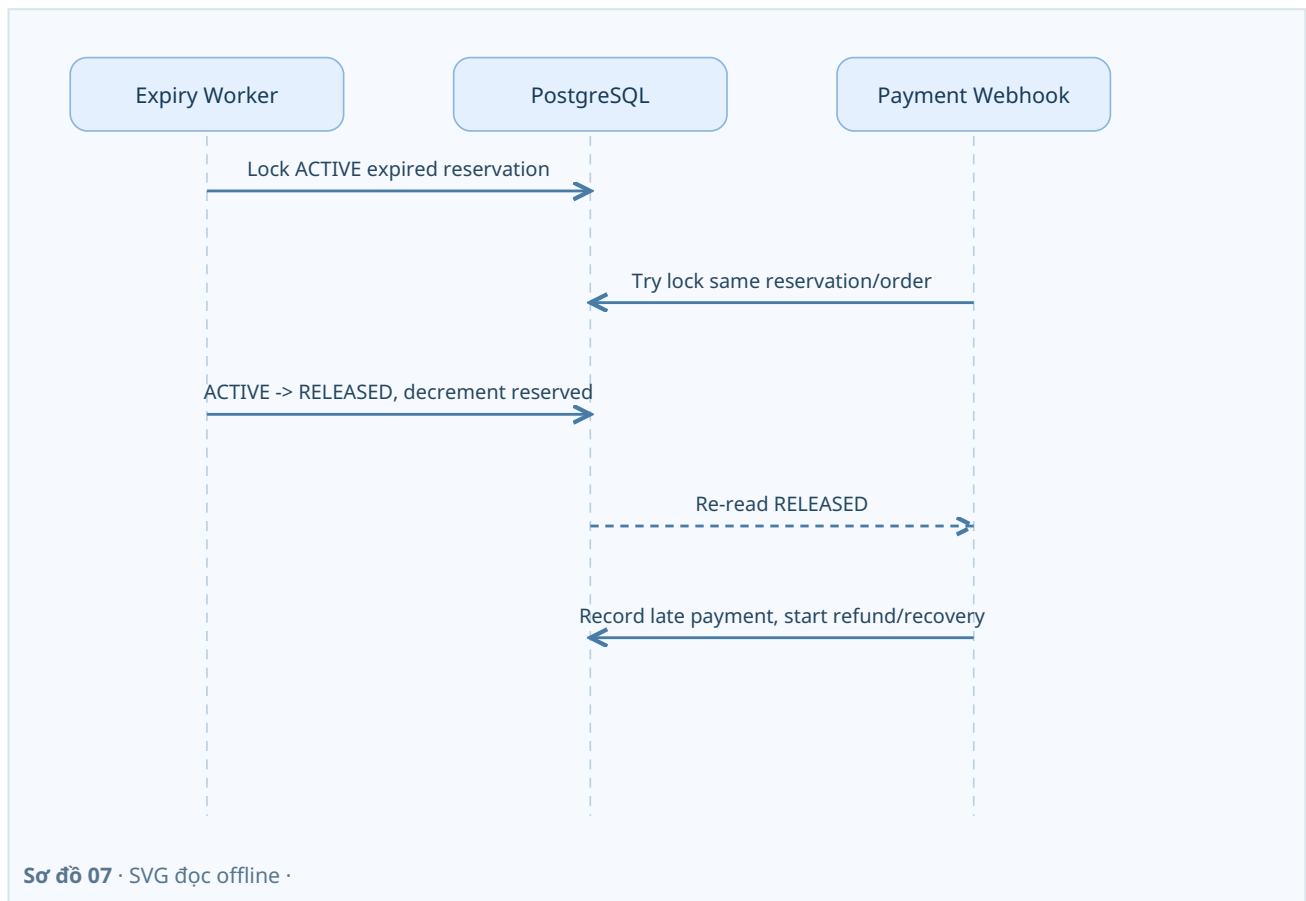
```
UPDATE orders
SET status = 'CANCELLING', version = version + 1
WHERE id = :order_id
  AND status = 'PENDING_PAYMENT'
  AND version = :expected_version;
```

`rows_updated = 0` có nghĩa trạng thái đã thay đổi hoặc version không còn phù hợp: **reload và quyết định lại**; không phải cứ retry cùng cập nhật cho đến khi thành. Với payment success đến sau khi order chuyển `CANCELLING`, handler phải xử lý nhánh late-payment theo policy; không ghi `CONFIRMED` bằng mọi giá.

**Chú ý ranh giới:** CAS trên order row ngăn lost update nhưng *không* tự đảm bảo side effect từ hệ thống bên ngoài chạy đúng một lần. Idempotency và reconciliation vẫn cần.

## 5.8. Race Condition 2: Reservation Expiry vs Payment Confirmation

Stock reservation có thể có `expires_at`. Worker tìm các reservation đã hết hạn để giải phóng. Cùng lúc webhook báo payment success.



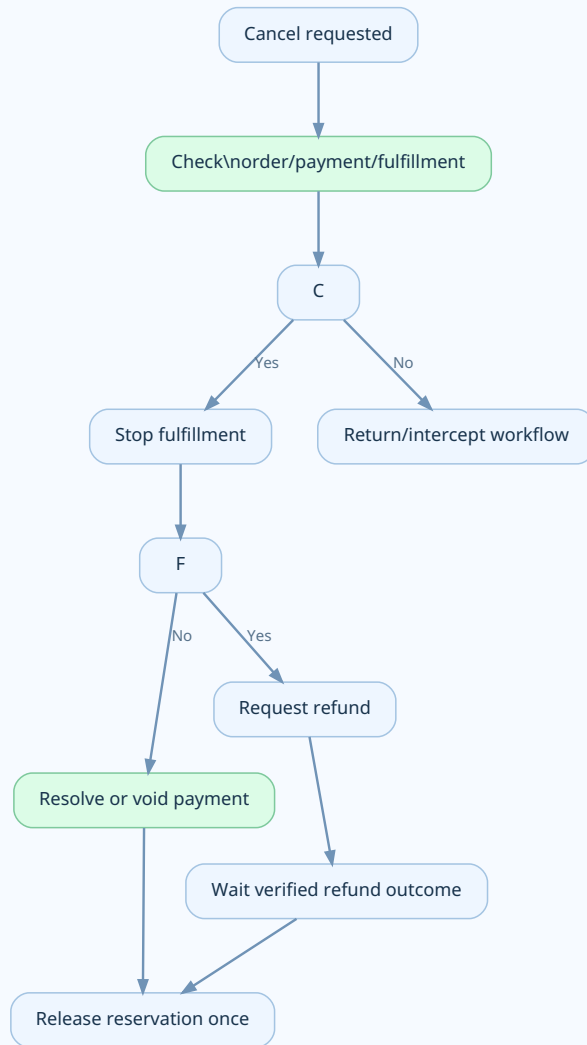
**Không được diễn giải expiry timestamp là quyền trừ kho tự do.** Cần điều kiện trạng thái + transaction + xác minh thanh toán đang xử lý và quy tắc xử lý payment trễ.

Một số hệ thống đặt expiry worker vào trạng thái `EXPIRING / CANCELLING` trước, sau đó đối soát PSP, rồi mới commit trạng thái release terminal. Việc chọn strategy phụ thuộc payment method và SLA. Với PSP có thể gửi success muộn, tự động release hàng chỉ vì deadline mà không xử lý late-success sẽ tạo ra paid-without-stock.

## 5.9. Cancellation là một workflow, không phải một lệnh set status

Phân biệt:

| Mốc                               | Nghịệp vụ        | Cách xử lý có thể cần                                                     |
|-----------------------------------|------------------|---------------------------------------------------------------------------|
| Chưa thanh toán, chưa fulfillment | Request hủy      | Resolve pending PSP, release reservation                                  |
| Đã thu tiền, chưa đóng gói        | Request hủy      | Dừng fulfillment, refund theo rule                                        |
| Đang đóng gói                     | Request hủy      | Kho có dừng được không? Nếu không thì đợi handover/return workflow        |
| Đã giao cho carrier               | Request hủy      | Có thể cần intercept hoặc return-to-origin, không phải đơn thuần rollback |
| Đã giao thành công                | Request trả hàng | Return Authorization + Refund process                                     |



Sơ đồ 08 · SVG đọc offline ·

Trong các sản phẩm có capture trước fulfilment, **CANCELLED** có thể nghĩa là **không giao hàng nữa**, còn refund đang **PENDING/FAILED** phải thể hiện độc lập. Không được quảng bá “đã hoàn tiền” chỉ vì đã gửi API refund mà chưa xác nhận kết quả.

## Policy matrix là dữ liệu kinh doanh

Ví dụ cấu hình (chỉ minh họa):

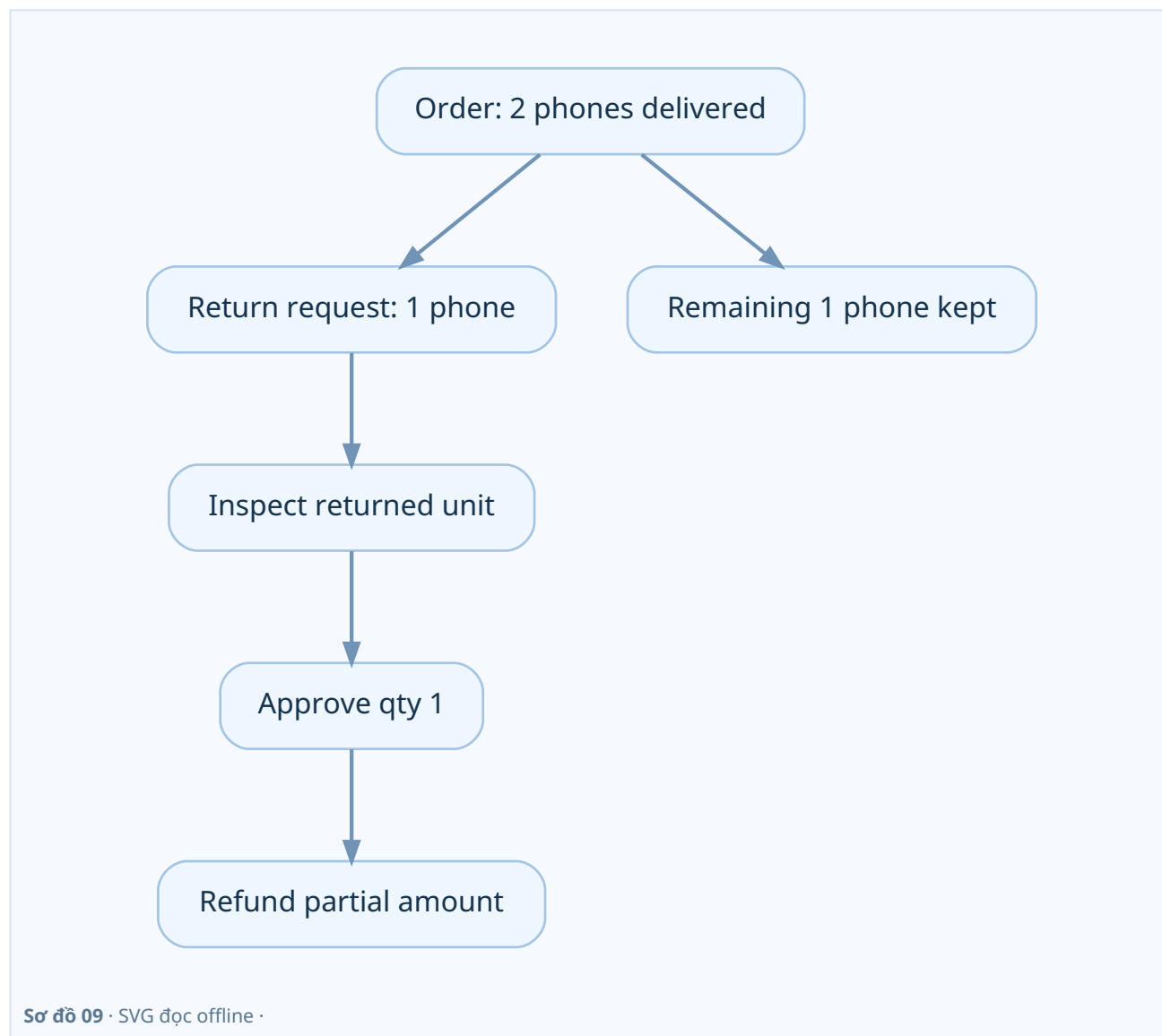
```

cancellationPolicy:
  beforePacked: allow
  afterHandover: requireReturnFlow
  paymentUnknown: waitForVerification
  customerRefundOnSuccessfulCancel: required
  
```

Khi policy thay đổi, nên lưu policy version hoặc đủ context để giải thích tại sao một đơn được hủy tại thời điểm đó.

## 5.10. Return ≠ Cancellation; Refund ≠ Order State

Đơn giao hai điện thoại, khách chỉ trả một chiếc. Hệ thống cần line-level quantity, return authorization và refund amount độc lập:



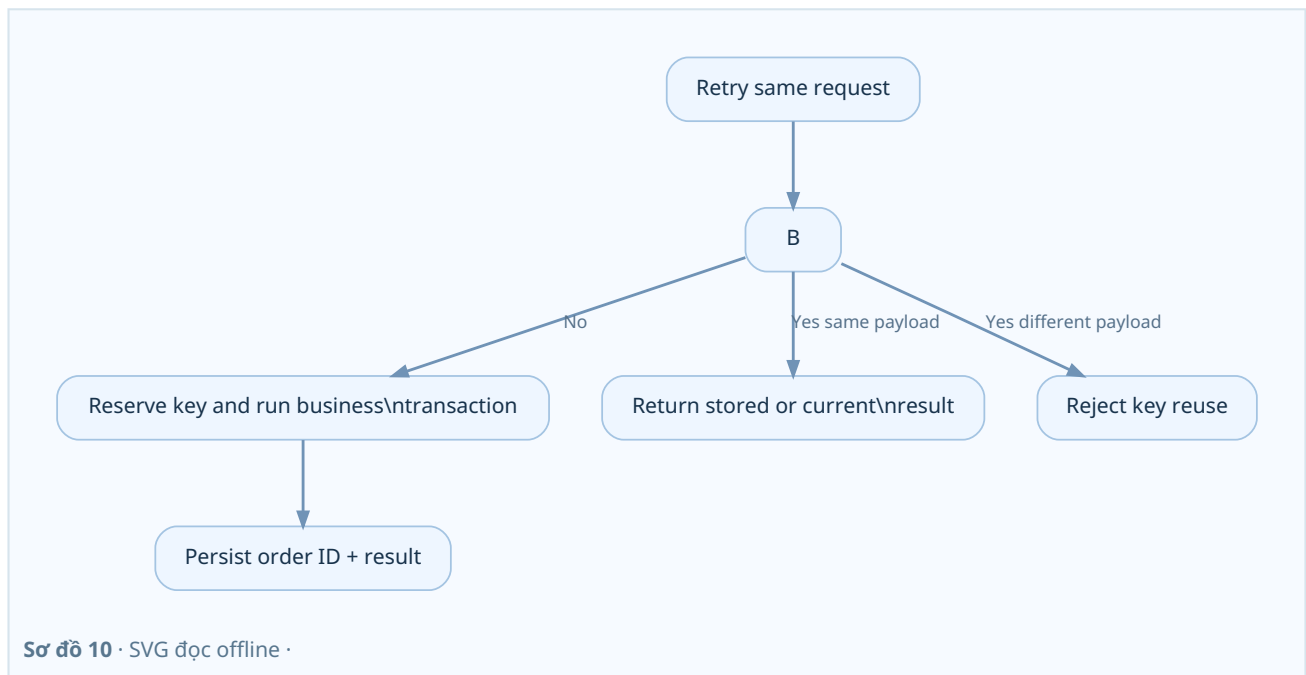
Invariant ở cấp dòng: `returned_qty + non_returnable_or_other_allocations <= fulfilled_qty` theo chính sách; refunded monetary sum không vượt phần có thể hoàn của giao dịch sau khi trừ lần hoàn trước. Shipping, vouchers và phí hoàn có thể phải phân bổ theo rule, không suy trực tiếp `unit_price × qty`.

Trong Marketplace, return/refund cần liên hệ `seller_order_id` và `shipment_id` đúng. Một marketplace checkout có thể có một payment tổng nhưng nhiều seller settlement; refund chỉ một seller có thể cần điều chỉnh commission/payable liên quan.

## 5.11. Idempotency ở bốn lớp, không chỉ ở nút Mua

Một retry có thể đến từ user, load balancer, Worker, broker hoặc PSP. Bốn lớp quan trọng:

| Lớp                  | Duplicate risk                      | Dedupe key/điều kiện                                                              |
|----------------------|-------------------------------------|-----------------------------------------------------------------------------------|
| Checkout request     | Tạo hai order                       | <code>(user_id, idempotency_key)</code> + request fingerprint                     |
| Message consumer     | Chạy business action hai lần        | <code>processed_messages(consumer, event_id)</code> unique + transaction          |
| Payment webhook      | Đánh dấu thanh toán hai lần         | <code>provider_event_id</code> hoặc provider payment unique; verify signature     |
| Stock release/refund | Cộng tồn kho hoặc hoàn tiền hai lần | Reservation terminal CAS; refund idempotency key + provider transaction reference |



Idempotency ở API của PSP giúp retry thao tác phía PSP, nhưng **không thay thế** idempotency tại API và database của bạn. Tài liệu Stripe mô tả idempotency key cho việc retry mà không tạo thao tác trùng, kèm giới hạn lưu kết quả phụ thuộc provider: xem tài liệu tham khảo.

## 5.12. Crash giữa Commit và Publish: Transactional Outbox

Bad case:

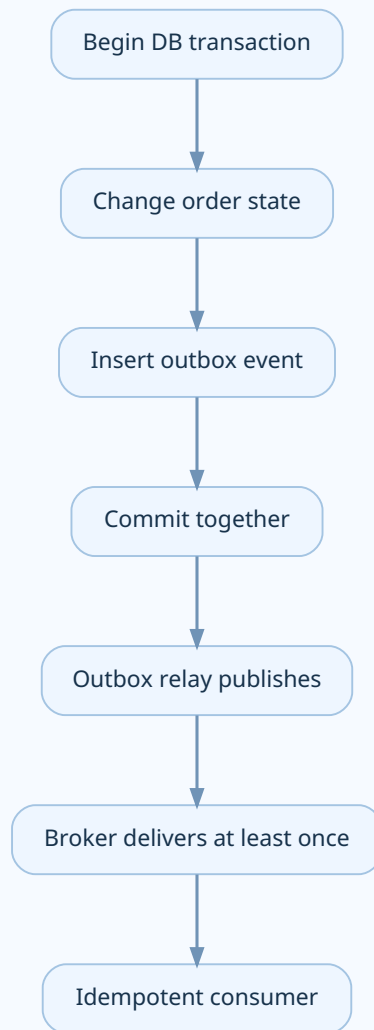
```

saveOrder();           // DB commit succeeded
rabbitTemplate.send(); // JVM crashes before this line

```

Database đã có đơn nhưng Inventory/Fulfillment không bao giờ nhận `OrderPlaced`.

Đảo thứ tự cũng nguy hiểm: message phát đi nhưng transaction tạo đơn rollback.



Sơ đồ 11 · SVG đọc offline ·

Trong cùng DB transaction, `orders` và `outbox_events` cùng commit hoặc rollback. Relay gửi message ngoài transaction rồi đánh dấu published. Relay vẫn có thể crash sau khi publish nhưng trước khi đánh dấu, dẫn đến **duplicate event**. Vì vậy consumer phải idempotent; outbox không đảm bảo exactly-once end-to-end một cách thần kỳ.

AWS Prescriptive Guidance mô tả outbox như cách xử lý dual write và lưu ý duplicate/order concerns. Xem phần tài liệu tham khảo.

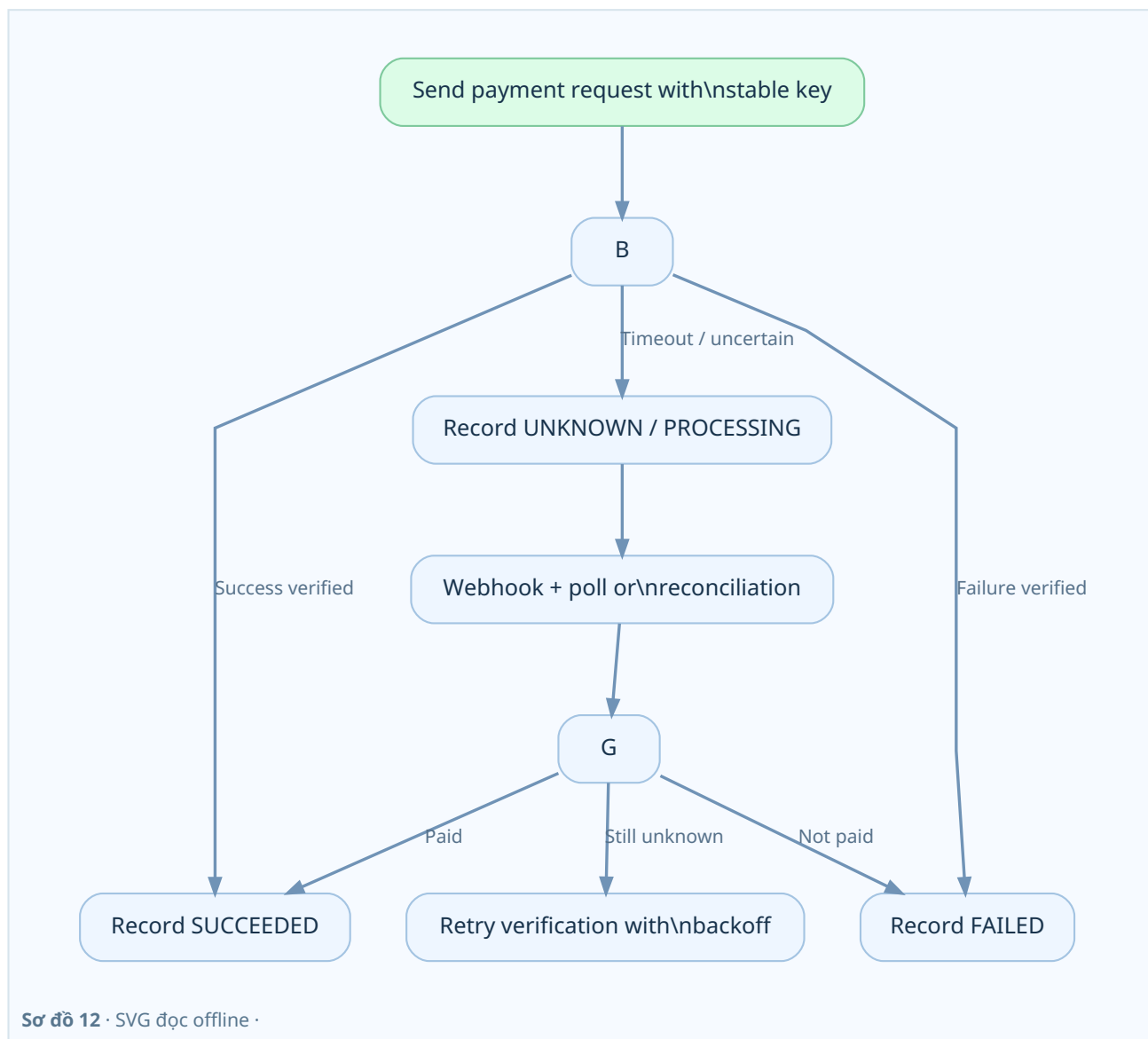
## 5.13. Payment timeout: UNKNOWN là thông tin, không phải sự thất bại

Bad case:

HTTP to PSP → timeout → set `PAYMENT_FAILED` → release stock

Nếu PSP đã thu tiền thành công nhưng response mất trên mạng thì bạn tạo trường hợp paid-without-order/stock.

Quy trình tốt hơn:

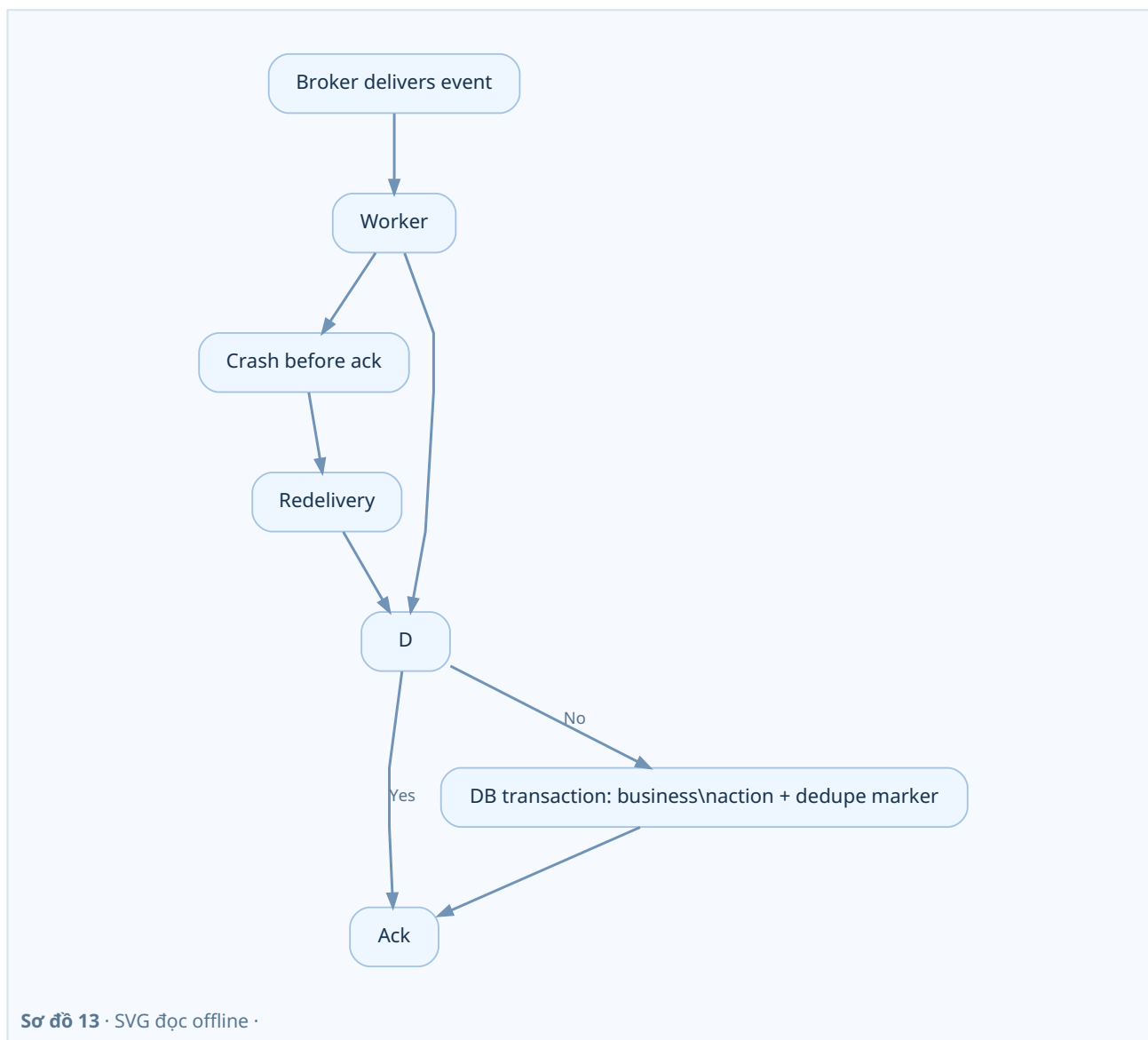


PSP webhooks là tín hiệu bên ngoài cần xác minh chữ ký và đối chiếu `payment reference`, số tiền, tiền tệ và đơn hàng. Có thể nhận duplicate hoặc sự kiện không theo thứ tự; đừng lấy thời gian nhận webhook làm thứ tự chân lý tuyệt đối. Đọc thêm Stripe PaymentIntents lifecycle và event handling tài liệu cuối chương.

**Tách triệt để:** `order_status`, `payment_status`, `refund_status`. Nếu `payment_status=SUCCEEDED` nhưng `order_status=CANCELLING`, system tiếp tục business workflow được khai báo, không mất một trong hai sự thật.

## 5.14. Worker crash, queue backlog và recovery design

Ví dụ Worker nhận `OrderConfirmed` rồi crash sau khi gọi kho nhưng trước khi ack broker. Broker redeliver; nếu không có dedupe, fulfilment tạo hai lần. Workflow an toàn phải nhận diện: order ID, fulfillment unique reference, processed message và command idempotency.



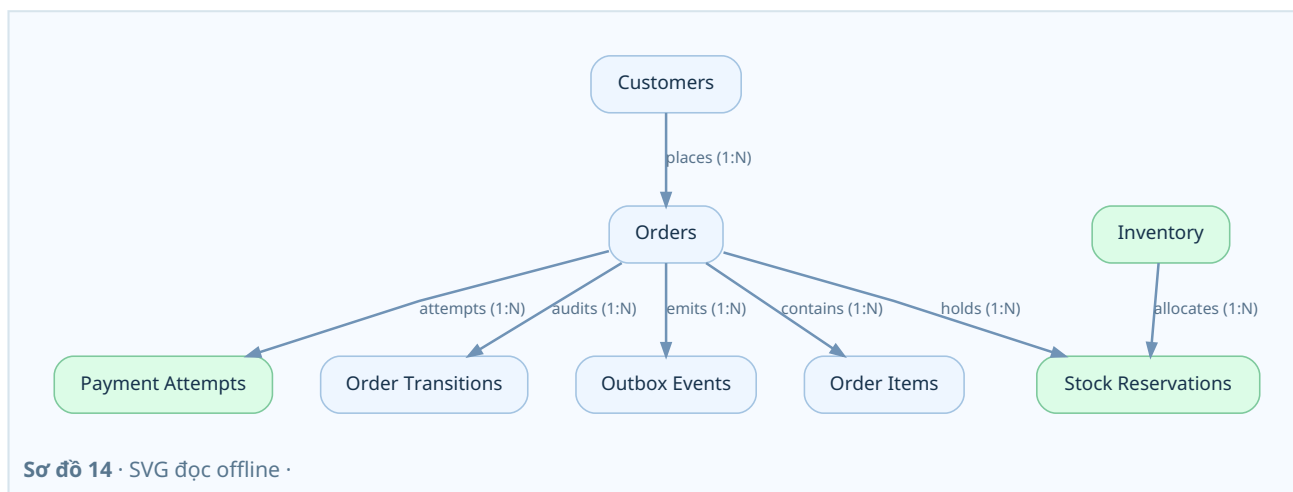
Các loại lỗi nên phân biệt:

- **Transient:** DB failover ngắn, network timeout → bounded retry với backoff/jitter.
- **Permanent business:** order terminal không hợp lệ, out of stock → ghi kết quả nghiệp vụ, không retry vô hạn.
- **Poison message:** dữ liệu sai schema hoặc bug → DLQ/quarantine + alert.
- **Ambiguous external effect:** refund request timeout → tra trạng thái provider trước khi lặp lại cùng idempotency key.

Tránh `catch(Exception e) { retry(); }` vô hạn gây retry storm. Worker concurrency cần giới hạn độc lập với HTTP API replica; queue length không phải chỉ số duy nhất, phải theo dõi **oldest message age**.

## 5.15. Database schema: thiết kế đủ để giải thích một đơn hàng

Nhóm bảng tối thiểu cho chapter lab:



**Tránh mô hình sai:** Không dùng `order_id` làm khóa duy nhất của `payment attempts` nếu một đơn được phép retry thanh toán. Không gộp trạng thái refund vào payment cũ mà mất refund transaction ID. `Order` có thể kết thúc nghiệp vụ bán trong khi return/refund vẫn đang mở.

Trong SQL Lab, bảng `orders` có `status`, `version`, `total_amount`, `currency`, `request_fingerprint`, `idempotency_key`; `order_items` lưu `unit_price` snapshot; `inventory` dùng `on_hand` và `reserved`; `stock_reservations` lưu terminal status riêng; `payment_attempts` có unique provider reference; `order_transitions` giữ audit; `outbox_events` giữ event bền vững.

## Các mẫu constraint quan trọng

```
CHECK (on_hand >= 0 AND reserved >= 0 AND reserved <= on_hand)
```

```
UNIQUE (user_id, idempotency_key)
```

```
UNIQUE (provider, provider_payment_id)
```

Các constraint này là tuyến phòng thủ cuối, không thay thế việc mô hình hóa nghiệp vụ trong application service.

## 5.16. Spring Boot design: command handler + state guards + local transaction

Một module đơn giản:

```
order/
  api/
    OrderController.java
  application/
    CheckoutService.java
    CancelOrderHandler.java
    ConfirmPaymentHandler.java
  domain/
    Order.java
    OrderStatus.java
    OrderPolicy.java
  persistence/
```

```
OrderRepository.java
OrderTransitionRepository.java
messaging/
  OutboxRelay.java
```

Pseudo-code quan trọng nhất của cancel handler:

```
@Transactional
public CancelResult requestCancel(long orderId, long actorId) {
    Order order = orderRepository.lockById(orderId);
    authz.assertCanCancel(actorId, order);

    if (order.isCancelled()) return CancelResult.alreadyCancelled(orderId);
    if (!policy.canRequestCancel(order)) throw new BusinessConflict("NOT_CANCELABLE");

    order.requestCancellation(); // validated PENDING PAYMENT/CONFIRMED -> CANCELLING
    transitions.append(orderId, "CancelRequested", actorId);
    outbox.append(orderId, "CancelRequested", order.version());
    return CancelResult.accepted(orderId);
}
```

Đây là **architectural pseudocode**, không phải class Java được biên dịch sẵn: repository/authz/policy/outbox là abstraction cần implement. `lockById()` trong cùng transaction có thể dùng `SELECT ... FOR UPDATE`; nếu dùng optimistic locking, phải kiểm tra affected rows hoặc version conflict rồi quyết định lại theo state hiện tại. Không dùng lock rồi đi gọi PSP trong transaction.

## Spring event không mặc định là reliable message broker

Gọi `applicationEventPublisher.publishEvent(...)` trong process không đồng nghĩa với sự kiện đã được lưu bền vững hoặc consumer khác nhận chắc chắn. Nếu event thúc đẩy bước nghiệp vụ quan trọng, cần thiết kế độ tin cậy phù hợp (outbox/retry/dedupe).

## 5.17. Observability: đo một lifecycle chứ không chỉ đo HTTP 200

Chỉ nhìn latency `POST /orders` là thiếu. Các metric có ý nghĩa nghiệp vụ:

| Metric                                        | Giúp trả lời                                      |
|-----------------------------------------------|---------------------------------------------------|
| <code>checkout_success_rate</code>            | Bao nhiêu request hợp lệ tạo đơn thành công?      |
| <code>orders_pending_payment_age</code>       | Đơn chờ tiền lâu bao nhiêu?                       |
| <code>payment_unknown_count</code>            | Bao nhiêu thanh toán đang mơ hồ?                  |
| <code>reservation_expired_count</code>        | Bao nhiêu giữ hàng hết hạn?                       |
| <code>refund_pending_age</code>               | Tiền hoàn đang bị chậm bao lâu?                   |
| <code>invalid_transition_total</code>         | Có bao nhiêu command/event trái trạng thái?       |
| <code>outbox_oldest_unpublished_age</code>    | Sự kiện quan trọng bị kẹt bao lâu?                |
| <code>broker_oldest_message_age</code>        | Worker có đang theo kịp traffic?                  |
| <code>inventory_reserved_vs_active_sum</code> | Reserved của kho có khớp tổng reservation ACTIVE? |

Đừng đưa `user_email`, địa chỉ nhận, số thẻ hoặc raw PSP secrets vào label metrics/logs. Sử dụng correlation ID/order ID có kiểm soát, masking và phân quyền đọc log.

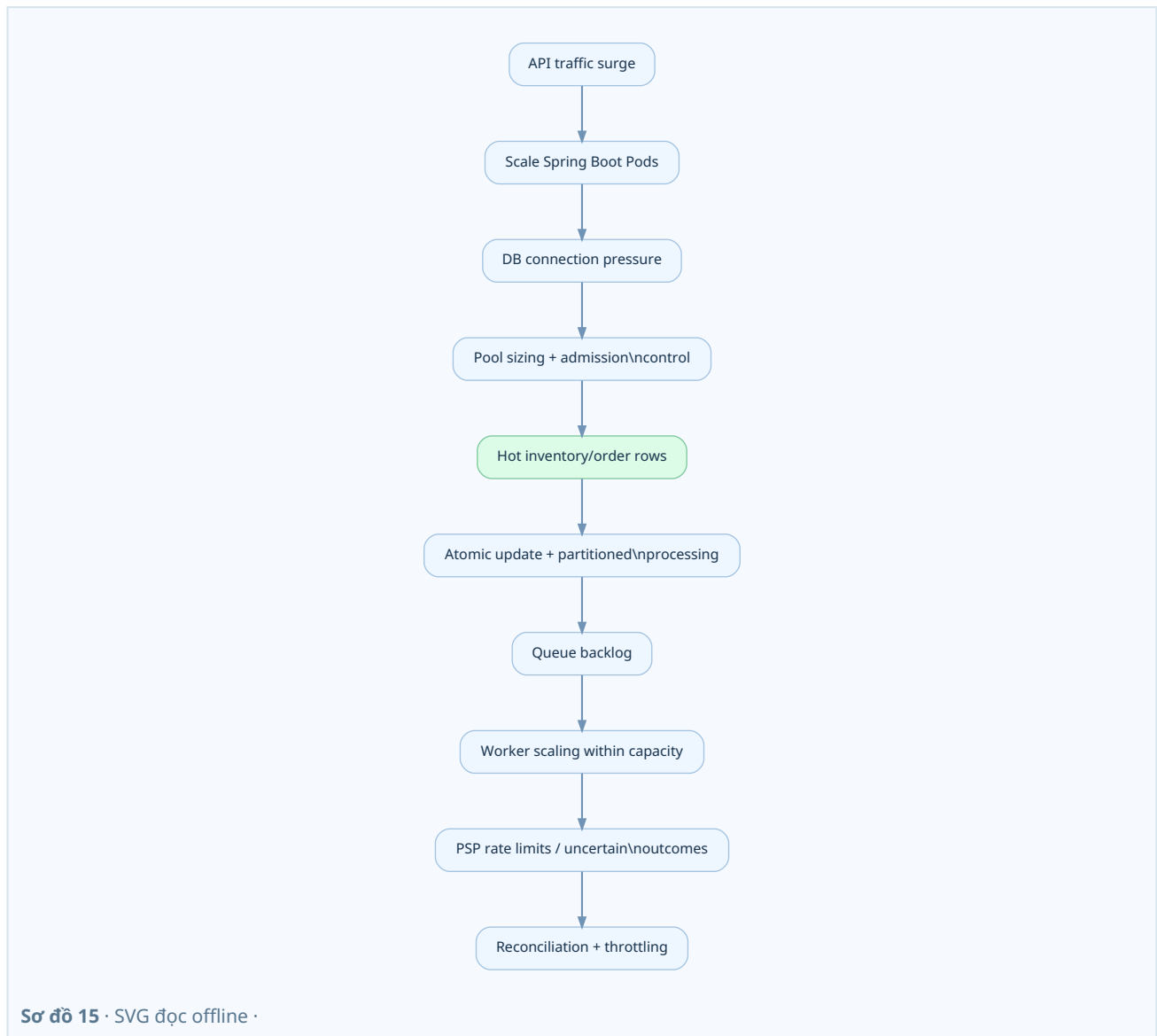
## Reconciliation jobs

- Payment provider **SUCCEEDED** nhưng local order chưa **CONFIRMED**/chưa có case xử lý.
- Order **CANCELLING** quá SLA; phát hiện PSP vẫn đang xử lý.
- Reservation **ACTIVE** quá hạn nhưng chưa có decision.
- `inventory.reserved` khác tổng `stock.reservations` **ACTIVE**.
- `outbox_events` chưa publish quá ngưỡng.
- Refund provider đã thành công nhưng local record còn pending.

**Reconciliation không phải dấu hiệu thiết kế kém:** trong hệ thống có external services và trạng thái bất đồng bộ, nó là lớp kiểm chứng và phục hồi thiết yếu.

## 5.18. Bottleneck Shifting trong Order Lifecycle

Cùng một thứ tự tư duy của cuốn sách:



Với Flash Sale, “scale mọi thứ lên 100 Pod” không khiến một SKU có thêm hàng hay PSP có thêm hạn mức. Nếu giữ transaction mở dài, nhiều Worker có thể chỉ tạo thêm hàng đợi lock. Khi mở rộng, cần đo chính xác **bottleneck hiện tại** trước khi tăng replica.

## 5.19. Engineering Lab — diễn tập một vòng đời đơn

Bộ file trong `labs/chapter-05/` gồm `schema.sql`, `seed.sql`, `queries.sql`, `test-scenarios.md` và `README.md`.

### Thiết lập

1. Tạo PostgreSQL DB thí nghiệm riêng.
2. Chạy `schema.sql`, sau đó `seed.sql` bằng `psql`.
3. Chạy từng transaction có chú thích trong `queries.sql`, nhất là **reserve once** và **release once**.
4. Mở hai session để mô phỏng cạnh tranh `CancelRequested` vs `PaymentVerified`.
5. Thử duplicate `idempotency_key`, duplicate `provider_event_id` và phát event sai trạng thái.

### Bài toán bắt buộc

**A. Reserve once:** `on_hand=1`, hai checkout đồng thời; nhiều nhất một reservation ACTIVE được commit.

**B. Release once:** hai Worker cùng cancel một order; `reserved` chỉ giảm đúng một đơn vị.

**C. Late payment:** order đã vào `CANCELLING`; callback success tới sau; không hồi sinh `CANCELLED` bằng update không guard.

**D. Crash after commit:** order/outbox đã lưu; giả lập relay crash; phát lại event an toàn với dedupe.

**E. Partial return:** đơn có 2 item, chỉ 1 được hoàn; payment/refund ledger khớp theo số tiền thực tế.

### Đáp án tư duy: test không thể thiếu

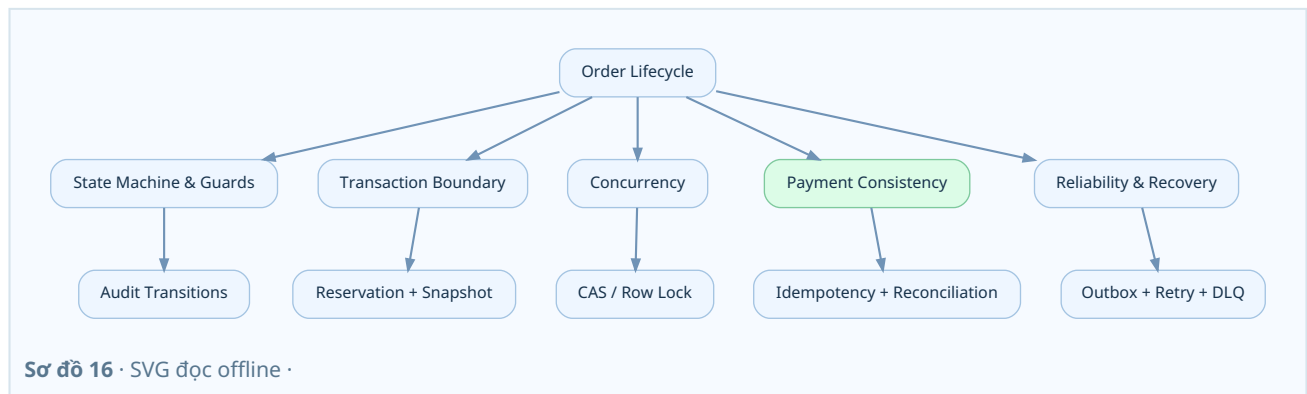
| Tình huống                                             | Kết quả kỳ vọng                                                      |
|--------------------------------------------------------|----------------------------------------------------------------------|
| <code>on_hand=1</code> , 2 concurrent checkout         | Tối đa 1 active reservation                                          |
| Cancel được gửi lại sau thành công                     | Không tăng available lần hai                                         |
| Hai webhook cùng provider event ID                     | Một bản ghi dedupe; một business side effect                         |
| Payment timeout                                        | <code>UNKNOWN / PROCESSING</code> , không tự coi <code>FAILED</code> |
| <code>SHIPPED</code> nhận <code>CancelRequested</code> | Từ chối simple cancellation hoặc chuyển workflow phù hợp             |
| Relay publish rồi crash                                | Event có thể redeliver, consumer không tạo double effect             |

## 5.20. Chapter Review — tư duy trước khi chọn công nghệ

1. Nếu order đã `CANCELLED` nhưng PSP báo `SUCCEEDED`, tại sao `UPDATE orders SET status='CONFIRMED'` là sai?
2. Vì sao cần phân biệt cancel request với cancelled fact?
3. Vì sao unique idempotency key trên order không đủ bảo vệ refund và webhook?
4. Nếu một payment đã thu tiền thành công còn fulfillment chưa diễn ra, chúng ta có thể hoàn stock trước khi resolve refund không? Rule nào chi phối?
5. CAS status/version giải quyết được loại race nào; loại external failure nào nó không giải quyết?
6. Khi nào local transaction giải quyết được consistency? Khi nào cần outbox và Saga?
7. Nếu `broker_oldest_message_age` tăng trong khi API latency thấp, điểm nghẽn nằm ở đâu?

8. Nếu một checkout chứa hai seller, nên tạo một order hay order group + seller orders? Vì sao?

## Knowledge Map



## Key Takeaways

- Order status không thay thế payment, reservation, fulfillment, return và refund state.
- Một state transition = guard + atomic state change + audit + reliable side effects.
- Không định nghĩa timeout thành business failure, nhất là với payment.
- Thao tác lặp phải không gây side effect trùng; terminal inventory release là one-way.
- Local transaction đúng lúc, ngắn; external calls chạy ngoài transaction.
- Outbox/consumer dedupe giúp phục hồi khi process chết, không làm event tự nhiên exactly-once.
- Bottleneck tiếp theo có thể là hot row, queue, PSP hay reconciliation, chứ không chỉ database CPU.

## Tài liệu tham khảo kỹ thuật

- PostgreSQL Documentation, *Explicit Locking*, row-level locking / transaction scope: <https://www.postgresql.org/docs/current/explicit-locking.html>
- AWS Prescriptive Guidance, *Transactional outbox pattern*: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html>
- AWS Prescriptive Guidance, *Saga choreography pattern*: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-choreography.html>
- Stripe API Reference, *Idempotent requests*: [https://docs.stripe.com/api/idempotent\\_requests](https://docs.stripe.com/api/idempotent_requests)
- Stripe API Reference, *PaymentIntents and their lifecycle*: [https://docs.stripe.com/api/payment\\_intents](https://docs.stripe.com/api/payment_intents)
- Stripe Docs, *Webhooks*: <https://docs.stripe.com/webhooks>

**Đóng Volume 1:** Chương 01–05 trả lời lần lượt: E-commerce có những luồng nghiệp vụ gì; ai sở hữu các luồng đó trong từng business model; domain được phân ranh ra sao; giá được xác định/chốt như thế nào; và cuối cùng một order được quản lý đúng qua mọi thay đổi/ngoại lệ. **Volume 2** sẽ bắt đầu từ Modular Monolith và schema/API implementation, không cần nhảy thẳng lên microservices.